



**Politecnico
di Torino**

Artificial Intelligence Algorithms for Electronic Component Recognition

MASTER DEGREE THESIS IN
DATA SCIENCE AND ENGINEERING

Author: **Giovanni Cadau**

Student ID: S304861

Advisor: Prof. Daniele Apiletti

Company advisors: Paolo Chiartano, Ing. Francesco Grassino

Academic Year: 2023-24

Abstract

In the challenging scenario of Automatic Electronic Testing of Printed Circuit Boards (PCBs), a crucial role is played by the recognition of specific electronic components, such as resistors, inductors, capacitors, and others.

This task usually requires an extensive amount of manual labor, necessitating the presence of a domain expert. The great heterogeneity of PCBs and the highly variable environmental conditions under which images are collected within a corporate production mechanism also demand high precision and significant time.

The objective of this thesis is to demonstrate the use of artificial intelligence (AI) algorithms, particularly machine learning (ML) and deep learning (DL) ones, to perform the task of image recognition, determining the position and nature of various components within PCBs. Additionally, this thesis aims to integrate the resulting models into a corporate context, bringing efficiency benefits.

Starting from an accurate and extensive collection of images, in order to ensure high data quality, and defining an exhaustive pipeline for preprocessing and augmentation, it was possible to develop ML and DL models that were integrated into the corporate management software. The aim was not only to optimize the accuracy of the models but also to address aspects particularly relevant in a corporate context, such as latency, memory usage, and energy consumption.

The use of Domain Randomization and Adaptive Domain Randomization techniques, combined with the use of optimization algorithms, both gradient-based and gradient-free, allowed for the creation of robust models, capable of handling the variability and complexity of the environment in which the images were collected and might be collected in the near future, without the need to retrain a new model.

Keywords: Automatic Electronic Testing, Electronic Component Recognition, Printed Circuit Boards, Machine Learning Models, Deep Learning Models, Gradient-free optimization algorithms

Contents

Abstract	i
Contents	iii
Summary	1
Introduction	5
1 Context and Rationale	9
1.1 Introduction to Electronic Testing	9
1.2 Printed Circuit Boards	10
1.2.1 Components on a PCB	11
1.2.2 Heterogeneity of PCBs	13
1.3 Specific component recognition on a Printed Circuit Board	16
1.3.1 Limitations of Manual PCB Component Recognition	16
1.3.2 Potential of Artificial Intelligence in PCB Component Recognition .	21
2 Background	23
2.1 Central Role of Data in AI Development	23
2.1.1 Introduction	23
2.1.2 Data Collection	24
2.1.3 Data Preprocessing	26
2.1.4 Data Augmentation	28
2.2 Machine Learning as a Core Component of AI	29
2.2.1 Introduction to Machine Learning	29
2.2.2 Key Machine Learning Algorithms	30
2.2.3 Introduction to Deep Learning	30
2.2.4 Key Deep Learning Architectures	30
2.2.5 Machine Learning algorithms for Image Recognition	31

2.2.5.1	SVM	31
2.2.5.2	Convolutional Neural Networks (CNNs)	31
2.2.5.3	Residual Networks (ResNet)	32
2.3	Optimization Algorithms in AI	33
2.3.1	Gradient-Based Optimization Algorithms	34
2.3.2	Gradient-Free Optimization Algorithm	36
3	Model Design and Architecture	39
3.1	Data collection	39
3.1.1	Data sources	39
3.2	Data Preprocessing	41
3.2.1	Performance Optimization	41
3.2.1.1	Caching	41
3.2.1.2	Shuffling	41
3.2.1.3	Batching	41
3.2.1.4	Prefetching	41
3.2.2	Data splitting	43
3.2.3	Data Transformations	43
3.2.3.1	Image Resizing	43
3.2.3.2	Image Normalization	43
3.3	Model architecture	44
3.3.0.1	SVM	44
3.3.0.2	CNN	45
3.3.0.3	ResNet	47
4	Adaptive Domain Randomization	51
4.1	Domain Randomization	51
4.1.1	Parameters to be randomized	51
4.1.2	How Parameters are Randomized	52
4.1.2.1	Parameter distributions	53
4.2	Adaptive Domain Randomization	54
4.2.1	Challenges in Choosing Randomization Coefficients	54
4.2.2	Coefficient Optimization	55
4.2.2.1	Implementation	55
4.2.2.2	Adaptive Domain Randomization advantages	56
5	Application of the Model	59
5.1	Image Classification	60

5.2	Object Detection	60
5.2.1	Adaptive Sliding Window Sizes	62
5.3	Appendix: Optimizing Inference time for Large-Scale Image Processing . .	63
5.3.1	Batch Processing	63
5.3.2	Parallelism	64
5.3.2.1	Choosing the Right Approach	65
6	Experimental setup and Results	67
6.1	Training and Evaluation	67
6.2	Model comparison	69
6.3	Adaptive Domain Randomization for Model Robustness	73
7	Performance Optimization of Deployed Models in Industrial Systems	77
7.1	Details of the Parameters to be Optimized	77
7.1.1	Distinction Between Model Size and Memory Usage	78
7.2	Details of the Techniques for Model Performance Optimization	79
7.2.1	Data reuse maximization	80
7.2.1.1	Fully connected layers	80
7.2.1.2	Convolutional layers	82
7.2.2	Deep Neural Networks' tolerance to approximation exploitation . .	83
7.2.2.1	Use of Efficient Layers	84
7.2.2.2	Pruning	88
7.2.2.3	Quantization	95
7.3	Evaluation of Optimization Techniques in Real-World Hardware Scenarios for Performance Optimization	108
8	Conclusions and future developments	115
8.1	Future Developments	115
	Bibliography	117
A	Parameter Distributions for Domain Randomization	121
A.1	Univariate Uniform distributions	121
A.2	Univariate Triangular distribution	121
A.3	Univariate Normal distribution	121
A.4	Multivariate Normal distribution	122

B	Probability vector $\mathbf{p}$ for (Adaptive) Domain Randomization	123
C	Parameter Distributions for Adaptive Domain Randomization	125
C.1	Univariate Uniform distributions	125
C.1.1	$\mathcal{U}(a, b)$: Lower Ranges, Upper Ranges, and Initial Values	126
C.2	Univariate Triangular distributions	126
C.2.1	$\mathcal{T}(a, m, b)$: Lower Ranges, Upper Ranges, and Initial Values	127
C.3	Univariate Normal Distribution	127
C.3.1	$\mathcal{N}(\mu, \sigma^2)$: Lower Ranges, Upper Ranges, and Initial Values	128
C.4	Multivariate Normal Distribution	128
C.4.1	$\mathcal{N}_k(\mu, \Sigma)$: Lower Ranges, Upper Ranges, and Initial Values	129
	List of Figures	131
	List of Tables	133
	Acknowledgements	135

Summary

Thesis Objectives

The primary objective of this thesis is to investigate the application of artificial intelligence (AI) techniques, specifically machine learning (ML) and deep learning (DL), for the recognition of electronic components on printed circuit boards (PCBs). The goal is to automate the traditionally manual process of identifying and classifying components, such as resistors, capacitors, and inductors, on PCBs, which are crucial in ensuring the functionality and quality of modern electronic devices. This automation seeks to address the limitations of manual inspection, which is labor-intensive, time-consuming, and prone to human error, particularly when handling large volumes of PCBs in industrial contexts.

The research focuses on developing a comprehensive image recognition pipeline that can handle the complexity and variability inherent in PCB environments, including challenges posed by different lighting conditions, overlapping components, and varying PCB designs. In addition, the thesis aims to integrate the trained models into an industrial production setting, ensuring that the AI-driven system can meet practical constraints such as inference time, memory usage, and energy efficiency. Furthermore, the development of robust models capable of operating under different environmental conditions without significant degradation in performance is a core objective of this study. These goals are crucial to achieving scalability and adaptability in real-world industrial applications.

The main goals include designing an effective data preprocessing pipeline, developing ML and DL models for automatic recognition, and optimizing these models for industrial environments. A particular emphasis is placed on exploring the balance between high accuracy and computational efficiency, given the constraints of industrial hardware.

Research Area and Contribution

This thesis contributes to multiple domains, including computer vision, machine learning, and electronic component testing. Its main research area lies in the automation of elec-

tronic component recognition on PCBs using AI techniques, addressing a critical aspect of modern electronics manufacturing. The automation of this task, which traditionally relies on human expertise, is essential for improving the speed and accuracy of the inspection process in large-scale production environments.

From a computer vision perspective, the study applies advanced image processing techniques to PCB images, utilizing state-of-the-art architectures such as Convolutional Neural Networks (CNNs) and Residual Networks (ResNets) for object detection and classification. These approaches help address the unique challenges posed by PCBs, such as densely packed components, overlapping parts, and varying environmental conditions. The complexity of PCB designs requires a level of precision in recognizing not only the components themselves but also their orientation and spatial configuration, which CNNs and ResNets are well-suited to handle.

In the field of machine learning, the thesis explores both traditional models like Support Vector Machines (SVMs) and more modern deep learning approaches. SVMs were found to struggle with the high-dimensional, spatially complex nature of PCB images, thus justifying the use of more powerful neural networks. Additionally, domain randomization and adaptive domain randomization techniques are employed to improve model robustness, allowing the system to generalize effectively across different PCB designs and environments. These techniques introduce controlled variability during the training phase, enhancing the model's ability to perform well under diverse real-world conditions.

Moreover, the work emphasizes practical contributions by optimizing the models for use in an industrial context. This includes ensuring that models can be deployed efficiently in terms of memory usage, inference time, and energy consumption, which are all critical factors in industrial-scale applications. Specific techniques employed to achieve these optimizations include maximizing data reuse in fully connected and convolutional layers, as well as exploiting the tolerance of deep neural networks to approximations. This allows for the implementation of efficient layers, which reduce computational costs without significantly compromising accuracy. Additionally, the models have been optimized through pruning, which reduces the number of connections in the network, and quantization, which lowers the precision of weights and activations, reducing memory usage and improving inference speed.

Results

The results of this thesis confirm that AI techniques can significantly automate the recognition of electronic components on PCBs, achieving high accuracy and efficiency. In

particular, deep learning models such as CNNs and ResNets demonstrated superior performance compared to traditional methods like SVMs. These models achieved accuracy rates above 98% on test datasets, highlighting their ability to generalize well across different PCB designs. The improved performance of ResNet-based models was evident in their ability to identify components that are small, partially obscured, or closely packed, addressing some of the most challenging aspects of PCB recognition.

The thesis also shows that the use of domain randomization and adaptive domain randomization greatly improved model robustness. These techniques allowed the models to handle variations in lighting, orientation, and environmental conditions without the need for frequent retraining, which is essential for industrial deployment. By training the models with augmented data that simulates different environmental conditions, the system can better cope with the real-world variability seen in PCB images, thus reducing the need for continuous manual intervention or recalibration.

In terms of industrial optimization, the models were fine-tuned to reduce inference time, memory usage, and energy consumption. These optimizations allowed the models to meet the stringent requirements of real-time PCB testing, demonstrating their suitability for integration into existing production pipelines. Tests conducted within the industrial pipeline showed that the optimized models were able to process PCBs at a rate that aligns with the production demands, offering significant time savings over traditional manual inspection processes, demonstrating their practical viability in such environment, particularly in terms of reducing manual labor and speeding up the testing process.

Finally, an analysis of class confusion patterns provided valuable insights into the model's performance. This analysis highlights the importance of refining the model's feature extraction process and potentially augmenting the dataset to better distinguish between challenging class pairs.

In conclusion, the results validate the use of AI for automating PCB component recognition in industrial settings. The developed models offer significant improvements in terms of efficiency, scalability, and accuracy compared to traditional methods.

Introduction

Machine learning and deep learning have revolutionized numerous fields of technology, including natural language processing, computer vision, and autonomous systems. These advanced techniques enable the development of intelligent systems capable of learning from data, identifying patterns, and making informed decisions with minimal human intervention.

In particular, within the context of electronic testing, the capability to recognize specific components and their positions on a printed circuit board (PCB) has always been of paramount importance. PCBs are essential in modern electronics, providing both mechanical support and electrical connections for electronic components. The accurate identification and testing of these components are crucial for ensuring the functionality and reliability of electronic devices.

Traditionally, this task involves a meticulous manual process where an expert in electronics inspection and testing painstakingly identifies the various components, verifies their presence or absence, and assesses their connections to ensure proper functionality. This manual process is fraught with several significant limitations:

- **Time-Consuming:** the manual inspection and testing of PCBs are extremely time-intensive, often leading to delays in the production and quality assurance stages. Each PCB must be carefully examined to identify components and verify their correct placement and connections, which can take a considerable amount of time.
- **Error-Prone:** human error is inevitable in such detailed and repetitive tasks, potentially leading to incorrect assessments and the need for rework. Mistakes in identifying components or connections can result in faulty products, necessitating costly and time-consuming re-inspections and repairs.
- **Domain Expertise Requirement:** the process demands the involvement of highly skilled experts who possess deep knowledge of electronic components and testing procedures. Such expertise is not only challenging to scale but also costly to maintain, as it requires continuous training and experience in the ever-evolving field of

electronics.

In this context, the introduction of artificial intelligence (AI), particularly through machine learning (ML) and deep learning (DL), offers transformative potential. By automating the recognition and classification of electronic components on PCBs, AI can significantly enhance the efficiency and accuracy of the electronic testing process [5, 25, 41]. Advanced AI models can learn to identify components and their configurations from vast amounts of labeled data, enabling rapid and precise inspection without the constant need for human oversight. This automation can lead to several benefits:

- **Increased Efficiency:** automated systems can perform inspections much faster than humans, reducing the time required for PCB testing and speeding up the overall production process.
- **Improved Accuracy:** AI systems can consistently identify and classify components with high precision, minimizing the likelihood of errors and ensuring higher product quality.
- **Scalability:** AI-driven inspection systems can be easily scaled to handle large volumes of PCBs, accommodating the needs of mass production without the need for proportional increases in human resources.

This need emerged during a collaborative project with Seica S.P.A.¹, an Italian company specialized in design and manufacturing of Automatic Test Equipment (ATE). It has been possible to highlight the critical need for more efficient and accurate methods of electronic component recognition and testing.

In a business context, the accuracy and effectiveness of the implemented models must also be considered in relation to certain metrics that make the implementation of these models within a production mechanism possible (and efficient). These metrics include:

- **Inference time:** the time taken by an AI model to make predictions on new data. For practical applications, it is crucial that the inference time is minimized to ensure rapid inspection and testing processes.
- **Energy consumption:** the amount of power required to run AI models, especially in large-scale or continuous operation scenarios. Efficient models that consume less energy are preferred to reduce operational costs and environmental impact.
- **Memory usage:** the amount of memory required by AI models during both training and inference phases. Models that are memory-efficient can be deployed on devices

¹<https://www.seica.com/it>

with limited resources and can handle larger datasets more effectively.

These factors need to be analyzed both when the models are executed on a local PC and within processes run on machines dedicated to PCB testing, such as those managing the Flying Probe² used by the company. The use of implementations compatible with the programming languages available in these environments, the available memory and the hardware available to perform the tasks are particularly relevant.

The thesis aims to explore the application of machine learning and deep learning techniques in the automated recognition of electronic components on PCBs. It will provide a detailed analysis of the methods employed, including data collection and preprocessing, model training and evaluation, and the potential improvements in the electronic testing domain.

Code Availability

All the code developed and used for this experimental research is available in a GitHub repository, which can be accessed through the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection>.

The different versions of the code can be accessed at the following link: <https://github.com/gcadau/Electronic-Components-AI-Detection/tags>. Each of them aims to address a specific task.

The code of the software implemented into the Company Software Management System to integrate the software is not available, as it belongs to the private data of the company involved in the project and cannot be made public.

Documentation with complete instructions for executing the code is available in various *README* files within the GitHub repository. These documents provide detailed guidance on the following aspects:

Training a Model Instructions for training the machine learning models are provided at the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection/blob/version-7.0/README.md>.

This document includes detailed steps on data preparation, model configuration, and the training process.

²<https://www.seica.com/flying-probe/flying-probe-testers/>

Testing a Model Steps for testing the trained models can be found at the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection/blob/version-8.0/README.md>.

It covers the procedures for validating the model performance, running test cases, and interpreting the results.

Object Detection Detailed instructions for performing object detection are available at the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection/blob/version-10.0/README.md>.

This document provides guidelines on setting up the object detection framework, executing detection tasks, and visualizing results.

Making Predictions Guidance on how to use the models to make predictions is provided at the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection/blob/version-11.0/README.md>.

It includes information on preparing input data, running prediction scripts, and handling output.

Object Detection and Image Classification Interface Instructions for using the interface for object detection and image classification are detailed at the following link:

<https://github.com/gcadau/Electronic-Components-AI-Detection/blob/version-13.0/README.md>.

This document explains how to interact with the user interface, perform classification tasks, and review results.

Trained Models Availability

The trained models to be used for the inference are available at the following link: https://drive.google.com/drive/folders/17It1971hF4U4KuJNMw1g-Z-0sHV_yxoa?usp=sharing.

Inside the repository documentation is also reported the code to download them and a detailed description on the way they have to be loaded and used to make inferences.

1 | Context and Rationale

This chapter introduces the context of electronic testing and the role of PCBs in modern electronics. The chapter also outlines the limitations of traditional manual inspection methods and highlights the potential benefits of integrating artificial intelligence into the process.

1.1. Introduction to Electronic Testing

Electronic testing is an essential process in the production and maintenance of electronic devices. It encompasses various techniques and methodologies used to verify the functionality, performance and reliability of electronic components and assemblies. The primary objective of electronic testing is to ensure that devices operate as intended and meet the required specifications.

One of the main stages of electronic testing occurs during the manufacturing of Printed Circuit Boards (PCBs, see next Section). PCBs serve as the foundation for most electronic devices, providing the necessary mechanical support and electrical connections for components such as resistors, capacitors, inductors, diodes, and integrated circuits. Given the complexity and density of modern PCBs, comprehensive testing is crucial to identify manufacturing defects and ensure quality control.

There are several types of electronic testing methodologies employed throughout the life-cycle of an electronic product:

- **Functional Testing:** This type of testing assesses the overall functionality of the electronic device by simulating normal operating conditions and verifying that the device performs its intended functions correctly [15]. Functional testing is often performed on fully assembled products.
- **In-Circuit Testing (ICT):** ICT involves testing individual components on the PCB while the board is still in the manufacturing process. This method uses test probes to access specific points on the board, checking for correct component placement, orientation, and functionality. ICT is effective for detecting assembly defects

such as open circuits, shorts, and incorrect component values.

- **Automated Optical Inspection (AOI):** AOI systems use high-resolution cameras and image processing algorithms to visually inspect PCBs for defects [29]. This method can quickly identify issues such as soldering defects, missing components, and misalignments. AOI is typically used during the early stages of PCB assembly.
- **Boundary Scan Testing:** This method utilizes the boundary scan standard (IEEE 1149.1) to test the interconnections on PCBs without using physical test probes [3]. It allows for testing of highly complex and densely packed boards, where traditional probing methods may not be feasible.

The rapid advancement of electronic technologies and the increasing complexity of devices have driven the need for more sophisticated and efficient testing methods. Traditional manual inspection methods, while still used, are often insufficient for modern production demands. They are labor-intensive, time-consuming, and prone to human error.

To address these challenges, the industry has increasingly adopted automated testing solutions. Automated Testing Equipment (ATE) and ICT systems have become standard in many manufacturing environments, offering high-speed, high-precision testing capabilities. These systems integrate advanced hardware and software to perform comprehensive testing with minimal human intervention, significantly improving throughput and accuracy.

The development and integration of advanced testing methodologies, including the potential use of artificial intelligence, will play a crucial role in meeting the demands of modern electronics manufacturing.

1.2. Printed Circuit Boards

A PCB, or Printed Circuit Board, is a flat board used in electronics to provide mechanical support and electrical connections for components.

It is typically made from an insulating material, such as fiberglass or composite epoxy, which is coated with copper traces. These copper traces form the electrical circuits that connect various components like resistors, capacitors, and integrated circuits.

PCBs can be single-layer, with copper traces on one side, or multi-layer, with several layers of traces separated by insulating materials. Components are mounted and soldered onto the board, and the design of the PCB is created using specialized software.

Overall, PCBs are essential for organizing and interconnecting electronic components in

a wide range of devices [26].

Figure 1.1 shows a photo of a PCB, collected through the company's Management System Software.

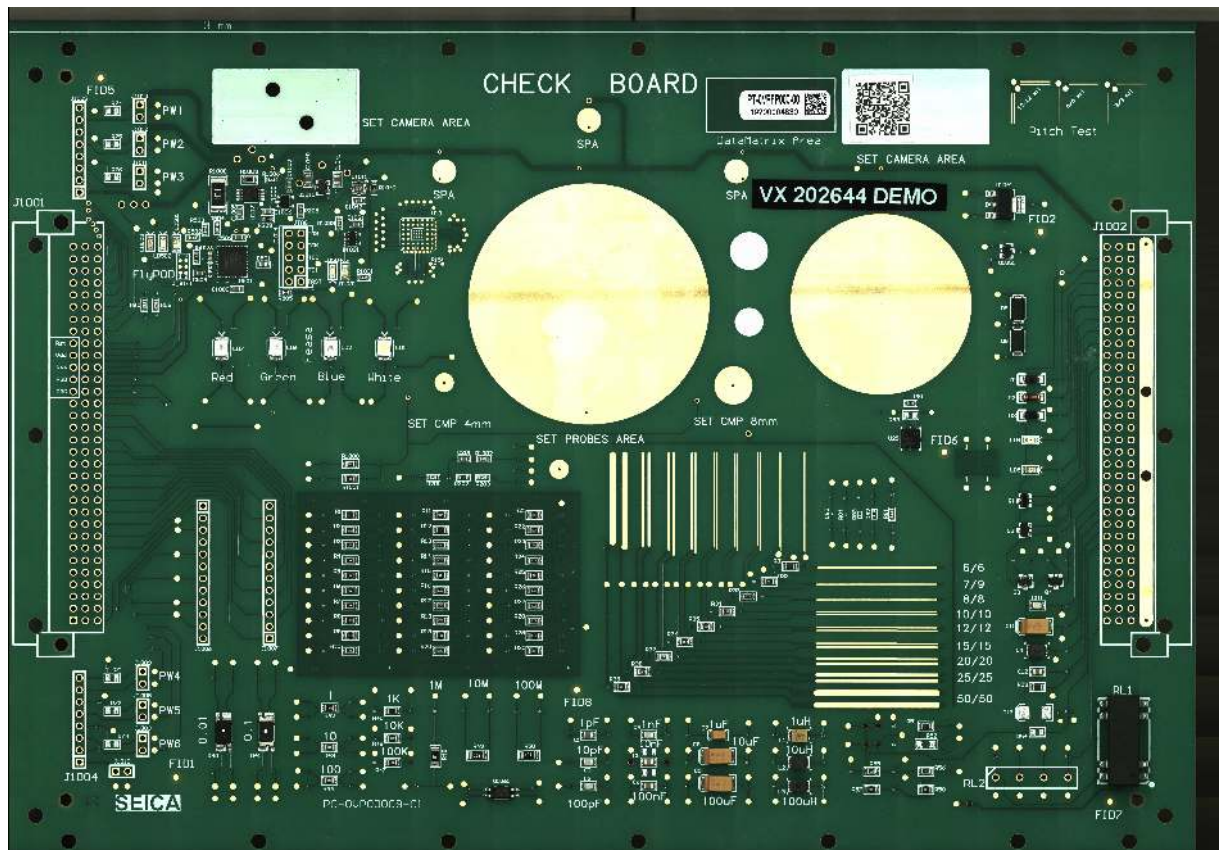


Figure 1.1: High resolution photo of a PCB used in Electronic Testing.

1.2.1. Components on a PCB

Printed Circuit Boards (PCBs) are integral to electronic devices, providing the platform for mounting and interconnecting various electronic components. The components on a PCB can be broadly categorized into passive and active components, each serving specific functions within an electronic circuit. Below is an overview of the primary components typically found on a PCB:

Passive Components

- **Resistors:** Resistors are used to limit current flow and divide voltages within a circuit. They come in various types, including fixed resistors, variable resistors (potentiometers), and specialized resistors like thermistors (temperature-sensitive)

and photoresistors (light-sensitive).

- **Capacitors:** Capacitors store and release electrical energy, playing crucial roles in filtering, timing, and energy storage applications. Types of capacitors include ceramic, electrolytic, tantalum, and film capacitors, each with unique characteristics suited to different applications.
- **Inductors:** Inductors store energy in a magnetic field when electrical current passes through them. They are used in filtering, energy storage, and in applications like transformers and chokes. Inductors can vary in core material, such as air-core, iron-core, and ferrite-core.

Active Components

- **Diodes:** Diodes allow current to flow in one direction only, providing rectification, voltage regulation, and signal demodulation. Common types of diodes include standard rectifier diodes, Schottky diodes, Zener diodes, and Light Emitting Diodes (LEDs).
- **Transistors:** Transistors act as switches or amplifiers and are fundamental in digital circuits for processing signals. Key types of transistors are Bipolar Junction Transistors (BJTs), Field-Effect Transistors (FETs), and Metal-Oxide-Semiconductor FETs (MOSFETs).
- **Integrated Circuits (ICs):** ICs integrate multiple electronic components (transistors, resistors, capacitors) on a single chip to perform complex functions like amplification, oscillation, and signal processing. ICs can be analog, digital, or mixed-signal, with applications ranging from microcontrollers to microprocessors.

Electromechanical Components

- **Connectors:** Connectors provide electrical connection points for external circuits or devices. They come in various forms, such as pin headers, USB connectors, HDMI connectors, and power connectors.
- **Switches:** Switches manually open or close a circuit, enabling control over the flow of electricity. Types of switches include toggle switches, push-button switches, rotary switches, and slide switches.
- **Relays:** Relays are electromechanical switches operated by an electrical signal. They are used to control a high-power circuit with a low-power signal, providing

isolation and control in various applications.

Sensors and Other Components

- **Sensors:** Sensors detect changes in environmental conditions (such as temperature, light, or pressure) and convert them into electrical signals. Examples include temperature sensors, light sensors, pressure sensors, and motion sensors.
- **Oscillators:** Oscillators generate periodic waveforms used for clock signals in digital circuits. Types include crystal oscillators, RC oscillators, and LC oscillators.

Additional Components

- **Voltage Regulators:** Voltage regulators maintain a constant output voltage despite variations in input voltage and load conditions. They are critical in ensuring stable power supply to sensitive electronic components.
- **Fuses and Circuit Protection Devices:** Fuses protect circuits from overcurrent conditions by breaking the circuit when the current exceeds a certain threshold. Other protection devices include varistors and transient voltage suppression (TVS) diodes.

Each of these components plays a specific role in the overall functionality of a PCB, and their correct placement and integration are essential for the proper operation of electronic devices.

1.2.2. Heterogeneity of PCBs

Printed Circuit Boards (PCBs) are highly versatile and come in a wide array of types, shapes, and sizes to accommodate the diverse needs of modern electronic applications. The design and construction of a PCB are tailored to its specific function, the complexity of the circuit it supports, and the environmental conditions it will encounter. This heterogeneity is essential to meet the varied demands of industries ranging from consumer electronics to aerospace and medical devices.

Types of PCBs

PCBs can be classified into several types based on their structure and design complexity:

- **Single-Sided PCBs:** These are the simplest type of PCBs, consisting of a single layer of substrate with copper traces on one side. They are commonly used in

low-cost, low-density applications such as calculators, power supplies, and simple sensors.

- **Double-Sided PCBs:** These PCBs have copper traces on both sides of the substrate, allowing for more complex circuitry compared to single-sided boards. They are used in a wide range of applications, including industrial controls, power amplifiers, and automotive electronics.
- **Multi-Layer PCBs:** Comprising three or more layers of copper separated by insulating materials, multi-layer PCBs are used in complex, high-density applications. They provide enhanced functionality and reduced signal interference, making them ideal for computers, smartphones, and other advanced electronic devices.
- **Rigid PCBs:** Made from solid, inflexible materials, rigid PCBs maintain their shape and are used in applications where the board must remain stationary and stable. Examples include desktop computers, televisions, and medical devices.
- **Flexible PCBs:** Constructed from flexible plastic substrates, these PCBs can bend and fold, making them suitable for dynamic and space-constrained applications. They are commonly found in wearable devices, flexible displays, and medical implants.
- **Rigid-Flex PCBs:** Combining elements of both rigid and flexible PCBs, these boards offer the durability of rigid PCBs with the flexibility of flexible PCBs. They are used in applications that require both stability and flexibility, such as aerospace and military electronics.

Shapes and Sizes of PCBs

PCBs are also manufactured in a variety of shapes and sizes to fit the physical constraints and requirements of different electronic devices. Some common variations include:

- **Standard Rectangular PCBs:** The most common shape, used in a wide range of applications due to its simplicity and ease of manufacturing.
- **Custom-Shaped PCBs:** Designed to fit specific product enclosures, these PCBs can be circular, triangular, or any other shape required by the application. Custom shapes are often used in automotive, aerospace, and consumer electronics.
- **Miniaturized PCBs:** As electronic devices become smaller and more compact, miniaturized PCBs are increasingly common. These small PCBs are used in smartphones, wearable devices, and implantable medical devices.

- **Large PCBs:** Used in applications such as industrial equipment and large-scale computing systems, these PCBs can handle complex circuits and high power requirements.

Materials Used in PCBs

The choice of materials for PCB construction also varies depending on the application requirements:

- **FR4 (Fiberglass Epoxy):** The most common PCB substrate material, known for its good electrical insulation, mechanical strength, and resistance to moisture. Used in general-purpose electronics.
- **Polyimide:** Used for flexible PCBs, polyimide offers excellent thermal stability and flexibility, making it suitable for high-temperature and dynamic applications.
- **Metal-Core PCBs:** Featuring a metal core (usually aluminum or copper) for better heat dissipation, these PCBs are used in high-power applications like LED lighting and power supplies.
- **Ceramic PCBs:** Offering high thermal conductivity and stability, ceramic PCBs are used in high-frequency and high-power applications such as RF modules and power amplifiers.

The heterogeneity of PCBs in terms of type, shape, size, and material allows them to meet the diverse and specific needs of modern electronic applications.

Challenges in Automatic Component Recognition

The heterogeneity of PCBs presents a significant challenge in the context of automatic component recognition. The variety in types, shapes, sizes, and materials of PCBs adds complexity to the task of identifying and classifying components accurately. This variability requires robust algorithms that can adapt to a wide range of conditions and configurations.

Effective automatic component recognition systems must account for:

- **Different PCB Designs:** Each PCB design may have unique features and layouts that need to be accurately recognized and processed by the algorithms.
- **Component Variability:** Components may vary in size, shape, and orientation, necessitating advanced image processing and machine learning techniques to accurately identify them.

- **Environmental Factors:** Factors such as lighting conditions, reflections, and shadows can affect the quality of the images used for recognition, requiring algorithms that are resilient to such variations.
- **Manufacturing Variations:** Slight differences in manufacturing processes can lead to variations in PCB appearance, which must be accounted for to ensure accurate recognition.

1.3. Component Recognition on a Printed Circuit Boards

The recognition of specific electronic components (see Section 1.2.1 for details) is carried out with the aim of obtaining the following data, to be used later for testing these components:

- Type of the component
- Name of the component
- Position of the component
 - Centroid
 - Dimensions
 - Shape
- Orientation of the component

As an example, Figures 1.2, 1.3, 1.4 and 1.5 show some recognized electronic components on the same PCB displayed in Figure 1.1. Note that the recognition is carried out on the complete board, Figures 1.3, 1.4 and 1.5 present the enlarged area where the recognition occurs only for visualization purposes.

This task requires a manual process where an domain expert recognizes the various components.

1.3.1. Limitations of Manual PCB Component Recognition

The manual process of recognizing electronic components on PCBs has several significant limitations:

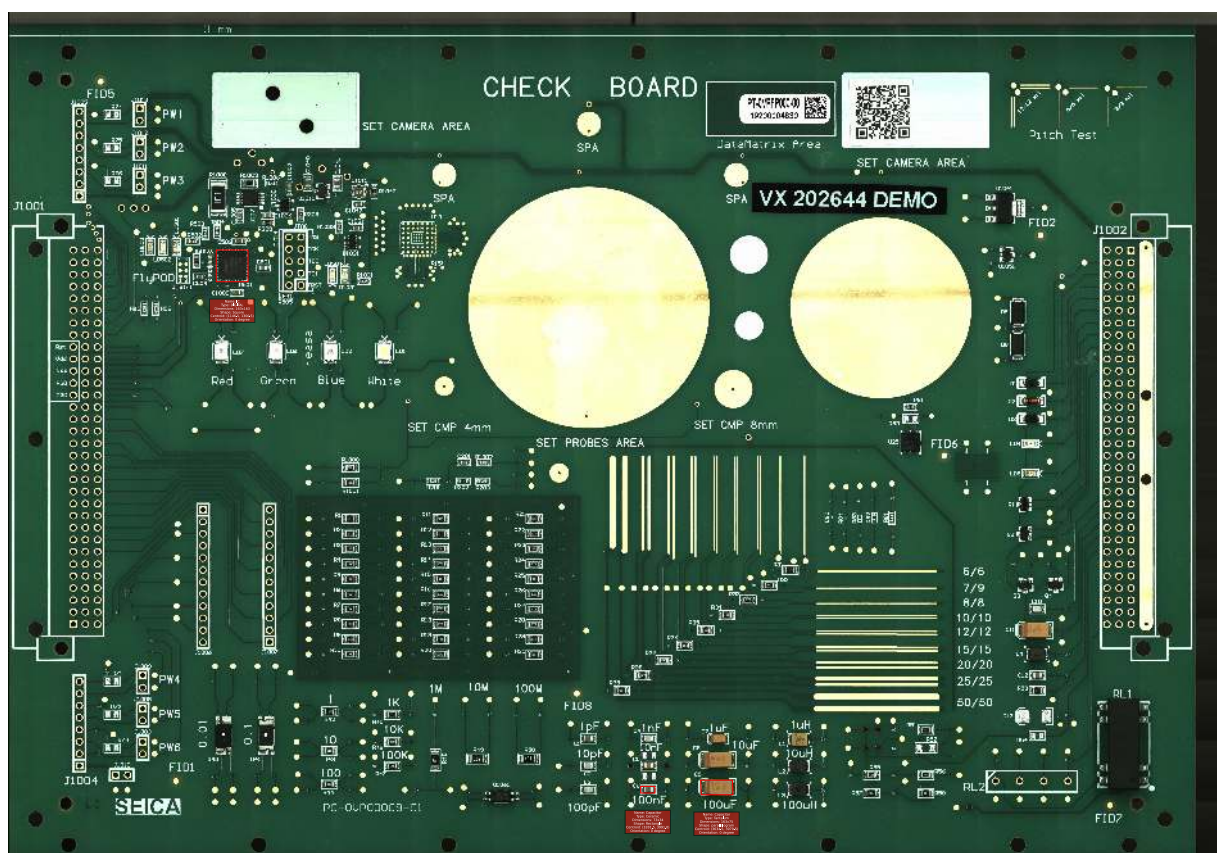


Figure 1.2: Component recognition on a PCB.

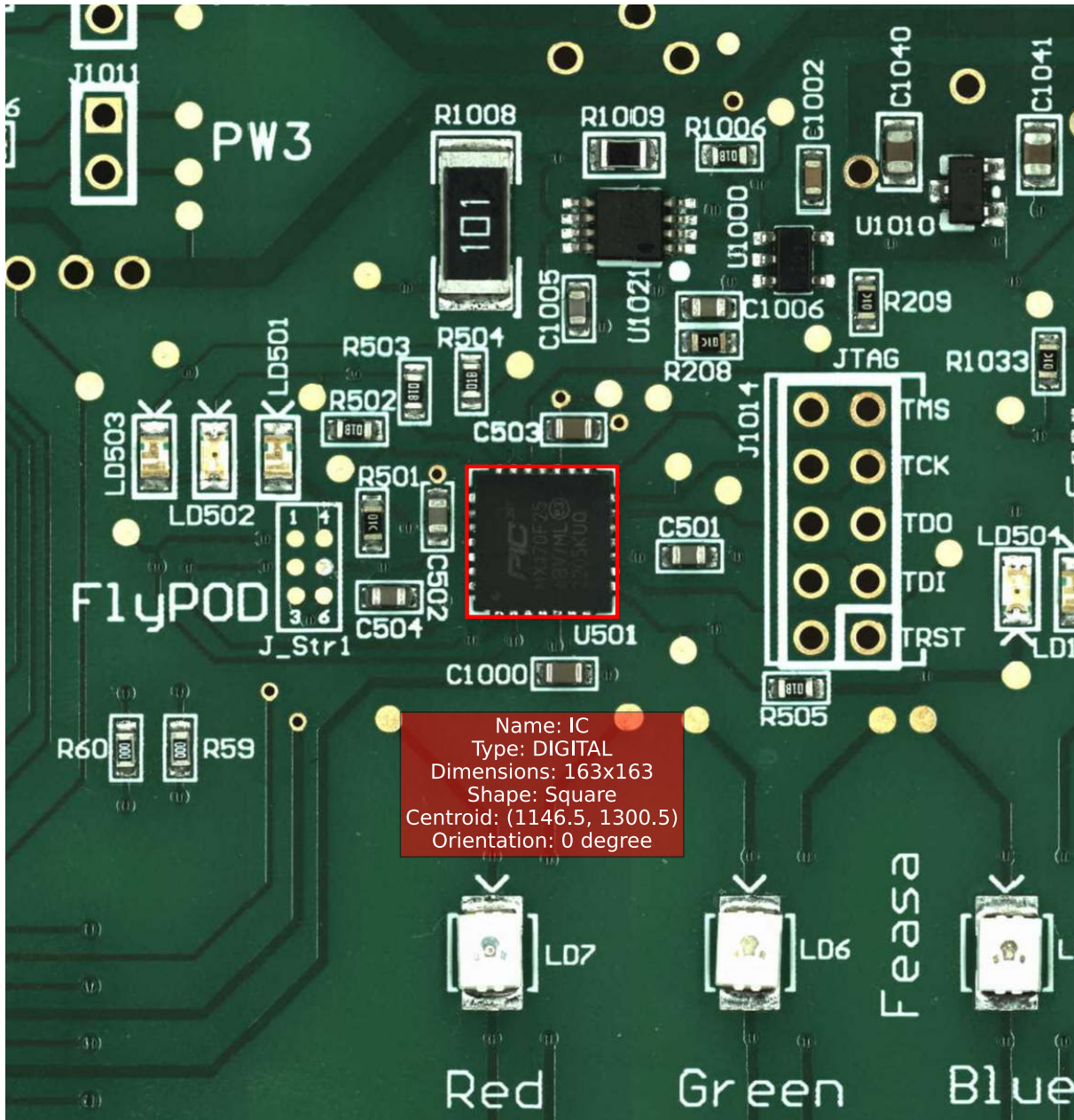


Figure 1.3: IC recognition on a PCB.

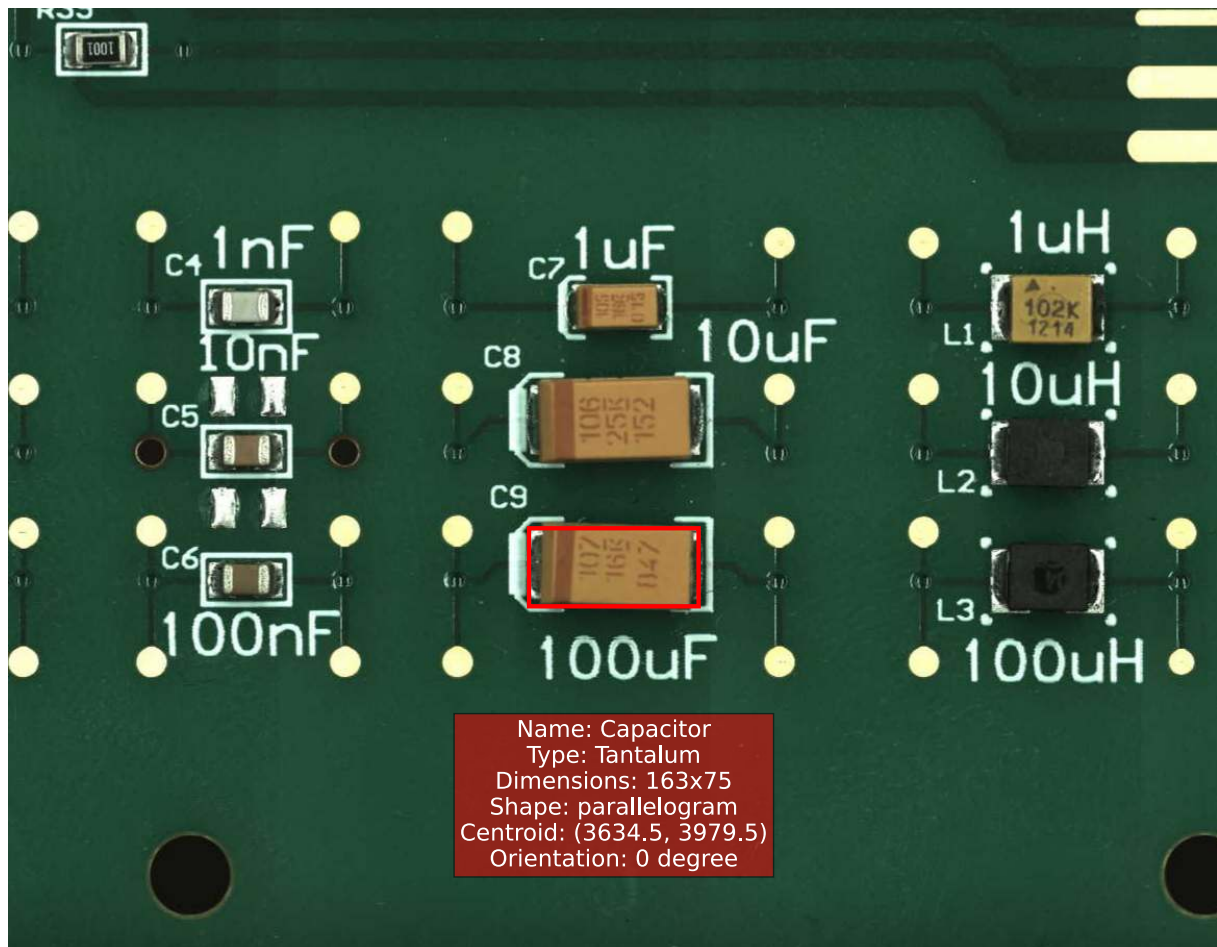


Figure 1.4: Tantalum capacitor recognition on a PCB.

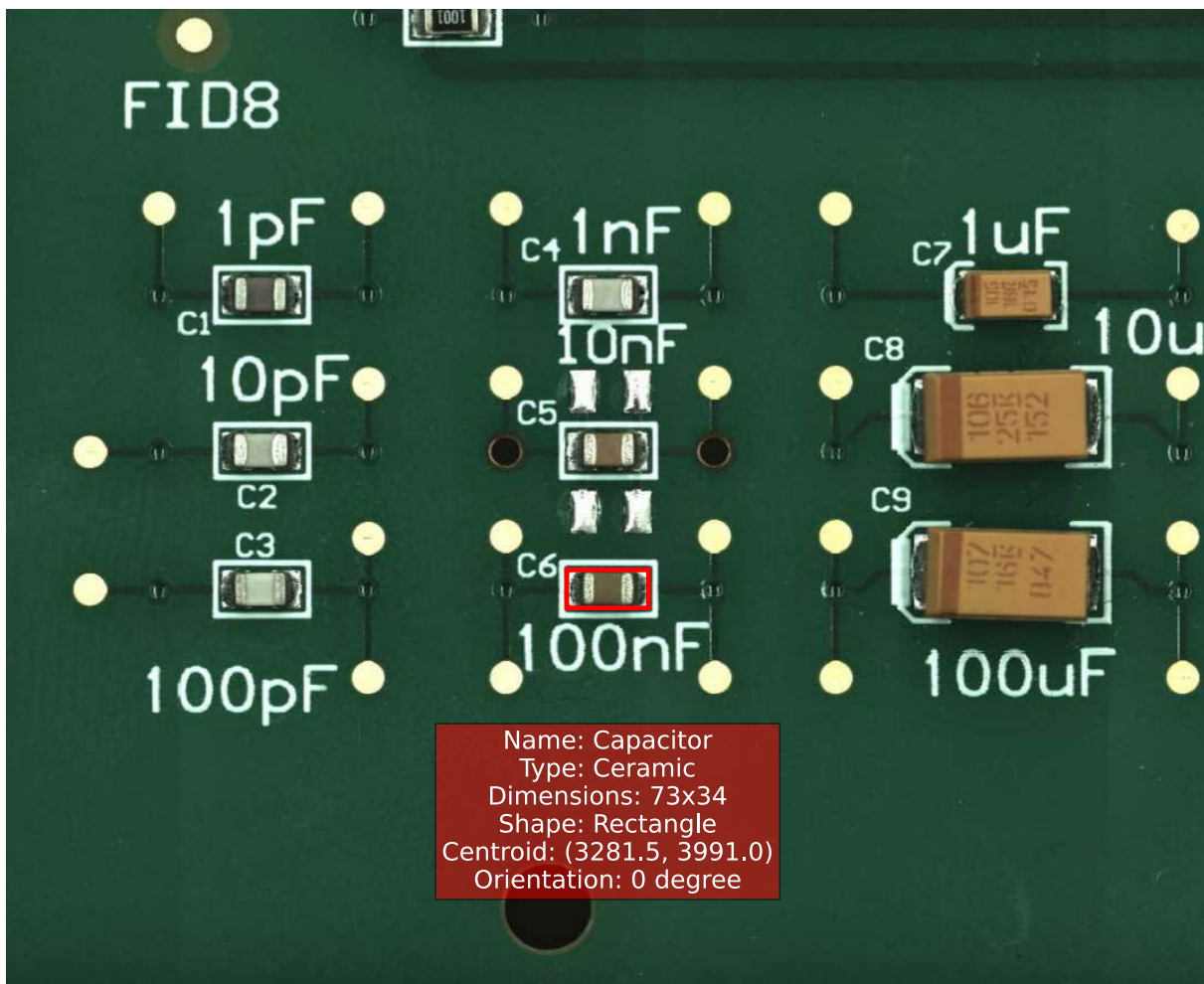


Figure 1.5: Ceramic capacitor recognition on a PCB.

Time-Consuming The manual recognition is time-intensive since, in a corporate production context, dozens of PCBs are analyzed and each PCB can present hundreds of components to be recognized.

Error-Prone Human error is inevitable in such detailed and repetitive tasks, since one or more people have to analyze all the areas of a single PCB to determine which components are present and their positions. It is a very repetitive and potentially tedious task, which requires great attention to detail and patience.

Domain Expertise Requirement The manual recognition needs the involvement of experts who possess knowledge of electronic components. It must also be considered that the components can vary greatly from one PCB to another, and the images of the latter can be taken under very different conditions of brightness, saturation, and contrast.

1.3.2. Potential of Artificial Intelligence in PCB Component Recognition

In this context, the introduction of Machine Learning and Deep Learning technologies offers some potential advantages:

Increased Efficiency Once the model is trained, it can be used to automatically recognize components in all new available PCBs, making this task very fast.

Improved Accuracy Trained models can consistently detect and classify components with high precision, minimizing the likelihood of errors.

Scalability Trained models can be used to develop AI-driven systems which can be easily scaled to handle large volumes of PCBs.

2 | Background

2.1. Central Role of Data in AI Development

2.1.1. Introduction

In the development of artificial intelligence models, particularly those based on machine learning and deep learning, data plays a central crucial role.

Indeed, the fundamental difference between machine learning algorithms and traditional models lies in how they learn and operate [18, 40].

Traditional Models Traditional models are typically based on predefined rules or equations derived from a theoretical understanding of the problem. These models require the analyst to explicitly encode the knowledge into the system. The rules and equations are manually determined and do not change unless modified by the analyst. This approach works well for problems where the dynamics are well understood and relatively static.

Machine Learning Algorithms Machine learning algorithms, on the other hand, learn directly from data. Here's why:

1. Automatic Learning

- **From Data to Models:** ML algorithms use data to identify patterns, correlations, and hidden relationships without requiring explicit rule encoding. This is crucial because many real-world problems are too complex or dynamic to be fully described through static rules.
- **Continuous Update:** ML algorithms can adapt to new data, improving their performance over time through continuous learning.

2. Ability to Handle Complexity and Variability

- **Data Complexity:** Real-world data is often complex and multidimensional. ML algorithms can handle this complexity better than traditional models, finding patterns in high-dimensional data.
- **Dynamic Environments:** ML algorithms are particularly useful in dynamic environments where relationships between variables can change over time. Traditional models would need continuous manual adjustments to keep up with these changes.

In problems like **image recognition**, it is practically impossible to write explicit rules that describe every possible variation of an object.

ML algorithms, such as convolutional neural networks (CNNs), can learn these variations directly from image data.

The quality, quantity, and diversity of data are critical factors that determine the performance and generalizability of these models. The following section explores the importance of data in AI, discussing various aspects such as data collection, preprocessing, and augmentation.

2.1.2. Data Collection

The first step in developing effective ML and DL models is the collection of relevant and high-quality data.

Data collection involves gathering raw data from various sources, which can include sensors, cameras, databases and user interactions. The collected data must accurately represent the real-world scenarios the model is intended to operate in.

- **Sources of Data:** Depending on the application, data can be sourced from structured databases, web scraping, IoT devices, manual annotations, and publicly available datasets.

Data Sources for Image Datasets

In image recognition tasks, images can be sourced from both public datasets and datasets created from images collected locally. Large-scale image datasets like ImageNet and COCO are commonly used [14, 21].

In general:

- **public datasets** are particularly useful because they usually have a very large amount of great-quality data, which can be useful to extract relevant fea-

tures from images; furthermore, many pre-trained models are trained with such datasets

- **local datasets**, whose data are strongly tied to the specific context of the problem, are indispensable because they allow the use of features extracted from a pre-trained model to map to specific categories relevant to the problem at hand.
- **Quality of Data:** High-quality data is free from noise, errors, and biases. Ensuring data quality involves rigorous validation and cleaning processes to remove inconsistencies and inaccuracies.

Data Quality in the Context of Images

High-quality image data ensures that models can learn meaningful patterns and make accurate predictions. Several aspects of data quality must be considered to ensure the success of image-based AI systems.

Image Resolution Image resolution refers to the amount of detail an image holds, typically measured in pixels. Higher resolution images contain more detail and can improve the performance of ML and DL models by providing more information for feature extraction. However, higher resolution also increases the computational load and storage requirements, necessitating a balance between resolution and resource constraints.

Image Clarity and Sharpness Clear and sharp images are essential for accurate feature extraction and recognition. Distorted images can obscure important details and lead to poor model performance. Ensuring that images are captured with appropriate focus and minimal motion blur is crucial for maintaining data quality.

Lighting and Contrast Proper lighting and contrast are vital for capturing high-quality images. Inconsistent lighting conditions can introduce shadows and highlights that obscure important features. Ensuring uniform lighting and optimizing contrast can significantly enhance the quality of image data.

Noise Reduction Image noise, such as graininess or pixelation, can degrade the quality of images and negatively impact model performance. Techniques such as noise filtering and smoothing can be applied to reduce noise and enhance image

quality. Noise reduction is particularly important in low-light conditions where noise levels tend to be higher.

Color Accuracy and Consistency Accurate and consistent color representation is important for tasks that rely on color information, such as object detection and classification. Variations in color due to different camera settings or lighting conditions can introduce biases in the data. Standardizing color profiles and performing color correction can help maintain color accuracy and consistency across the dataset.

Image Annotation Quality For supervised learning tasks, high-quality annotations are essential. Incorrect or inconsistent labeling can mislead the model during training, resulting in poor performance. Ensuring accurate and consistent annotations through rigorous quality control processes, such as manual review and correction, is crucial for maintaining high data quality.

Data Diversity and Representativeness A high-quality image dataset should be diverse and representative of the real-world scenarios the model will encounter. This includes variations in object appearance, background, lighting conditions, and viewpoints. Ensuring data diversity helps improve the generalization capabilities of the model, enabling it to perform well on unseen data.

Handling Imbalanced Data In many image datasets, certain classes may be over-represented while others are under-represented. This class imbalance can lead to biased models that perform well on majority classes but poorly on minority classes. Techniques such as data augmentation, resampling, and the use of class weights can help address class imbalance and improve model performance.

- **Data Volume:** Sufficient volume of data is necessary to train robust models. In DL, especially, large datasets are crucial to leverage the full potential of deep neural networks.

2.1.3. Data Preprocessing

Raw data collected from various sources often needs to be preprocessed to make it suitable for training ML and DL models. Data preprocessing involves several steps aimed at transforming raw data into a clean and structured format.

Data Preprocessing for Image Datasets

ML and DL models receive input data organized in a structured and, in most cases, uniform format, designed to best describe the characteristics of the represented records.

In the context of image processing with such models, single images are usually represented using 3D tensors, used to encapsulate the spatial and color information in a format that is easily manageable by ML and DL algorithms.

Specifically, an image can be represented as a 3D tensor with the following dimensions:

1. **Height (H):** The number of pixels along the vertical axis of the image.
2. **Width (W):** The number of pixels along the horizontal axis of the image.
3. **Channels (C):** The number of color channels in the image, typically:
 - (a) 3 for RGB images (Red, Green, Blue)
 - (b) 4 for RGBA images (Red, Green, Blue, Alpha)
 - (c) 1 for Grayscale images.

The first two dimensions allow to keep track of the spatial relationships between pixels, both along the x and y coordinates, while the number of channels used depends on the features to be extracted from the input data. For instance, in a pattern recognition task, usually Grayscale format images are preferred, while in classification tasks RGB format images are often used. Indeed, in pattern recognition tasks, the features to be extracted are more closely related to the shape and size of the objects depicted in the images, while in classification tasks, the color and brightness of individual pixels also play a fundamental role.

In order to obtain data in this format for image processing, several pre-processing steps for the raw data are necessary.

Data Loading and Splitting The initial step involves loading image data from the specified source and splitting it into training and validation sets.

The dataset is usually shuffled to ensure randomness before being split based on a specified ratio, typically with a default split of 80% for training and 20% for validation. This helps in creating a robust model by providing diverse data samples for training and a separate set for validating model performance.

Image Transformations Transformations are applied to the images to standardize and enhance the dataset. The primary transformations include:

- **Resizing:** Images are resized to a specified height and width to ensure uniform input dimensions for the model. This is crucial for maintaining consistency across the dataset and reducing computational load.
- **Normalization:** Images are normalized using specified mean and standard deviation values. This process adjusts the pixel values to a common scale, typically by transforming them to have a mean of zero and a standard deviation of one. Normalization helps in accelerating the training process and improving model convergence.

One-Hot Encoding For classification tasks, labels are converted into one-hot encoded vectors. One-hot encoding transforms categorical labels into binary vectors, where each vector has a length equal to the number of classes, and only the index corresponding to the label is set to 1. This encoding facilitates the use of categorical data in training ML and DL models.

2.1.4. Data Augmentation

Data augmentation is a technique used to artificially increase the size and diversity of a training dataset without actually collecting new data. This is particularly useful in DL where large datasets are required.

Data Augmentation techniques for Image Data

Data augmentation techniques involve applying various transformations to the existing images to generate additional training samples, enhancing the robustness of the model. The most used transformations include:

Brightness Adjustment Brightness adjustment modifies the brightness of images to simulate different lighting conditions. This helps the model learn to recognize objects under varying illumination levels.

Contrast Adjustment Contrast adjustment aims to modify the contrast of images, by exposing the model to a wider range of image intensities, improving its ability to distinguish between objects with subtle differences in color and texture.

Horizontal and Vertical Flips Flipping images horizontally and vertically introduces variations in the orientation of objects within the images. Horizontal and vertical flips ensure that the model does not become biased towards any particular orientation, making it more robust in recognizing objects regardless of their placement within the frame.

Hue Adjustment Hue adjustment modifies the hue of images, adding variations in color that can occur due to different lighting conditions or camera settings. This technique helps the model generalize better by learning to recognize objects with slight color variations.

JPEG Quality Adjustment JPEG quality adjustment is used to simulate the compression artifacts that can occur when images are saved in the JPEG format. By varying the JPEG quality, the model becomes more resilient to image compression artifacts, which is particularly useful when dealing with images from diverse sources.

Saturation Adjustment Saturation adjustment modifies the saturation levels of images, creating variations in color intensity. This technique enhances the model's ability to handle images with different color saturation levels, which can occur due to varying lighting conditions or image processing steps.

2.2. Machine Learning as a Core Component of AI

2.2.1. Introduction to Machine Learning

Machine Learning (ML) refers to the field of study that gives computers the ability to learn from data without being explicitly programmed [23].

ML techniques can be broadly categorized into three types: supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, the algorithm is trained on a labeled dataset, meaning that each training example is paired with an output label. Unsupervised learning algorithms, on the other hand, deal with data without labeled responses, focusing on finding the underlying structure in the data. Reinforcement learning involves an agent that interacts with its environment to perform actions and learns to achieve goals through trial and error.

2.2.2. Key Machine Learning Algorithms

- **Linear Regression:** One of the simplest algorithms, used for predictive modeling and establishing a relationship between independent and dependent variables
- **Decision Trees:** A non-parametric supervised learning method used for classification and regression. It splits the data into subsets based on the value of input features.
- **Support Vector Machines (SVMs):** These are used for classification and regression tasks, and they work by finding the hyperplane that best divides a dataset into classes .
- **k-Nearest Neighbors (k-NN):** A simple, instance-based learning algorithm used for classification and regression, which makes predictions based on the closest training examples in the feature space.

2.2.3. Introduction to Deep Learning

Deep Learning (DL) is a subset of ML that utilizes neural networks with many layers (hence *deep*). These models are designed to automatically and adaptively learn features from data, typically large-scale datasets [8].

Neural networks are composed of nodes (neurons) organized in layers: input layer, hidden layers, and output layer. Each connection between neurons has an associated weight that is adjusted during training to minimize the prediction error.

2.2.4. Key Deep Learning Architectures

- **Convolutional Neural Networks (CNNs):** Primarily used for image and spatial data processing. CNNs utilize convolutional layers to automatically and adaptively learn spatial hierarchies of features from input images.
- **Recurrent Neural Networks (RNNs):** Designed for sequential data and time series analysis. RNNs have connections that form directed cycles, allowing them to maintain information in ‘memory’ over time.
- **Long Short-Term Memory Networks (LSTMs):** A special kind of RNN capable of learning long-term dependencies. They are well-suited for tasks where context is crucial, such as language modeling and translation.

2.2.5. Machine Learning algorithms for Image Recognition

To address problems like **image recognition**, it is possible to implement several ML algorithms.

2.2.5.1. SVM

Support Vector Machines (SVM) can be employed for image classification tasks by transforming images into feature vectors using various feature extraction techniques [4, 32].

Indeed, the main challenge in applying such models concerns the fact that they require mono-dimensional arrays as input, instead of multidimensional tensors, which are designed to better represent spatial information.

Feature Extraction To use SVM for image classification, images must be converted into feature vectors. Common feature extraction techniques include:

- **Histogram of Oriented Gradients (HOG)**: Extracts features based on the directions of gradients within image blocks.
- **Scale-Invariant Feature Transform (SIFT)**: Detects key points invariant to scale and orientation.
- **Local Binary Patterns (LBP)**: Analyzes textures based on local pixel patterns.
- **Gabor Filters**: Captures texture information at various scales and orientations.

2.2.5.2. Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are a class of deep neural networks that have proven particularly effective for image recognition tasks. CNNs automatically and adaptively learn spatial hierarchies of features from input images [35]

Architecture of CNNs The architecture of CNNs typically consists of several types of layers:

- **Convolutional Layers**: These layers apply a set of filters to the input image to produce a feature map. Each filter scans across the image and captures different features such as edges, textures, and patterns.
- **Pooling Layers**: Also known as subsampling or down-sampling layers, pooling layers reduce the dimensionality of the feature map while retaining the most important information. Common types of pooling include max pooling and average pooling.

- **Fully Connected Layers:** After several convolutional and pooling layers, the output is flattened and fed into one or more fully connected layers. These layers perform the final classification based on the features extracted by the convolutional layers.

2.2.5.3. Residual Networks (ResNet)

Residual Networks (ResNets) are a type of CNN designed to address the degradation problem observed when the depth of a neural network increases. Traditional deep networks often face issues with vanishing gradients, leading to higher training error as the network grows deeper. ResNet, overcomes this by utilizing residual blocks that allow gradients to propagate more effectively through the network [20].

Residual Blocks A residual block consists of a few stacked layers where the input of the block is added to its output. This shortcut connection, also known as a skip connection, enables the network to learn identity mappings, which makes it easier to train deeper networks. The residual block can be mathematically represented as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}$$

where $\mathbf{x}$ is the input, $\mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\})$ represents the residual mapping to be learned, and $\mathbf{y}$ is the output of the block.

Architecture of ResNet The ResNet architecture consists of several such residual blocks stacked together. Different variants of ResNet have been developed, such as ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152, with the number indicating the depth of the network. For instance, ResNet-50 contains 50 layers, including convolutional, batch normalization, and fully connected layers.

The overall architecture follows a simple design:

- **Initial Layers:** Typically includes a convolutional layer followed by batch normalization and a ReLU activation function.
- **Residual Blocks:** Several stacked residual blocks, where each block contains a series of convolutional layers with batch normalization and ReLU activation, along with the shortcut connections.
- **Final Layers:** A global average pooling layer followed by a fully connected layer and a softmax activation for classification tasks.

2.3. Optimization Algorithms in AI

Optimization algorithms play a crucial role in AI and (in particular) in DL, by finding the best parameters, hyperparameters, or structures that improve model performance.

These algorithms can be broadly categorized into gradient-based and gradient-free methods, each suitable for different types of problems.

Algorithm 2.1 General Optimization Algorithm

```

1: Initialization
2: Initialize parameters  $\theta_0 = \text{initialize}()$ 
3: Define hyperparameters (learning rate  $\alpha$ , maximum iterations  $N$ , etc.)
4: for  $t = 0, 1, 2, \dots, N - 1$  do
5:   Evaluate the objective function  $J(\theta_t)$ 
6:   Compute the update:
7:   if gradient-based then
8:     Compute the gradient  $g_t = \nabla_{\theta} J(\theta_t)$ 
9:     Update the parameters  $\theta_{t+1} = \theta_t - \alpha g_t$ 
10:  else
11:    Generate a new candidate solution  $\theta_{\text{candidate}} = \text{generate}(\theta_t)$ 
12:    if  $J(\theta_{\text{candidate}}) < J(\theta_t)$  then
13:      Update the parameters  $\theta_{t+1} = \theta_{\text{candidate}}$ 
14:    else
15:      Keep the current parameters  $\theta_{t+1} = \theta_t$ 
16:    end if
17:  end if
18:  Check for convergence:
19:  if  $|J(\theta_{t+1}) - J(\theta_t)| < \epsilon$  then
20:    Break the loop
21:  end if
22: end for
23: Return the optimal parameters  $\theta^* = \theta_t$ 

```

The algorithm represented in 2.1 describes a general optimization procedure for minimizing an objective function $J(\theta)$. In this context, J represents the objective function to be minimized, such as a loss function in machine learning problems. The parameters θ denote the set of model parameters that are being optimized during the process.

The algorithm starts by initializing the parameters θ_0 and defining the hyperparameters, including the learning rate α and the maximum number of iterations N . The learning rate controls the step size of each update in the parameter space.

In each iteration, the algorithm evaluates the objective function $J(\theta_t)$ and computes the update for the parameters, which can be done using gradient-based methods (where the gradient g_t of the objective function is calculated and used to update θ) or gradient-free methods (where a new candidate solution is generated and evaluated).

The convergence is checked in each iteration by comparing the change in the objective function value to a threshold ϵ . If the change is smaller than ϵ , the loop breaks, and the algorithm returns the optimized parameters θ^* .

2.3.1. Gradient-Based Optimization Algorithms

Gradient-Based Optimization algorithms are fundamental in the context of neural networks, as they determine how the network learns from data.

During the training process of a neural network, the goal is to minimize a loss function, which measures how much the network's predictions differ from the actual values. Gradient-Based Optimization algorithms are generally used to update the weights of the network to reduce this loss.

Some of the most used optimizers in such context are the following [37]:

- **Stochastic Gradient Descent (SGD)**. It uses a randomly selected subset of data to compute the gradient and update the model parameters:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t) \quad (2.1)$$

where θ represents the model parameters, η is the learning rate, and $J(\theta)$ is the loss function.

It's a basic and widely used optimization algorithm.

- **RMSprop (Root Mean Square Propagation)**. It adapts the learning rate for each parameter by dividing the learning rate by an exponentially decaying average of squared gradients:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma) g_t^2 \quad (2.2)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t \quad (2.3)$$

where $E[g^2]_t$ is the moving average of squared gradients, γ is the decay rate, and ϵ is a small constant for numerical stability.

- **Adagrad (Adaptive Gradient Algorithm).** It adjusts the learning rate for each parameter individually, scaling it inversely proportional to the square root of the sum of all past squared gradients:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i} \quad (2.4)$$

where G_t is the diagonal matrix where each element i, i is the sum of the squares of the gradients up to time step t .

- **Adadelta.** It is an extension of Adagrad that seeks to reduce its aggressive, monotonically decreasing learning rate by considering only a fixed-size window of past gradients.
- **Adafactor:** It reduces memory usage by approximating second-order moments with lower-rank factorization.
- **Adam (Adaptive Moment Estimation):** It combines the advantages of both Adagrad and RMSprop by maintaining per-parameter learning rates that are adapted based on the first and second moments of the gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (2.5)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2.6)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (2.7)$$

$$\theta_{t+1} = \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \quad (2.8)$$

where m_t and v_t are the first and second moment estimates, and β_1, β_2 are decay rates.

- **Adamax:** It is a variant of Adam based on the infinity norm, which can be more stable in certain cases.

2.3.2. Gradient-Free Optimization Algorithm

Gradient-free optimization algorithms, also known as derivative-free or zero-order optimization algorithms, do not require gradient information.

They are suitable for non-differentiable, noisy, expensive-to-evaluate objective functions or when the objective function has not an explicit mathematical known formulation (i.e., the set of parameters to be optimized and the value of the objective function to be minimized are known, but how such value depends on those parameters is not known).

Some of the most used optimizers in such context are the following [2, 33, 39]:

- **Random Search:** A simple method that explores the search space by randomly sampling points and selecting the best one.
- **Genetic Algorithms:** Inspired by the process of natural selection. It uses techniques such as selection, crossover, and mutation to evolve a population of solutions, articulated in the following phases.
 - **Initialization:** The initial process where a population of candidate solutions is generated, typically represented as strings or arrays of values (chromosomes).
 - **Selection:** The phase where the best solutions from the current population are chosen to reproduce and create the next generation.
 - **Crossover (or Recombination):** A genetic operator that combines two parent chromosomes to produce one or more offspring chromosomes. This process aims to mix the features of the parents to create better solutions.
 - **Mutation:** A genetic operator that introduces variations in the offspring chromosomes by randomly changing some of their values. This helps maintain genetic diversity in the population and avoids premature convergence to sub-optimal solutions.
 - **Termination:** The criterion that determines when the genetic algorithm should stop. This can be based on a fixed number of generations, a fitness threshold reached, or no significant improvement after a certain number of generations.
- **Particle Swarm Optimization (PSO):** Simulates the social behavior of birds flocking or fish schooling. Each particle adjusts its position based on its own experience and the experience of neighboring particles:
 - **Position:** represents the current solution of the particle.

- **Velocity**: represents the rate of change of the particle's position.
- **Best-known Position**: the best solution found by the particle and its neighbors.
- **Simulated Annealing**: Mimics the annealing process in metallurgy. It probabilistically decides to move to a new solution, allowing for occasional uphill moves to escape local minima.
- **CMA-ES (Covariance Matrix Adaptation Evolution Strategy)**: An evolutionary algorithm that adapts the covariance matrix of the search distribution, focusing on promising regions of the search space.
- **Bayesian Optimization**: Uses a probabilistic model to predict the objective function and selects the next point to evaluate based on this model.

3 | Model Design and Architecture

This chapter details the design and architecture of the machine learning model developed for the recognition of electronic components on PCBs (see Section 1.3 for details).

The entire process can be represented as a pipeline, which visualizes the sequential flow of data through various stages of preparation, training, and evaluation.

Figure 3.1 shows such pipeline, with all the different stages outlined. These stages are described in the following sections.

3.1. Data collection

Data collection is performed according to all principles outlined in Section 2.1.2.

3.1.1. Data sources

The images of individual components (see Section 1.2.1) are obtained by photographing the PCBs (Section 1.2) available at the company.

The work is partially manual and partially automated through software. Generally, within the company's management software system, there is an option to integrate code to display enlarged images of a single PCB, specifically mounted, via a camera. Additionally, there is the capability to move the camera using a joystick to view different areas. Once a specific area is reached, it is possible to call previously implemented routines to capture a photo of that area and save it to the company's database.

Once a PCB is mounted and the software for executing these routines is implemented, the camera can be manually moved to collect different photos.

This is a labor-intensive manual task, but it is necessary to obtain a large dataset with high-quality images. Moreover, once the images for training are collected, this task is no longer required, making it a long and manual but ultimately episodic task.

Additionally, routines can be implemented to automatically move the joystick along var-

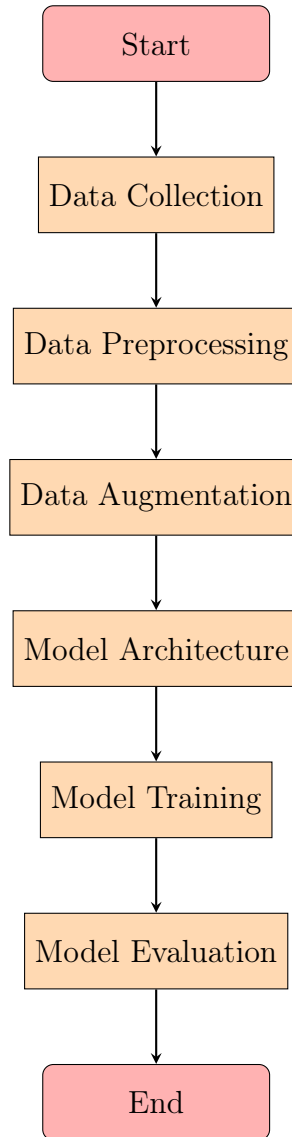


Figure 3.1: Pipeline of the Machine Learning Workflow

ious sections of a PCB to expedite the photo acquisition process.

However, this process can only be partially automated, as each photo must be manually checked to ensure that the component in question is clearly visible, not partially cut off, and that optimal conditions such as image clarity and sharpness, lighting and contrast, and color accuracy and consistency are met (see Subsection of Section 2.1.2 for the importance of data quality in this context).

Additionally, while images are collected and stored in the database, they are annotated with a specific label describing the component.

3.2. Data Preproceesing

All stages of Data Preprocessing are thoroughly justified in Section 2.13, where the rationale and objectives behind these techniques are highlighted.

The pipeline shown in Figure 3.2 illustrates the various stages implemented for Data Preprocessing, highlighting each step in the workflow.

In the following Subsections some implementation details are described.

3.2.1. Performance Optimization

Several techniques are employed to optimize the performance of the data pipeline.

3.2.1.1. Caching

Caching the dataset to avoid redundant computations and speed up data loading during training epochs. This step ensures that once data is loaded and preprocessed, it can be quickly accessed in subsequent epochs without reprocessing, thus significantly improving the efficiency of the training process.

3.2.1.2. Shuffling

Shuffling the dataset to ensure randomization, which is essential for preventing the model from learning the order of the training samples. By randomizing the order of the data, shuffling helps to improve the generalization capability of the model and reduces the risk of overfitting to specific sequences in the training data.

3.2.1.3. Batching

Dividing the dataset into smaller batches for efficient training. The batch size is chosen based on memory constraints and computational resources. Batching allows the model to update its weights more frequently and helps in utilizing the hardware resources more effectively, leading to faster convergence during training.

Batch size used: 32

3.2.1.4. Prefetching

Prefetching data to overlap data loading and model training, reducing idle time and improving training efficiency. By loading the next batch of data while the current batch

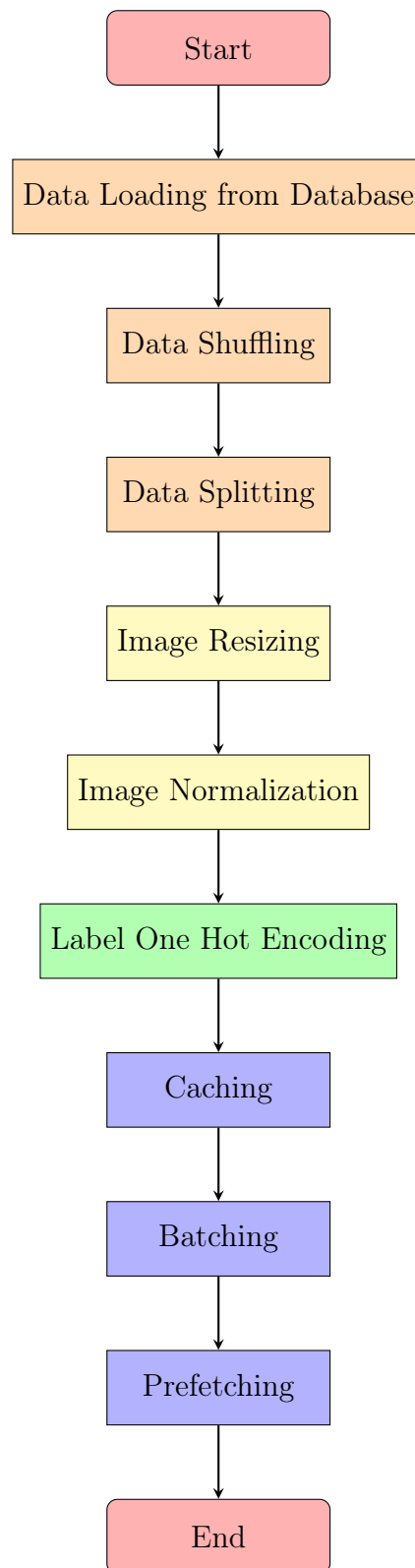


Figure 3.2: Pipeline of Data Preprocessing Workflow

is being processed, prefetching ensures that the model always has data ready for training, thus maximizing the usage of computational resources and minimizing waiting times.

3.2.2. Data splitting

Data splitting involves dividing the data into distinct subsets, a training set and a validation set

The training set is used to train the model, while the validation set is used to tune hyperparameters and evaluate the model's performance during training.

Percentages used for the split: 80% Training Set, 20% Validation Set

3.2.3. Data Transformations

3.2.3.1. Image Resizing

Standard Resize Parameters: height:128, width: 128

3.2.3.2. Image Normalization

The values used for image transformation depend on the type of format used for the tensor representation of the images. The specific values for normalization are determined by the color mode of the images. Below are the standard values used for different color modes and their respective normalization parameters.

Color Modes and Standard Normalization Parameters

RGB:

- Mean: [0.00392157, 0.00392157, 0.00392157]
- Standard Deviation (Std): [0.229, 0.224, 0.225]

Grayscale:

- Mean: [0.5]
- Std: [0.5]

RGBA:

- Mean: [0.485, 0.456, 0.406, 0.0]

- Std: [0.229, 0.224, 0.225, 1.0]

Note that RGB and RGBA mean values are scaled to the range [0, 1] by dividing by 255.

3.3. Model architecture

For what concerns the model architecture, three different types architecture are used and tested.

3.3.0.1. SVM

The input of a SVM model is a flatten version of the 3D tensor describing an image (See Section 2.2.5.1 for details).

Below are reported the implementation details chosen after an hypertuning process for using Support Vector Machine (SVM) in the design of the model.

Model 1.

Feature Engineering

- **Method:** Histogram of Oriented Gradients (HOG)
- **Parameters:**
 - Cell size: 8x8 pixels
 - Block size: 2x2 cells
 - Number of bins: 9

SVM Model Configuration

- **Kernel:** Linear
- **Regularization parameter (C):** 1.0
- **Kernel parameter (gamma) for RBF:** Not applicable (used linear kernel)

Model 2.

Feature Engineering

- **Method:** Histogram of Oriented Gradients (HOG)

- **Parameters:**
 - Cell size: 8x8 pixels
 - Block size: 2x2 cells
 - Number of bins: 9

SVM Model Configuration

- **Kernel:** Radial Basis Function (RBF)
- **Regularization parameter (C):** 10.0
- **Kernel parameter (gamma) for RBF:** 0.01

Model 3.

Feature Engineering

- **Method:** Principal Component Analysis (PCA)
- **Parameters:**
 - Number of components: 50

SVM Model Configuration

- **Kernel:** Polynomial
- **Regularization parameter (C):** 1.0
- **Degree:** 3
- **Coefficient (coef0):** 1.0

3.3.0.2. CNN

Below are reported the implementation details chosen after an hypertuning process for using Convolutional Neural Network (CNN, see Section 2.2.5.2 for details) in the design of the model.

Model 4.

Architecture

- **Convolutional Layers:**

- Conv1: 32 filters, kernel size 3x3, ReLU activation
- Conv2: 64 filters, kernel size 3x3, ReLU activation

- **Pooling Layers:**

- Pool1: MaxPooling, pool size 2x2
- Pool2: MaxPooling, pool size 2x2

- **Fully Connected Layers:**

- FC1: 128 units, ReLU activation
- Output: Number of classes, Softmax activation

Model 5.

Architecture

- **Convolutional Layers:**

- Conv1: 64 filters, kernel size 3x3, ReLU activation
- Conv2: 128 filters, kernel size 3x3, ReLU activation
- Conv3: 128 filters, kernel size 3x3, ReLU activation

- **Pooling Layers:**

- Pool1: MaxPooling, pool size 2x2
- Pool2: MaxPooling, pool size 2x2

- **Fully Connected Layers:**

- FC1: 256 units, ReLU activation
- Output: Number of classes, Softmax activation

Model 6.

Architecture

- **Convolutional Layers:**
 - Conv1: 32 filters, kernel size 3x3, ReLU activation
 - Conv2: 64 filters, kernel size 3x3, ReLU activation
 - Conv3: 128 filters, kernel size 3x3, ReLU activation
 - Conv4: 256 filters, kernel size 3x3, ReLU activation
- **Pooling Layers:**
 - Pool1: MaxPooling, pool size 2x2
 - Pool2: MaxPooling, pool size 2x2
 - Pool3: MaxPooling, pool size 2x2
- **Fully Connected Layers:**
 - FC1: 512 units, ReLU activation
 - Output: Number of classes, Softmax activation

3.3.0.3. ResNet

Below are reported the implementation details chosen after an hypertuning process for using Residual Network (ResNet, see Section 2.2.5.3 for details) in the design of the model.

Model 7.

Architecture

- **Base Model:** ResNet18
- **Layers:**
 - Standard ResNet18 architecture

Model 8.

Architecture

- **Base Model:** ResNet152 (without top layers)
- **Layers:**

- Base Model: ResNet152 with pretrained weights on ImageNet
- Flatten Layer: Flattens the output of the ResNet152 base model.
- Dense Layer 1: 1000 units, ReLU activation.
- Dense Layer 2: Number of classes, Softmax activation.

Model 9.

Architecture

- **Base Model:** ResNet152 (without top layers, frozen)
- **Layers:**
 - Base Model: ResNet152 with pretrained weights on ImageNet
 - Frozen Layers: All layers in the base model are frozen (not trainable).
 - Flatten Layer: Flattens the output of the ResNet152 base model.
 - Dense Layer 1: 1000 units, ReLU activation.
 - Dense Layer 2: Number of classes, Softmax activation.

Model 10.

Architecture

- **Base Model:** ResNet152 (without top layers)
- **Layers:**
 - Base Model: ResNet152 with pretrained weights on ImageNet
 - Flatten Layer: Flattens the output of the ResNet152 base model.
 - Dense Layer 1: 1024 units, ReLU activation.
 - Dense Layer 2: 2048 units, ReLU activation.
 - Dense Layer 3: 1024 units, ReLU activation.
 - Dense Layer 4: Number of classes, Softmax activation.

Model 11.

Architecture

- **Base Model:** ResNet152 (without top layers, frozen)
- **Layers:**
 - Base Model: ResNet152 with pretrained weights on ImageNet
 - Frozen Layers: All layers in the base model are frozen (not trainable).
 - Flatten Layer: Flattens the output of the ResNet152 base model.
 - Dense Layer 1: 1024 units, ReLU activation.
 - Dense Layer 2: 2048 units, ReLU activation.
 - Dense Layer 3: 1024 units, ReLU activation.
 - Dense Layer 4: Number of classes, Softmax activation.

4 | Adaptive Domain Randomization

Domain Augmentation techniques described in Section 2.1.4 can be used to perform a Domain Randomization approach with the aim of enhancing the robustness and generalization capabilities of developed models.

This chapter explores how Domain Randomization approach has been used to help models generalize better to new, unseen environments.

In the last part of the chapter techniques to optimize the domain parameters distributions are described, in an approach known as Adaptive Domain Randomization.

4.1. Domain Randomization

Domain Randomization refers to a set of techniques used to increase the robustness of ML models.

The core idea behind Domain Randomization is to expose the model to a wide variety of artificially generated variations (through Data Augmentation techniques) during training [30]. By doing so, the model learns to generalize better to new, unseen environments and conditions.

The variations introduced help the model become more resilient to the diverse and often unpredictable conditions it may encounter in real-world applications. Indeed, PCBs can be photographed under vastly different conditions. Additionally, the PCBs themselves can vary significantly in terms of size, color, and layout.

4.1.1. Parameters to be randomized

In the context of **image recognition**, Domain Randomization involves representing the specific domain by a set of parameters, concerning image specific properties.

The following parameters have been considered:

- Brightness
- Contrast
- Horizontally flip
- Vertically flip
- Hue
- JPEG quality
- Saturation

4.1.2. How Parameters are Randomized

The parameters mentioned in Section 4.1.1 are randomized according to some distributions.

Specifically, the algorithm used for parameter randomization is described as follows:

Algorithm 4.1 Parameter Randomization Algorithm

Require: Parameters set $\mathbf{P}$, the list of parameters to be randomized

Require: Probability vector $\mathbf{p}$, where $\mathbf{p}_i$ is the probability that the i -th parameter is randomized

Require: Number of epochs E

Require: Batch size B

```

1: for each epoch  $e$  from 1 to  $E$  do
2:   for each batch  $b$  from 1 to  $B$  do
3:     for each parameter  $i$  in  $\mathbf{P}$  do
4:       if  $\text{random.rand}() > \mathbf{p}_i$  then
5:         Sample a value for the parameter  $i$  from the parameter distribution  $\mathcal{D}_i$ 
6:         Generate an augmented image to be used for training
7:       end if
8:     end for
9:   end for
10: end for
```

Algorithm 4.1 begins by defining a probability vector $\mathbf{p}$, where each element represents the probability that a specific parameter $i \in \mathbf{P}$ (with $\mathbf{P}$: list of parameters to be randomized) will be randomized for a single image (or batch of images). The algorithm iterates over a specified number of epochs and batches. For each batch, it checks each parameter $i \in \mathbf{P}$ against its corresponding probability. If a random value (between 0 and 1) exceeds the probability for a given parameter, a new value is sampled from the parameter's distribution

$\mathcal{D}_i$ and an augmented image is generated for the training. This process continues until all epochs and batches have been processed.

This algorithm is integrated within the model training process, ensuring that the training data is continuously augmented with randomized parameters, thereby enhancing the robustness and generalization capabilities of the model.

4.1.2.1. Parameter distributions

The following distributions have been used:

Uniform distributions

- $\mathcal{U}(a_{\text{brightness}}, b_{\text{brightness}})$
- $\mathcal{U}(a_{\text{contrast}}, b_{\text{contrast}})$
- $\mathcal{U}(a_{\text{horizontal flip}}, b_{\text{horizontal flip}})$
- $\mathcal{U}(a_{\text{vertical flip}}, b_{\text{vertical flip}})$
- $\mathcal{U}(a_{\text{hue}}, b_{\text{hue}})$
- $\mathcal{U}(a_{\text{jpeg quality}}, b_{\text{jpeg quality}})$
- $\mathcal{U}(a_{\text{saturation}}, b_{\text{saturation}})$

with $a_i, b_i, i \in \mathbf{P}$ representing the lower and upper bounds of the uniform distribution for each parameter, respectively.

Triangular distributions

- $\mathcal{T}(a_{\text{brightness}}, m_{\text{brightness}}, b_{\text{brightness}})$
- $\mathcal{T}(a_{\text{contrast}}, m_{\text{contrast}}, b_{\text{contrast}})$
- $\mathcal{T}(a_{\text{horizontal flip}}, m_{\text{horizontal flip}}, b_{\text{horizontal flip}})$
- $\mathcal{T}(a_{\text{vertical flip}}, m_{\text{vertical flip}}, b_{\text{vertical flip}})$
- $\mathcal{T}(a_{\text{hue}}, m_{\text{hue}}, b_{\text{hue}})$
- $\mathcal{T}(a_{\text{jpeg quality}}, m_{\text{jpeg quality}}, b_{\text{jpeg quality}})$
- $\mathcal{T}(a_{\text{saturation}}, m_{\text{saturation}}, b_{\text{saturation}})$

with $a_i, b_i, m_i, i \in \mathbf{P}$ representing the lower bound, mode, and upper bound of the triangular distribution for each parameter, respectively.

Univariate normal distributions

- $\mathcal{N}(\mu_{\text{brightness}}, \sigma_{\text{brightness}}^2)$
- $\mathcal{N}(\mu_{\text{contrast}}, \sigma_{\text{contrast}}^2)$
- $\mathcal{N}(\mu_{\text{horizontal flip}}, \sigma_{\text{horizontal flip}}^2)$
- $\mathcal{N}(\mu_{\text{vertical flip}}, \sigma_{\text{vertical flip}}^2)$
- $\mathcal{N}(\mu_{\text{hue}}, \sigma_{\text{hue}}^2)$
- $\mathcal{N}(\mu_{\text{jpeg quality}}, \sigma_{\text{jpeg quality}}^2)$
- $\mathcal{N}(\mu_{\text{saturation}}, \sigma_{\text{saturation}}^2)$

with $\mu_i, \sigma_i^2, i \in \mathbf{P}$ representing the mean and variance of the normal distribution for each parameter, respectively.

Multivariate normal distribution

$$\mathcal{N}_7 \left(\begin{bmatrix} \mu_{\text{brightness}} \\ \mu_{\text{contrast}} \\ \mu_{\text{horizontal flip}} \\ \mu_{\text{vertical flip}} \\ \mu_{\text{hue}} \\ \mu_{\text{jpeg quality}} \\ \mu_{\text{saturation}} \end{bmatrix}, \begin{bmatrix} \sigma_{\text{brightness}}^2 & \sigma_{\text{brightness,contrast}} & \cdots & \sigma_{\text{brightness,saturation}} \\ \sigma_{\text{contrast,brightness}} & \sigma_{\text{contrast}}^2 & \cdots & \sigma_{\text{contrast,saturation}} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{\text{saturation,brightness}} & \sigma_{\text{saturation,contrast}} & \cdots & \sigma_{\text{saturation}}^2 \end{bmatrix} \right)$$

μ_i : mean, σ_i^2 : variance, and σ_{ij} : covariance

with $i, j \in \mathbf{P}, i \neq j$

4.2. Adaptive Domain Randomization

4.2.1. Challenges in Choosing Randomization Coefficients

The main challenge in Domain Randomization approach is the need to find suitable values for the distribution parameters or coefficients¹ (i.e., $a_i, b_i, m_i, \mu_i, \sigma_i^2, \sigma_{ij}$ with $i \in \mathbf{P}$ as defined in Subsection 4.1.2.1).

¹In the following, the term 'coefficients' will be used exclusively, as 'parameters' has already been used for the domain parameters, in order to avoid confusion for the reader.

1. A very restrictive choice can lead to limited variety in the randomization process (e.g., similar values for a and b in uniform and triangular distributions and low σ values in normal distributions).
2. Conversely, a very broad choice can result in conditions that describe highly improbable and unrealistic environments, thereby harming the model training process.
3. In general, it is not easy to find suitable values, as this requires in-depth domain knowledge (in this case, the parameters of the image) and a lengthy process of manual fine-tuning.

4.2.2. Coefficient Optimization

One possible way to overcome this issue is to optimize these coefficients.

By using optimization algorithms, it is possible to systematically find the set of coefficients that provide the best performance for the model. This approach can help in balancing the variety introduced through randomization and ensuring that the training conditions remain realistic and beneficial for model learning.

4.2.2.1. Implementation

In the implementation of the neural network, a parallel branch can be included.

While the main branch focuses on optimizing the neural network weights (using gradient-based algorithms, see Subsection 2.3.1), the second branch is dedicated to optimizing the coefficients of the distribution.

This dual approach ensures that both the model parameters and the randomization parameters are fine-tuned simultaneously, enhancing the overall performance and robustness of the model.

Unlike the first branch, the second branch cannot use gradient-based optimization algorithms, as the objective function does not have an explicit, differentiable mathematical formulation. Therefore, it is necessary to use gradient-free optimization algorithms (see Subsection 2.3.2).

Algorithm 4.2 describes an improved version of Algorithm 4.1: after the forward propagation in the neural network and the loss $\mathcal{L}$ evaluation, the loss value is used to update the coefficients $\mathbf{c}_i$ of the distribution $\mathcal{D}_i(\mathbf{c}_i)$, used to sample parameters in the next step.

At the end of the training process, besides the optimal weights minimizing the loss function

Algorithm 4.2 Parameter Randomization and Optimization Algorithm

Require: Parameters set $\mathbf{P}$, the list of parameters to be randomized

Require: Probability vector $\mathbf{p}$, where $\mathbf{p}_i$ is the probability that the i -th parameter is randomized

Require: Number of epochs E

Require: Batch size B

Require: Distribution $\mathcal{D}_i(\mathbf{c}_i)$ for each parameter i , where $\mathbf{c}_i$ are the coefficients to be optimized

```

1: for each epoch  $e$  from 1 to  $E$  do
2:   for each batch  $b$  from 1 to  $B$  do
3:     for each parameter  $i$  in  $\mathbf{P}$  do
4:       if  $\text{random.rand}() > \mathbf{p}_i$  then
5:         Sample a value for the parameter  $i$  from the optimized distribution  $\mathcal{D}_i(\mathbf{c}_i)$ 
6:         Generate an augmented image to be used for training
7:         Forward propagation in the Neural Network
8:         Evaluate Loss  $\mathcal{L}$ 
9:         Optimize coefficients  $\mathbf{c}_i$  of the distribution  $\mathcal{D}_i(\mathbf{c}_i)$ , based on the value of  $\mathcal{L}$ 
10:      end if
11:    end for
12:  end for
13: end for

```

$\mathcal{L}$, the output will also include the optimized values $\mathbf{c}_i^*$ for the distribution coefficients $\mathcal{D}_i(\mathbf{c}_i)$.

4.2.2.2. Adaptive Domain Randomization advantages

In this way, it is not necessary to fix the values for the distribution coefficients $\mathbf{c}_i$ a priori, but it is sufficient to:

1. Define initial values for the coefficients.
2. Define the parameter space: identify the range of parameters to be optimized. Only a broad range is necessary (e.g., brightness factor must be between -1 and 1, therefore a, b, m, μ must be in the range $[-1, 1]$).
3. Define any constraints (e.g., $a \leq b, \sigma^2 \geq 0$).
4. Use a gradient-free optimizer.

Note In order to avoid remaining in a local minimum for many steps, thus reducing the possibility of finding global minima, it is important to use a gradient-free algorithm that frequently perturbs the current optimal solution (e.g., the Simulated Annealing in Subsection 2.3.2). This approach allows for effective exploration of the parameter space,

enabling the optimization process to escape local minima and increasing the likelihood of identifying global minima.

5 | Application of the Model

The output of the pipeline shown in 3.1 is a trained model for the recognition of single electronic components.

This model can be used to recognize components on complete PCBs, or in specific areas of them.

To address these two tasks, the trained model can be used to perform inference for both image classification and object detection, as summarized in 5.1.

This chapter describes in details both image classification and object detection methods.

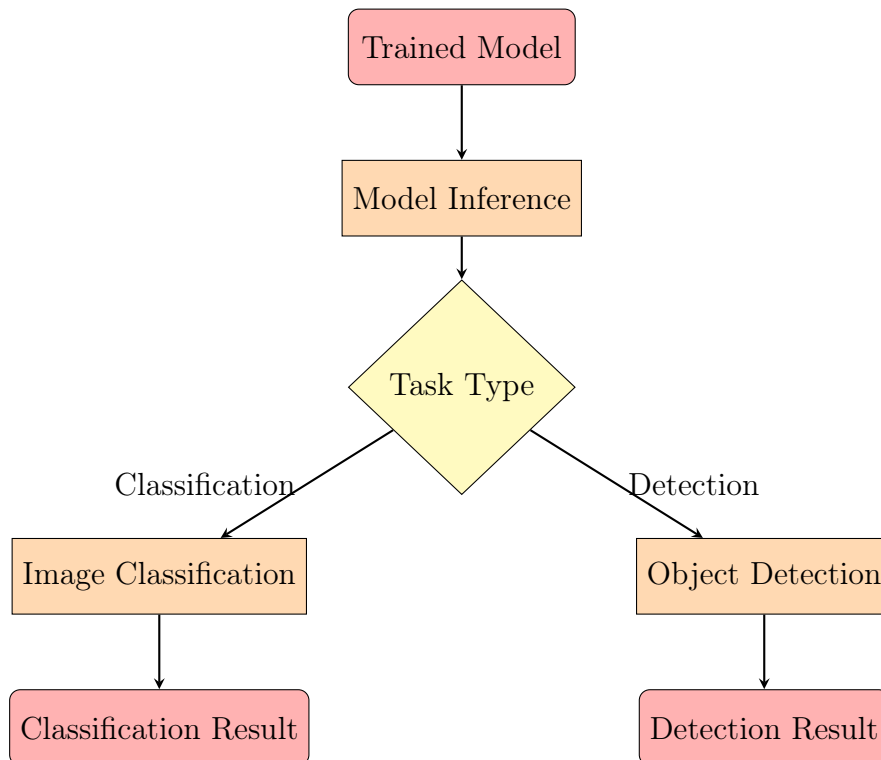


Figure 5.1: Pipeline from Trained Model to Inference for Image Classification and Object Detection

5.1. Image Classification

When the model is used for the image classification task, the trained model must be loaded into memory and used to make inferences on new images.

The output of the model will be a vector of likelihoods for each possible category. This vector can be interpreted as a probability distribution over the categories. By processing this output, it is possible to obtain a string describing the category (label) and its confidence (a probability value between 0 and 1).

For example, given a new image, the model might output the following likelihood vector:

$$[0.1, 0.3, 0.6]$$

This indicates the model's confidence for each category. By interpreting this vector, we can determine that the model predicts the image belongs to the category with the highest likelihood (in this case, the third category) with a confidence of 0.6.

Thus, the model not only provides the predicted category but also the associated confidence level, enabling a more precise understanding of the model's predictions.

5.2. Object Detection

When the model is used for object detection, the trained model must be loaded into memory and used to make inferences on new images. The process involves several steps.

Load Trained Model First, the trained model is loaded from memory. This ensures that the model is ready to make predictions on new images.

Load and Preprocess Image Next, the image to be processed is loaded and, if necessary, preprocessed.

Sliding Window Approach The image is then divided into smaller windows using a sliding window approach. Each window is further preprocessed to match the input requirements of the model, such as resizing and normalization. This preprocessing step is crucial to ensure compatibility with the model's input requirements.

Classify Windows Each window is classified by the model. The model outputs a vector of likelihoods for each possible category. Only the windows with a confidence score

above a certain threshold are considered as containing an object.

Non-Maximum Suppression (NMS) To reduce multiple detections of the same object, Non-Maximum Suppression (NMS) is applied. NMS is a technique used to select the best bounding box for each detected object based on the Intersection over Union (IoU) metric. The steps involved in NMS are as follows:

First, the detections are sorted by their confidence scores in descending order. For each detection d_i , the IoU with every other detection d_j is calculated. The IoU is defined as:

$$\text{IoU}(d_i, d_j) = \frac{\text{Area}(d_i \cap d_j)}{\text{Area}(d_i \cup d_j)}$$

where $\cap$ denotes the intersection of the two bounding boxes and $\cup$ denotes their union.

If the IoU of d_i with any other detection d_j exceeds a predefined threshold τ , d_j is suppressed (i.e., it is removed from consideration). This process is repeated for all detections, resulting in a set of final detections where each object is represented by the bounding box with the highest confidence score.

Mathematically, NMS can be described as:

$$\text{NMS}(D, \tau) = \{d_i \in D \mid \forall d_j \in D, \text{IoU}(d_i, d_j) < \tau\}$$

where D is the set of all detections and τ is the IoU threshold.

Calculate Centroids For each detected object, the centroid of the bounding box is calculated and added to the detection output. The centroid coordinates (c_x, c_y) are given by:

$$c_x = \frac{x_{\min} + x_{\max}}{2}, \quad c_y = \frac{y_{\min} + y_{\max}}{2}$$

where $x_{\min}$ and $x_{\max}$ are the minimum and maximum x-coordinates of the bounding box, and $y_{\min}$ and $y_{\max}$ are the minimum and maximum y-coordinates of the bounding box.

Output Detection Results Finally, the output includes the bounding boxes, class labels, confidence scores, and centroids for each detected object. This comprehensive output provides detailed information about the detected objects, facilitating further analysis and decision-making.

5.2.1. Adaptive Sliding Window Sizes

When images have significantly different dimensions, it is important to adapt the sliding window sizes dynamically to ensure effective object detection.

There are various approaches to handle this situation.

Multi-Scale Approach One effective method is the **Image Pyramid** approach [1].

This method involves resizing the image to various scales and applying the sliding windows to each scale. This allows the detection of objects of different sizes without having to predefine the window size.

Algorithm 5.1 Image Pyramid Algorithm

Require: Image I , scale factor $s > 1$, minimum size $(w_{\min}, h_{\min})$

Ensure: List of scaled images P

```

1: Initialize the pyramid  $P \leftarrow \{I\}$ 
2:  $W \leftarrow \text{width}(I)$ 
3:  $H \leftarrow \text{height}(I)$ 
4: while  $W \geq w_{\min}$  and  $H \geq h_{\min}$  do
5:    $W \leftarrow \lfloor W/s \rfloor$ 
6:    $H \leftarrow \lfloor H/s \rfloor$ 
7:   Resize the image  $I$  to  $(W, H)$ 
8:   Add the resized image to the pyramid  $P \leftarrow P \cup \{I\}$ 
9: end while
10: return  $P$ 

```

Algorithm 5.1 describes how to generate an image pyramid, which creates multiple scaled versions of the input image:

1. The algorithm starts by initializing the pyramid P with the original image I .
2. It then retrieves the width W and height H of the original image.
3. The algorithm enters a while loop that continues as long as the width and height of the image are greater than or equal to the specified minimum size $(w_{\min}, h_{\min})$.
4. Within the loop:
 - The width W and height H of the image are reduced by dividing by the scale factor s .
 - The image I is resized to the new dimensions (W, H) .
 - The resized image is added to the pyramid P .

5. The loop exits when the image dimensions fall below the minimum size, and the algorithm returns the pyramid P , which contains all the scaled versions of the original image.

This algorithm generates a list of resized images, starting from the original image and progressively reducing its size by the specified scale factor until the minimum size is reached. The resized images can then be used for sliding window object detection at different scales, allowing the model to detect objects of various sizes more effectively.

5.3. Appendix: Optimizing Inference time for Large-Scale Image Processing

When inference needs to be performed on many images (for example, when there are many images to classify in the image classification task or when there are numerous windows to classify in the object detection task), the process can slow down considerably.

To improve execution time, two complementary methods are possible.

5.3.1. Batch Processing

Batch processing involves grouping multiple images together into a single batch and performing inference on the entire batch at once. This method leverages the reuse of weights and reduces the overhead associated with individual image processing.

Why Batch Processing Works When processing images in batches, the neural network weights are loaded once and reused across all images in the batch. This reduces the computational overhead and allows for more efficient use of the hardware, particularly GPUs, which are optimized for parallel processing. By reducing the number of times the weights need to be loaded and applied, the overall execution time can be significantly reduced.

1. **Weights Reuse:** Weights are loaded once for the entire batch and reused for each image in the batch.
2. **Parallel Processing:** Batches allow for parallel processing of multiple images, leveraging the full capacity of the hardware.

5.3.2. Parallelism

Parallelism can be achieved through either processes or threads. Each approach has its advantages and disadvantages.

Processes Using multiple processes involves running separate instances of the inference code in parallel. Each process has its own memory space, which avoids issues with shared state but comes with higher memory overhead.

Advantages:

- **Isolation:** Each process runs in its own memory space, reducing the risk of interference between processes.
- **Scalability:** Processes can be scaled across multiple CPUs or machines.

Disadvantages:

- **Memory Overhead:** Each process has its own memory allocation, leading to higher overall memory usage.
- **Inter-Process Communication (IPC):** Communication between processes can be slower and more complex compared to threads.

Threads Using multiple threads involves running concurrent execution paths within the same process. Threads share the same memory space, which allows for faster communication but can lead to issues with shared state.

Advantages:

- **Low Memory Overhead:** Threads share the same memory space, leading to lower memory usage.
- **Fast Communication:** Communication between threads is faster and simpler compared to processes.

Disadvantages:

- **Shared State Issues:** Threads share the same memory space, which can lead to issues with data consistency and race conditions.
- **Limited Scalability:** Threads are limited to the CPU cores available on a single machine.

5.3.2.1. Choosing the Right Approach

The choice between processes and threads depends on the specific requirements of the task and the available hardware resources. For tasks that require high isolation and scalability, processes may be more appropriate. For tasks that benefit from fast communication and lower memory overhead, threads may be more suitable.

By combining batch processing with parallelism, it is possible to significantly improve the execution time of large-scale image processing tasks, making the model more efficient and scalable.

6 | Experimental setup and Results

In Section 3.3, 11 main developed models are described.

They have been trained in the same conditions and evaluated in the same way.

6.1. Training and Evaluation

- **Cross-validation:** 5-fold cross-validation
- **Evaluation Metrics:**
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
- **Neural Network Training Parameters¹:**
 - **Epochs²:** 1000
 - **Learning Rate:** 0.001

¹These parameters are valid only for Models 4, 5, 6, 7, 8, 9, 10, and 11.

²The number of epochs used varied based on the dataset size and the model being trained. In general, Models 4, 5, 6, 7, and 8, trained on the full dataset, showed stabilization in both accuracy and loss after approximately 420 epochs, while Models 9, 10, and 11 stabilized after around 965 epochs. Once this threshold was reached, both the loss and accuracy remained unchanged, indicating that the optimization had likely reached a local minimum, which either represented a global minimum or was very difficult to improve further. Additionally, the weight values (inspected across epochs using the TensorBoard tool) showed no significant change beyond this threshold.

A total of 1000 epochs was used during the evaluation phase to ensure a uniform comparison across all models.

- **Optimizer**³: Adam
- **Loss Function**: Categorical Crossentropy

Accuracy Accuracy is the ratio of correctly predicted instances to the total instances. It is a common evaluation metric for classification problems and is defined as:

$$\text{Accuracy} = \frac{\sum_{i=1}^n TP_i}{\sum_{i=1}^n (TP_i + TN_i + FP_i + FN_i)}$$

where:

- TP_i = True Positives for class i
- TN_i = True Negatives for class i
- FP_i = False Positives for class i
- FN_i = False Negatives for class i
- n = Total number of classes

Precision Precision for each class is the ratio of correctly predicted positive observations to the total predicted positives. It is a measure of the accuracy of the positive predictions and is defined as:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

The overall precision can be calculated as the average of the precision values for all classes.

Recall Recall, also known as Sensitivity or True Positive Rate, for each class is the ratio of correctly predicted positive observations to all the observations in the actual class. It is a measure of the completeness of the positive predictions and is defined as:

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

The overall recall can be calculated as the average of the recall values for all classes.

F1-score The F1-score for each class is the harmonic mean of Precision and Recall. It is a single metric that combines both precision and recall, providing a balance between

³During the hyperparameter tuning phase for each model, all the optimizers listed in Section 2.3.1 were tested (using the TensorFlow library). In general, the results did not show significant variation depending on the optimizer used.

The Adam optimizer was chosen for the evaluation phase of the various models to ensure a consistent comparison.

the two. The F1-score is defined as:

$$\text{F1-score}_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

The overall F1-score can be calculated as the average of the F1-score values for all classes.

Confusion Matrix A Confusion Matrix is a table used to describe the performance of a classification model for a multiclass classification problem. It provides a comprehensive view of the true positives, true negatives, false positives, and false negatives for each class, allowing for detailed analysis of the model's performance. The confusion matrix is structured as follows:

	Predicted Class 1	Predicted Class 2	...	Predicted Class n
Actual Class 1	$C_{1,1}$	$C_{1,2}$	...	$C_{1,n}$
Actual Class 2	$C_{2,1}$	$C_{2,2}$	...	$C_{2,n}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Actual Class n	$C_{n,1}$	$C_{n,2}$	...	$C_{n,n}$

Table 6.1: Confusion Matrix for Multiclass Classification

6.2. Model comparison

Figure 6.1 illustrates the accuracy progression of the tested models on the training set and the test set. The y-axis represents the normalized accuracy, while the x-axis shows the model identifier (*Model ID*). Accuracy is computed separately for the training set (*train*) and the test set (*test*), as indicated in the legend.

From the graph, it is evident that for the first models (1 to 3), accuracy is relatively low, with values below 50%. Starting from model 4, there is a significant improvement, with accuracy steadily increasing until reaching values above 90% for models 9, 10, and 11.

Table 6.2 presents the performance metrics for the models evaluated, including accuracy, precision, recall, and F1-score, expressed as percentages. The metrics are reported for both the training and test sets, across the different models. It can be observed that models 1 through 3 perform poorly, with all metrics below 30%. Starting from model 4, there is a notable improvement, with accuracy and other metrics exceeding 90% for the best-performing models (models 10 and 11).

These results highlight a consistent increase in performance as the models become more complex, in particular with employing a neural network. Among such models, ResNet-

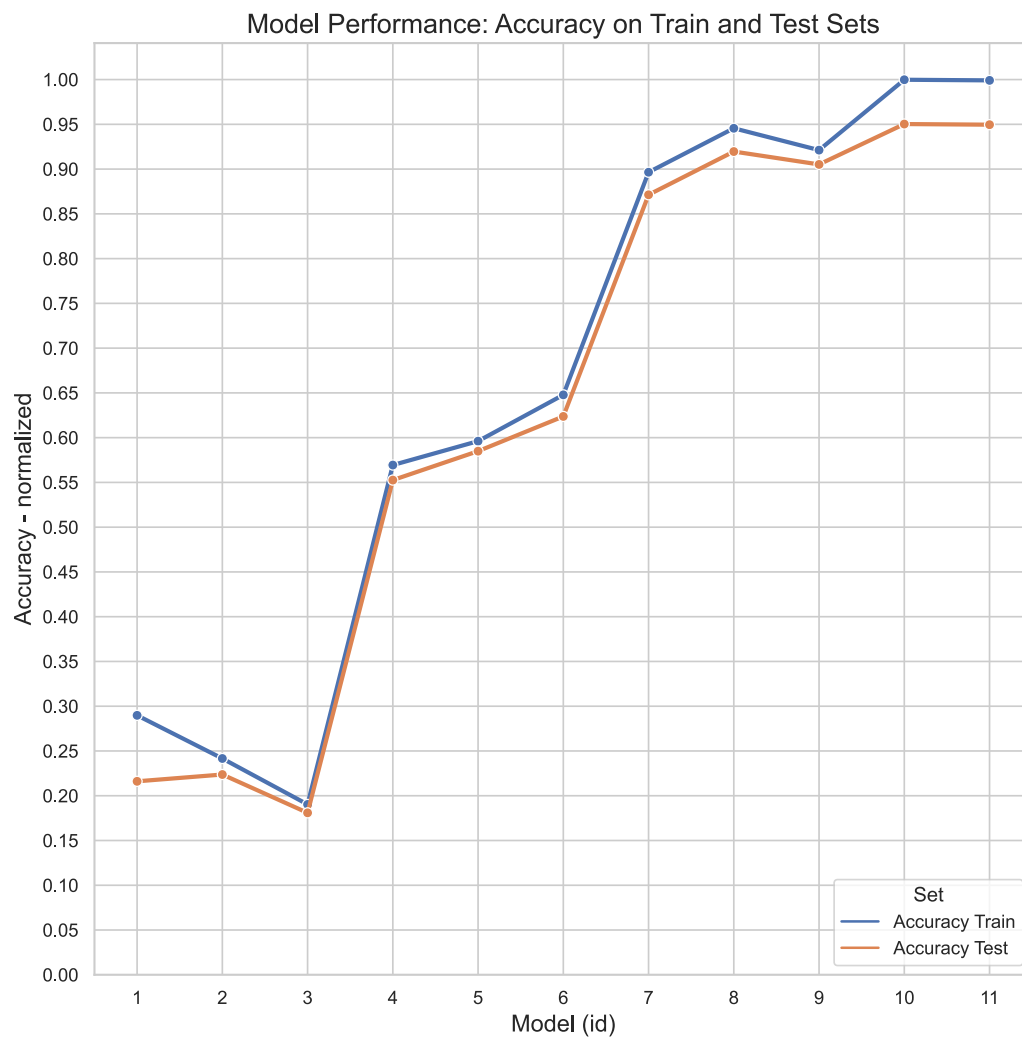


Figure 6.1: Comparison of model accuracy on the training and test sets.

based models achieve better performance in terms of accuracy, with *ResNet152* being the best performing model due to its greater depth (i.e., higher number of layers and residual blocks each network contains), which allows it to capture more complex and fine-grained features from the data. Furthermore, the addition of more layers on top of the ResNet architecture slightly improves performance. This is because the earlier layers are responsible for extracting high-level features, while the layers on top are used to map these features to the relevant classes.

Figure 6.2 shows the log-scaled confusion matrix for Model 10 (with respect to the test set). The logarithmic transformation $\log_e(1 + x)$ was applied to handle the wide range of values in the confusion matrix, where the diagonal entries (correct classifications) were significantly higher than the off-diagonal ones (misclassifications). This transformation makes the smaller differences between misclassifications more visible and allows for better interpretation of the model's performance on different classes.

The confusion matrix was analyzed to identify patterns of classification errors and to determine which classes are frequently misclassified with one another. The following insights were derived:

- **Total Errors per Class.** Each class was evaluated in terms of total errors, calculated as the sum of all misclassifications (off-diagonal elements) for each true class. The highest number of misclassifications occurred for several classes, suggesting that the model performs relatively consistently across different classes, but with a moderate number of errors.
- **Class Confusion Patterns.** For each true class, the class with which it was most often confused was identified. This helps to understand the specific class pairs that the model struggles to distinguish. For instance, Class 0 (corresponding to *Ceramic capacitor*) is most often confused with Class 3 (corresponding to *Tantalum capacitor*), while Class 1 (*Jumper*) is most confused with Class 6 (*Jumper cut*). This pattern of confusion can provide insights into the similarities between these classes in terms of features or representation within the dataset, potentially indicating areas for further model refinement.
- **General Trends.** The confusion matrix reveals that most errors are distributed across a small number of specific class pairs. This indicates that while the model has a relatively balanced performance, there are particular classes that require additional attention, either in terms of feature engineering, model tuning, or data augmentation, to reduce confusion.

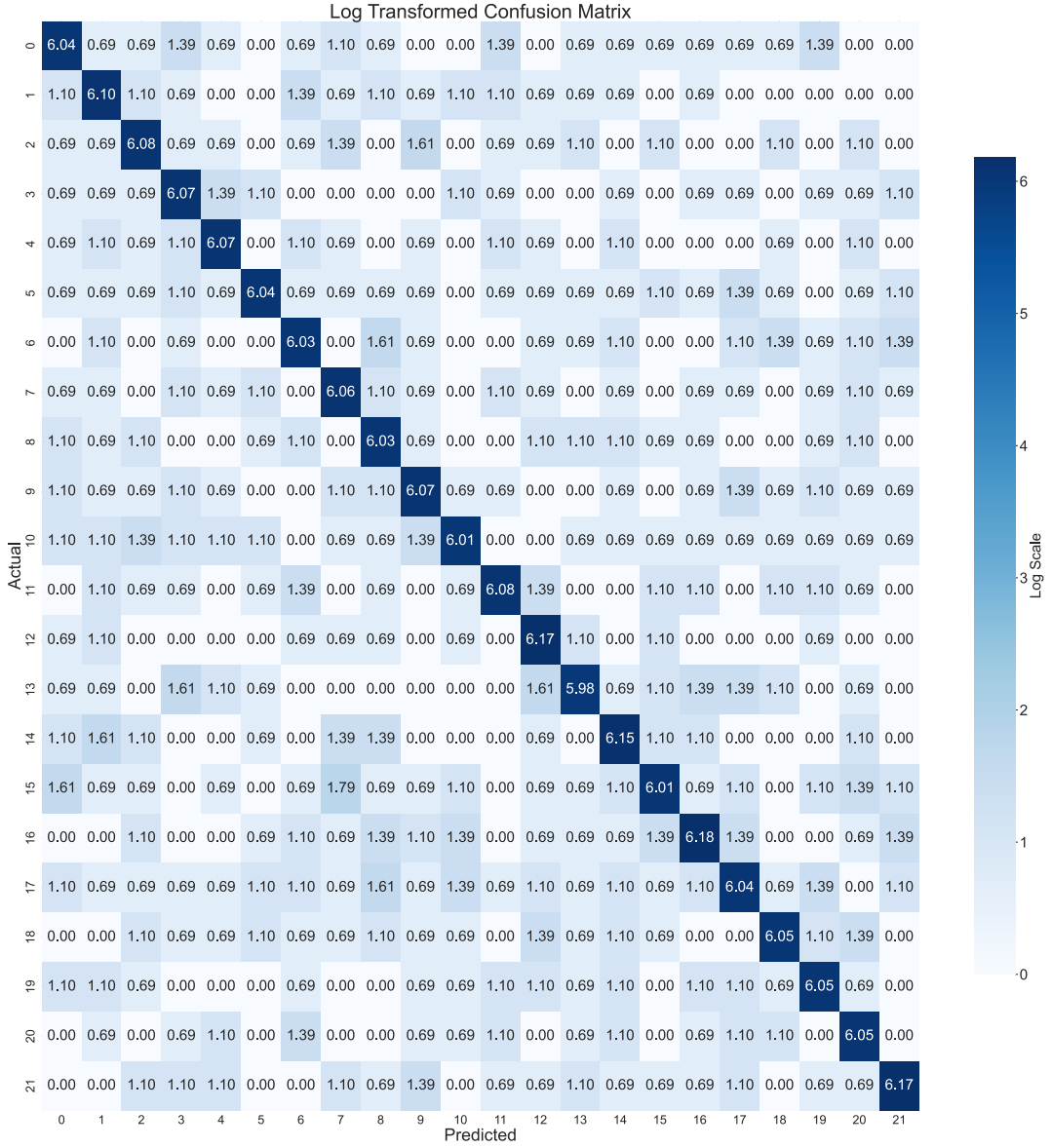


Figure 6.2: Log-scaled confusion matrix for Model 10. The values have been log-transformed using the natural logarithm of $1 + x$, where x is the original value in the matrix (1 has been added to avoid computing $\log 0$, which is not defined) to better highlight differences between predictions, especially when the accuracy, precision, and recall are very high.

Model ID	Accuracy (%)		Precision (%)		Recall (%)		F1-Score (%)	
	Train	Test	Train	Test	Train	Test	Train	Test
1	28.98	21.61	29.00	21.61	29.00	21.64	28.97	21.61
2	24.16	22.37	24.16	22.39	24.19	22.40	24.16	22.38
3	19.03	18.09	19.04	18.09	19.05	18.11	19.03	18.08
4	56.94	55.26	56.98	55.25	56.96	55.28	56.92	55.23
5	59.61	58.49	59.59	58.48	59.63	58.51	59.59	58.46
6	64.78	62.37	64.79	62.36	64.81	62.40	64.77	62.36
7	89.65	87.13	89.62	87.11	89.64	87.10	89.62	87.09
8	94.56	91.96	94.56	91.95	94.54	91.95	94.54	91.94
9	92.12	90.52	92.12	90.51	92.11	90.51	92.11	90.50
10	99.98	95.03	99.98	95.01	99.98	95.03	99.98	95.01
11	99.91	94.96	99.91	94.94	99.91	94.94	99.91	94.94

Table 6.2: Performance metrics (accuracy, precision, recall, and F1-score) expressed as percentages for the training and test sets across different models.

6.3. Adaptive Domain Randomization for Model Robustness

In the following analysis, Model 10, which was shown to exhibit the best performance in the previous subsection, has been selected for further evaluation.

To enhance the assessment of the effectiveness of the techniques described in Chapter 4, a new dataset was introduced. This dataset (hereafter referred to as *new dataset*) contains images that are significantly different from those (hereafter referred to as *old dataset*) used in the training phase described in Section 6.1, as well as from those used in the training phase of this evaluation. The images were collected under varied conditions and from PCBs that were not part of the original dataset used previously.

Table 6.3 shows the comparison between:

- The accuracy of the Baseline Model when trained, without parameter randomization, in the old dataset and when tested in the new dataset.
- The accuracy of the model when trained, with parameter randomization, in the old dataset and when tested in the new dataset. The distributions described in Subsection 4.1.2.1 are used to sample the parameter values. (See: Algorithm 4.1).

Details of the coefficients defining the distributions are listed in the Appendix.

Details of the Probability vector $\mathbf{p}$, where $\mathbf{p}_i$, probability that the i -th parameter is randomized, are listed in the Appendix.

- The accuracy of the model when trained, with parameter randomization and distribution coefficient optimization, in the old dataset and when tested in the new dataset. The distributions described in Subsection 4.1.2.1 are used to sample the parameter values. (See: Algorithm 4.2).

Details of the ranges and initial values used to define the search space of the coefficient distributions to be optimized are listed in the Appendix.

Details of the Probability vector $\mathbf{p}$, where $\mathbf{p}_i$, probability that the i -th parameter is randomized, are listed in the Appendix.

Model	Acc Test	Acc Train
Baseline (no rand.)	82.46%	99.98%
Rand., $\mathcal{U}$ dist	89.91%	91.45%
Rand., $\mathcal{T}$ dist	90.31%	92.65%
Rand., Univ $\mathcal{N}$ dist	89.25%	92.84%
Rand., Multiv $\mathcal{N}$ dist	87.88%	89.13%
Rand., $\mathcal{U}$ dist + opt	93.24%	94.22%
Rand., $\mathcal{T}$ dist + opt	91.39%	93.57%
Rand., Univ $\mathcal{N}$ dist + opt	91.85%	92.99%
Rand., Multiv $\mathcal{N}$ dist + opt	99.88%	99.93%

Table 6.3: Accuracy for models with different distributions used for parameter randomization.

Table 6.3 highlights how the the Baseline Model trained without parameter randomization exhibits significantly worse performance when applied to a dataset that is markedly different from the one used for training, which simulates the high heterogeneity of PCBs (see Subsection 1.2.2). This gap between the training and test sets, reflecting the disparity between the training dataset and real-world conditions, is gradually reduced through the application of *domain randomization* techniques. Further improvement is observed by implementing *adaptive domain randomization* techniques. Specifically, the use of a **multivariate normal distribution**, where the μ (mean) and Σ (covariance matrix) parameters are optimized, yields the best results. This is due to the fact that the multivariate distribution captures the relationships between different parameters.

The optimal values found for the coefficients of the distribution result in the following distribution:

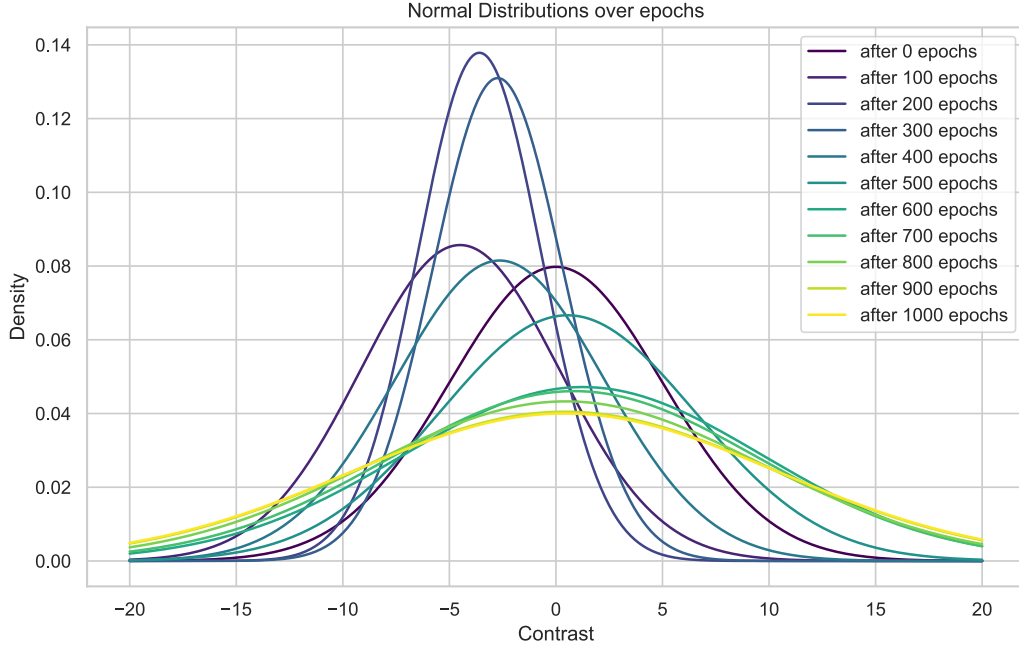


Figure 6.3: Normal distributions corresponding to the second component of the multi-variate normal distribution (*Contrast* parameter) during the optimization process, highlighting how its coefficients μ and Σ vary over epochs.

$$\mathcal{N}_7 \left(\begin{bmatrix} -0.117 \\ 0.407 \\ 0.984 \\ 1.227 \\ 0.810 \\ 49.788 \\ 0.369 \end{bmatrix}, \begin{bmatrix} 0.4 & 0.037 & -0.140 & -0.065 & -0.129 & -0.072 & 0.028 \\ 0.037 & 9.970 & 0.126 & -0.097 & -0.069 & 0.164 & 0.048 \\ -0.140 & 0.126 & 10.0 & 0.334 & -0.028 & -0.045 & 0.036 \\ -0.065 & -0.097 & 0.334 & 10.0 & 0.382 & 0.012 & 0.054 \\ -0.129 & -0.069 & -0.028 & 0.382 & 0.253 & 0.443 & 0.044 \\ -0.072 & 0.164 & -0.045 & 0.012 & 0.443 & 25.0 & 0.208 \\ 0.028 & 0.048 & 0.036 & 0.054 & 0.044 & 0.208 & 10.0 \end{bmatrix} \right)$$

As an example, Figure 6.3 illustrates the various normal distributions corresponding to the second component of the multivariate normal distribution (*Contrast* parameter) during the optimization process, highlighting how its coefficients μ and Σ vary over epochs.

Optimizer tuning Results shown in Table 6.3 are obtained with CMA-ES optimizer.

All the optimizers listed in Section 2.3.2 have been tested (using the Nevergrad python li-

brary⁴). Among this, the CMA-ES optimizer demonstrated the best overall performance, consistently outperforming the others. Random Search, on the other hand, yielded significantly lower results, showing much poorer performance compared to the other methods. The remaining optimizers, including Genetic Algorithms, PSO, and Bayesian Optimization, did not exhibit substantial variations in performance, generally reaching values slightly below those of CMA-ES. Notably, Simulated Annealing achieved the highest results among the non-CMA-ES optimizers, although still lower than CMA-ES.

⁴<https://facebookresearch.github.io/nevergrad/>

7 | Performance Optimization of Deployed Models in Industrial Systems

Once the architecture of the chosen model has been defined, with the primary goal of optimizing the evaluation metrics described in Section 6.1 —particularly accuracy— a further evaluation of the model is required to optimize additional parameters. These parameters become especially important when the model is deployed for inference in real-world industrial environments.

In practical applications, beyond the model's accuracy, other key factors play a critical role in determining the model's suitability for deployment. These include inference latency, memory usage, and energy consumption, all of which directly impact the efficiency and feasibility of using the model in an industrial setting. Optimizing these aspects ensures that the model is not only accurate but also capable of delivering fast and reliable predictions while operating under resource constraints typical of industrial environments.

This chapter focuses on optimizing the model's performance in such a context, analyzing its behavior when integrated into a real-time system, and discussing strategies for balancing the trade-offs between accuracy and other performance-critical parameters. By addressing these factors, the goal is to ensure that the model is both robust and efficient enough to meet the demands of industrial production processes.

7.1. Details of the Parameters to be Optimized

In the context of deploying machine learning and deep learning models in industrial systems, several key parameters must be considered and optimized to ensure efficient and reliable operation. These parameters are crucial not only for achieving high accuracy but also for meeting the performance requirements of real-time industrial applications. The following are the main parameters that require careful tuning:

- **Inference Latency.** The time required for the model to make a prediction on new data. In industrial systems, especially those requiring real-time decision-making, low latency is essential to avoid bottlenecks.
- **Memory Usage.** The amount of memory required by the model during inference. This is particularly important in embedded systems or hardware with limited resources, such as edge devices used for on-site data processing.
- **Energy Consumption.** The power consumption of the model during operation. Optimizing energy usage is critical in industrial systems where energy efficiency directly impacts operating costs, especially when models are deployed on devices with constrained power supplies.
- **Model Size.** The size of the trained model, which influences both memory usage and computational efficiency. Smaller models typically require fewer resources and can be deployed more easily on hardware-constrained environments.

By optimizing these parameters, the model can meet the specific demands of industrial environments, ensuring that it operates efficiently under real-world constraints while maintaining the accuracy needed for reliable predictions.

7.1.1. Distinction Between Model Size and Memory Usage

While both memory usage and model size are related to the efficiency and resource demands of a machine learning model, they refer to distinct aspects of the system's performance.

Model Size refers to the storage space required to save the model's architecture and parameters (such as weights and biases). This is typically measured in megabytes (MB) or gigabytes (GB). A smaller model size allows for easier deployment, especially on devices with limited storage. The model size depends on factors like the number of layers, parameters, and the precision (e.g., 32-bit floating point or 8-bit integer) used to store these values.

Memory Usage, on the other hand, refers to the amount of memory required by the system during inference or training. It includes not only the storage of the model parameters but also the memory allocated for intermediate computations (e.g., activations, gradients, temporary tensors). Memory usage is dynamic and can fluctuate depending on the batch size, input size, and the complexity of operations being performed at each layer. In environments with limited RAM, optimizing memory usage is critical to ensure the model can run efficiently without exceeding hardware limits.

Model size is a static measure of the space required to store the model, while memory usage is dynamic and encompasses all memory needs during computation, including the model itself and any temporary data structures required during inference or training.

Some techniques (e.g., pruning, a technique used to reduce the complexity of a model by setting a portion of the model's weights to zero) allows to use a sparse matrix structure for the model's weights, which can significantly reduce the *model size* when stored. Compression algorithms can take advantage of this sparsity to store the model more efficiently, reducing the overall storage requirements. However, *memory usage* during inference or training may not decrease in proportion to the reduction in model size. This is because most hardware architectures and computation frameworks are not optimized to perform operations directly on compressed or sparse matrices. Instead, they typically load and process the entire matrix (including the zero weights), which means that the memory allocated during computation remains comparable to that of a dense matrix: while the model size is reduced due to pruning, the memory usage during execution may not see a significant reduction, as the hardware may still process the full-sized matrices, ignoring the sparsity.

7.2. Details of the Techniques for Model Performance Optimization

In order to optimize the parameters described in Section 7.1, various techniques can be applied. These optimizations are crucial in industrial applications where resource efficiency is paramount.

However, it is important to note that while these methods aim to improve efficiency, they may come at the cost of model accuracy. Striking the right balance between maintaining acceptable accuracy and optimizing for other performance metrics is a key challenge.

In order to achieve this goal, two major families of approaches can be employed:

1. **Maximizing data reuse across multiple memory hierarchy levels.** This approach focuses on optimizing the flow and reuse of data at different levels of the memory hierarchy (e.g., cache, main memory, and external storage). By minimizing the need for repeated data transfers between these levels, the overall memory usage and latency during inference can be significantly reduced.
2. **Exploiting the tolerance of deep neural networks to approximation.** Deep neural networks (DNNs) are known for their robustness to approximations. This

property can be leveraged to reduce the precision of computations (e.g., through quantization or reduced bit-width operations) or by introducing approximations in the data or model architecture. These approximations can lead to considerable improvements in terms of memory and computational efficiency, with minimal impact on the model's performance, as long as the accuracy degradation is kept within acceptable limits.

7.2.1. Data reuse maximization

The maximization of data reuse has a dual objective.

1. **Reducing model latency.** This is particularly important for deep neural networks (DNNs) that include layers which are memory-bound. In these cases, the overall speed of the model is not limited by compute performance, but by the data transfer speed between the CPU and memory. By optimizing data reuse, the need for repeated memory transfers is reduced, significantly decreasing latency.
2. **Improving energy efficiency.** Maximizing data reuse also enhances the energy efficiency of the model. Each access to main memory consumes over 100 times more energy compared to instructions that only use register operands. Therefore, reducing the number of memory transfers, and maximizing the reuse of data already loaded in faster memory hierarchies (such as caches or registers), can lead to substantial energy savings during inference.

In the architecture of the chosen model (see Chapter 6), described in Chapter 3, the convolutional layers are particularly well-suited for data reuse. These layers involve heavy data transfers, and by applying data reuse strategies the overall efficiency and performance of these layers can be improved. While the final fully connected layer also benefits from this approach, its impact is less significant compared to the convolutional layers.

7.2.1.1. Fully connected layers

The Roofline Model The Roofline Model [34] is a visual performance model that helps to assess and optimize the performance of a system by examining the relationship between computation and memory access. It provides insights into whether a system is limited by its memory bandwidth (memory-bound) or by its computational capacity (compute-bound). The key metric in the Roofline Model is the operational intensity, defined as the ratio of floating-point operations (FLOPs) to memory accesses (bytes). This metric determines where a particular operation falls on the roofline, helping to identify whether

the bottleneck lies in memory transfers or computational power.

In the context of deep neural networks, fully connected layers often tend to be strongly memory-bound. These layers involve large matrix-vector multiplications, which require frequent memory transfers between the CPU and memory to load weights and activations. However, the computational intensity (i.e., the number of FLOPs) in fully connected layers is typically lower compared to convolutional layers, which results in memory transfers becoming the limiting factor in terms of performance.

The Operational intensity (OI) for a fully connected layer can be expressed as the ratio of FLOPs to the amount of data transferred (in bytes) from memory. For a fully connected layer with n_{in} input neurons and n_{out} output neurons, the OI is calculated as:

$$OI_{\text{fc}} = \frac{2 \times n_{\text{in}} \times n_{\text{out}}}{(n_{\text{in}} + n_{\text{out}}) \times \text{size of each element in bytes}}$$

Here, the term $2 \times n_{\text{in}} \times n_{\text{out}}$ represents the number of FLOPs required for a matrix-vector multiplication (each weight is multiplied by an input and added to the result), while $(n_{\text{in}} + n_{\text{out}}) \times \text{size of each element in bytes}$ represents the data transfer between memory and CPU, including both input activations and weights.

To mitigate the memory bottleneck in fully connected layers, one effective strategy is the use of **batching**. When batching is applied with a batch size of b , the operational intensity increases as more inputs are processed together. The formula for the OI with batching becomes:

$$OI_{\text{fc, batch}} = \frac{2 \times n_{\text{in}} \times n_{\text{out}} \times b}{(n_{\text{in}} + n_{\text{out}}) \times b \times \text{size of each element in bytes}}$$

By increasing the batch size b , the reuse of input activations and weights across multiple samples increases, thereby improving the operational intensity and reducing the frequency of memory transfers¹.

Thus, batching allows multiple input samples to be processed simultaneously, which increases the operational intensity by performing more computations relative to memory accesses. Instead of loading weights and activations for each individual input, batching enables the reuse of data across multiple inputs within the same batch, reducing the frequency of memory transfers. This leads to a higher operational intensity, pushing the

¹The operational intensity (OI) can reach a maximum value when the layer becomes fully compute-bound, where memory transfers are minimized. In practice, the OI for fully connected layers can go up to approximately 10 FLOPs/byte, depending on the hardware and batch size used.

performance of the fully connected layers closer to the Roofline’s compute-bound region.

7.2.1.2. Convolutional layers

Although convolutional layers tend to be more compute-bound compared to fully connected layers, data reuse can still play a crucial role in optimizing their performance. The high computational demand of convolutional layers arises from the sliding window operation applied to the input feature maps, where each filter kernel is convolved over the entire spatial dimensions of the input. By analyzing these layers through the Roofline Model, it is possible to assess the balance between compute and memory transfers, and identify opportunities to improve data reuse.

In convolutional layers, the primary computation involves six nested loops, which represent different dimensions of the convolution process. These loops iterate over:

- Batch size (B): loops over the batch of input samples.
- Output channels (C_{out}): loops over each output channel, corresponding to different filters applied.
- Input channels (C_{in}): loops over each input channel, representing the depth of the input feature maps.
- Output height (H_{out}): loops over the spatial height of the output feature map.
- Output width (W_{out}): loops over the spatial width of the output feature map.
- Kernel size ($K \times K$): loops over the height and width of the convolutional filter kernel.

The critical operation in each pass through these loops is the sum of products between a patch of the input feature map and the weights of the convolutional filter. Specifically, for each spatial position in the input, the filter slides across the input feature map and performs a dot product between the filter weights and the input patch, generating a single output activation.

In many modern CPU implementations, convolutional operations are often reduced to matrix-matrix multiplications (GEMM). This transformation helps optimize the computation by flattening the input feature maps and filter weights into matrices, which are then multiplied together. This matrix-based formulation takes advantage of highly optimized libraries for matrix operations, such as BLAS (Basic Linear Algebra Subprograms), which significantly speed up the convolutional process. By transforming convolutions into GEMM operations, not only is the computational load reduced, but the opportunities for

reusing input data and filter weights are maximized, further reducing memory traffic and increasing the overall efficiency of the convolutional layers.

The nested structure of these loops and the matrix-matrix multiplication reduction allow for a highly repetitive and structured computation. Data reuse in this context refers to reusing both the input feature maps and the filter weights during the convolution process. By minimizing the need to reload weights and input data from main memory for each iteration, data reuse can significantly reduce memory traffic and improve the overall efficiency of the layer.

Beyond **batching**, convolutional layers enable multiple data reuse patterns, which can be leveraged to further enhance performance. These patterns include [28]:

- **Weights Reuse.** In this pattern, the weights are kept stationary while the input feature map (IFmap) is shifted across them. This allows the reuse of both the filter weights and portions of the input patches across multiple spatial locations. By loading only one new *partial column* of the input at each step, memory traffic is reduced, and the efficiency of the computation is improved.
- **IFmap Reuse.** This pattern focuses on loading each input feature map patch only once and applying all filter weights to it before moving on to the next patch. This maximizes the reuse of the input data across all filters, reducing the need to reload the input patches multiple times from memory.

While convolutional layers are generally compute-bound, meaning that their performance is often limited by the number of FLOPs, the careful management of memory transfers and data reuse can push their performance closer to the roofline, ensuring more efficient use of computational resources.

7.2.2. Deep Neural Networks' tolerance to approximation exploitation

Deep neural networks (DNNs) are highly tolerant to approximations: changing slightly input data and/or internal computations often does not change the final results. This characteristic can be leveraged to improve the efficiency of the models without significantly compromising their performance.

Why DNNs are tolerant to approximation The above mentioned tolerance stems from several factors.

Tolerance from Stochastic Training DNNs are trained using stochastic gradient descent (SGD), which inherently deals with small deviations from the correct gradient direction during each mini-batch update. As a result, slight inaccuracies in the computations do not prevent the model from converging effectively.

Tolerance in Output Requirements In many applications, the exact output value of a DNN is not critical. For example, in multi-class classification tasks, the primary focus is on the predicted class (i.e., the index of the highest probability score, or the argmax) rather than the precise probability values for each class. This implies that minor approximations during computation rarely affect the final outcome.

Tolerance Due to Over-Parameterization Another reason for this tolerance is that DNNs are often over-parameterized, using many more parameters and computations than necessary to achieve a certain level of accuracy. This redundancy provides a buffer that allows the model to function effectively even when approximations are applied. As a result, the network architecture can be simplified — by reducing the number of layers, changing the type of layers (e.g., using depth-wise or group-wise convolutions), or downscaling the input— without a significant loss in accuracy.

The tolerance of deep neural networks to approximations enables a wide range of optimization techniques that can significantly reduce computational complexity and resource usage, while maintaining acceptable performance levels. Key techniques include pruning, quantization and use of efficient layers. Each of these techniques leverages the inherent robustness of DNNs to minor errors, allowing for more efficient use of memory, processing power, and energy.

7.2.2.1. Use of Efficient Layers

One effective approach to optimize the computational efficiency of neural networks is to replace standard layers with more efficient alternatives. These efficient layers maintain performance while significantly reducing the computational cost and memory requirements. Key examples of these techniques applied to convolutional layers are detailed in the following paragraphs.

7.2.2.1.1 Spatially Separable Convolutions

Spatially separable convolutions reduce the complexity of a standard $K \times K$ convolution by decomposing it into two 1D convolutions: a $K \times 1$ convolution followed by a $1 \times K$ convolution. This reduces the number of multiplications significantly.

The complexity of a standard convolution is given by:

$$\text{Complexity}_{\text{standard}} = K \times K \times C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}}$$

For a spatially separable convolution, the complexity is:

$$\text{Complexity}_{\text{spatially sep}} = 2 \times K \times C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}}$$

Thus, the *ratio* between the complexity of the standard and spatially separable convolutions is approximately:

$$\frac{\text{Complexity}_{\text{standard}}}{\text{Complexity}_{\text{spatially sep}}} = \frac{K}{2}$$

For $K = 3$, this leads to a 1.5x reduction in computational cost.

7.2.2.1.2 Depthwise Separable Convolutions

Depthwise separable convolutions [9, 13] further enhance efficiency by separating the convolutional process over the depth (channels) of the input. Instead of performing a standard convolution over all input channels and filters, a depthwise convolution applies a $K \times K$ filter to each input channel separately, followed by a *pointwise* (1x1) convolution that combines the channels. This approach reduces the number of operations compared to standard convolutions.

The complexity of a standard convolution is:

$$\text{Complexity}_{\text{standard}} = K \times K \times C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}}$$

For a depthwise separable convolutions, the process is split into two steps:

1. *Depthwise Convolution*: A $K \times K$ filter is applied independently to each input channel. The complexity is:

$$\text{Complexity}_{\text{depthwise}} = K \times K \times C_{\text{in}} \times W_{\text{out}} \times H_{\text{out}}$$

2. *Pointwise Convolution*: A 1×1 convolution is then applied to combine the channels, with complexity:

$$\text{Complexity}_{\text{pointwise}} = 1 \times 1 \times C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}}$$

The total complexity of the depthwise separable convolution is the sum of these two steps:

$$\text{Complexity}_{\text{depthwise sep}} = (K \times K \times C_{\text{in}} \times W_{\text{out}} \times H_{\text{out}}) + (C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}})$$

Thus, the *ratio* between the complexity of the standard convolution and the depthwise separable convolution is:

$$\frac{\text{Complexity}_{\text{standard}}}{\text{Complexity}_{\text{depthwise sep}}} = \frac{K \times K \times C_{\text{in}} \times C_{\text{out}}}{(K \times K \times C_{\text{in}}) + (C_{\text{in}} \times C_{\text{out}})}$$

This ratio shows the reduction in complexity achieved by using depthwise separable convolutions. For large values of C_{out} , the ratio approaches approximately:

$$\frac{\text{Complexity}_{\text{standard}}}{\text{Complexity}_{\text{depthwise sep}}} \approx K^2$$

This indicates that the depthwise separable convolution is approximately K^2 times more efficient than the standard convolution, especially when C_{out} is significantly larger than K^2 .

For example, with $K = 3$, $C_{\text{in}} = 32$, and $C_{\text{out}} = 64$, the complexity of a standard convolution is:

$$\text{Complexity}_{\text{standard}} = 3 \times 3 \times 32 \times 64 \times W_{\text{out}} \times H_{\text{out}} = 18,432 \times W_{\text{out}} \times H_{\text{out}}$$

Whereas the depthwise separable convolution's complexity is:

$$\begin{aligned} \text{Complexity}_{\text{depthwise sep}} &= (3 \times 3 \times 32 \times W_{\text{out}} \times H_{\text{out}}) + (32 \times 64 \times W_{\text{out}} \times H_{\text{out}}) \\ &= (288 \times W_{\text{out}} \times H_{\text{out}}) + (2,048 \times W_{\text{out}} \times H_{\text{out}}) = 2,336 \times W_{\text{out}} \times H_{\text{out}} \end{aligned}$$

Thus, in this example, the depthwise separable convolution reduces the number of operations by a factor of approximately 7.9x compared to the standard convolution.

7.2.2.1.3 Grouped Convolutions

Grouped convolutions divide the input channels into groups, applying separate convolutions to each group and then combining the outputs. This method reduces computational complexity by a factor proportional to the number of groups. Depthwise convolutions can be seen as a special case of grouped convolutions, where the number of groups equals the number of input channels. This technique can be used in architectures like ShuffleNet [42],

where grouping is combined with channel reshuffling to ensure that each output channel receives information from all input channels.

The complexity of a standard convolution is:

$$\text{Complexity}_{\text{standard}} = K \times K \times C_{\text{in}} \times C_{\text{out}} \times W_{\text{out}} \times H_{\text{out}}$$

For grouped convolutions, the input channels are divided into g groups, and each group is convolved separately. The complexity of a grouped convolution is:

$$\text{Complexity}_{\text{grouped}} = K \times K \times \left(\frac{C_{\text{in}}}{g} \right) \times \left(\frac{C_{\text{out}}}{g} \right) \times W_{\text{out}} \times H_{\text{out}} \times g$$

Simplifying:

$$\text{Complexity}_{\text{grouped}} = \frac{K \times K \times C_{\text{in}} \times C_{\text{out}}}{g} \times W_{\text{out}} \times H_{\text{out}}$$

Thus, the *ratio* between the complexity of the standard convolution and the grouped convolution is:

$$\frac{\text{Complexity}_{\text{standard}}}{\text{Complexity}_{\text{grouped}}} = g$$

This means that using g groups in a grouped convolution reduces the computational complexity by a factor of g . For example, if $g = 4$, the grouped convolution will be 4 times more efficient than the standard convolution.

7.2.2.1.4 Resolution Multiplier

The resolution multiplier (ρ) is a technique [13] used to reduce the input resolution, thereby reducing the computational complexity and the number of operations in the subsequent convolutional layers. By downscaling the input image, the resolution multiplier reduces the number of pixels the network processes, which in turn reduces the workload of each convolutional layer.

The reduction in complexity can be approximated as follows:

$$\text{Complexity}_{\text{resolution}} = \rho^2 \times \text{Complexity}_{\text{standard}}$$

where ρ is the scaling factor, typically ranging from 0.5 to 1, and $\text{Complexity}_{\text{standard}}$ is the complexity of the standard convolution. For example, if $\rho = 0.5$, the number of operations in each layer is reduced by a factor of 4.

This method is particularly useful in applications where high input resolution is not critical, allowing for significant computational savings at the expense of minor reductions in accuracy.

7.2.2.1.5 Width Multiplier

The width multiplier (α) is another technique [13] that reduces the computational complexity by uniformly decreasing the number of channels in each layer. This is effectively a form of manual pruning of channels, leading to reduced computational requirements in both the depthwise and pointwise convolutions.

The complexity of a depthwise separable convolution with a width multiplier α is:

$$\text{Complexity}_{\text{width}} = \alpha^2 \times \text{Complexity}_{\text{standard}}$$

where α is the width scaling factor (typically between 0.5 and 1), and $\text{Complexity}_{\text{standard}}$ is the complexity of the convolution without the width multiplier. For example, if $\alpha = 0.75$, the number of operations in both depthwise and pointwise convolutions is reduced by approximately 44%.

This method is useful in scenarios where the full capacity of the network is not required, allowing for a more efficient network at a minor cost in accuracy.

7.2.2.2. Pruning

Pruning [12] is a widely-used optimization technique aimed at reducing the computational complexity and memory footprint of neural networks by removing unimportant weights or entire neurons. The primary goal of pruning is to simplify the model while preserving its accuracy. This is particularly beneficial when deploying deep neural networks (DNNs) in resource-constrained environments.

The core idea of pruning is to identify and remove weights or neurons that have minimal impact on the network's performance. By removing these less important components, the model becomes smaller, faster, and more energy-efficient, which is crucial for real-time applications.

The technique can also be used in conjunction with fine-tuning or retraining to recover any potential loss in accuracy after pruning.

Pruning Strategies: Weights vs Activation Pruning

Weights Pruning Weights pruning focuses on eliminating individual weights that have little impact on the final output of the model. This method exploits the inherent *redundancy* present in over-parameterized neural networks, where many weights contribute minimally to the network’s predictions. The process typically involves analyzing the magnitude of the weights and removing those with values close to zero, under the assumption that such weights contribute minimally to the network’s predictions. By reducing the number of active weights, the model’s computational complexity and memory requirements are significantly reduced, particularly in dense layers where a large number of weights are involved.

Weights pruning is performed at training time, more complex (and potentially more beneficial) pruning approaches also involve re-training.

Activation Pruning Activation pruning [28], on the other hand, exploits the *sparsity* that often emerges in the outputs of neurons, particularly in networks that use activation functions such as ReLU, where many activations are zero. It focuses on the outputs of neurons rather than the weights. In this method, activations that are consistently close to zero across various layers of the network are pruned. The rationale behind this approach is that neurons with near-zero activations have little contribution to the network’s output, and removing them has minimal impact on performance.

Activation pruning can be seen as a dynamic form of pruning, where certain neurons or channels are selectively turned off during inference, reducing the number of computations needed. This approach is particularly effective in deep networks with many layers, where the activation of some neurons may become redundant or irrelevant for certain tasks. By pruning activations, both computational efficiency and memory bandwidth can be improved, especially in convolutional architectures.

Pruning Schedules: Constant vs Gradually Increasing

Constant Pruning In constant pruning, a fixed percentage of weights is pruned at each iteration throughout the entire training process. This approach is simple and can quickly lead to a reduction in model size. However, constant pruning can sometimes be too aggressive, particularly at the start of training, when the network still contains a large number of redundant parameters that might later prove important for model performance.

Gradually Increasing Pruning Gradually increasing pruning [12], on the other hand, aims to slowly ramp up the amount of pruning applied over time. Early in training,

only a small fraction of weights is pruned, allowing the network to adjust gradually. As training progresses and the network becomes more confident in its parameters, the pruning rate is increased. This method generally leads to better overall accuracy compared to constant pruning, as the network has more time to adapt to the reduced capacity and avoid overfitting to pruned weights.

Pruning Granularity: Structured vs Unstructured Pruning In pruning, granularity refers to the level at which the pruning process is applied within the neural network. This is closely related to the distinction between structured and unstructured pruning, where granularity determines whether the pruning targets individual weights or larger structures such as neurons, filters, or channels.

7.2.2.2.1 Unstructured Pruning (Fine-Grained)

Unstructured pruning, also referred to as fine-grained pruning, operates at the level of individual weights. This method provides a high level of flexibility, allowing for the removal of single, unimportant weights across the network. The resulting sparse model can achieve high compression rates, but often leads to irregular memory access patterns, making it more challenging to optimize on standard hardware. While unstructured pruning offers high flexibility, realizing computational speedups may require specialized hardware or software support for sparse matrices.

7.2.2.2.1.1 Magnitude-Based Pruning

Magnitude-based pruning [12] is one of the most straightforward and widely-used methods for unstructured pruning. The underlying principle is to remove weights that have the smallest magnitudes, under the assumption that weights with values close to zero contribute minimally to the network's output. The rationale is that these small weights do not significantly affect the gradients during backpropagation, and thus their removal will have a minimal impact on the model's overall performance.

In practice, this technique involves ranking all the weights in the network by their absolute values and pruning a certain percentage of the smallest weights. Magnitude-based pruning can be applied layer by layer or globally across the entire network. While this method is simple and effective, it often requires careful tuning of the pruning threshold or percentage to avoid excessive accuracy loss. The main benefit of this approach is that it can lead to significant sparsity in the model, reducing the number of active weights and making the model lighter.

One key challenge with magnitude-based pruning in unstructured settings is that it can produce sparse matrices, which are harder to optimize on standard hardware without specialized support for sparse matrix computations. However, the flexibility of removing individual weights makes it attractive for applications where model compression is prioritized over computational speedups.

7.2.2.2.1.2 Weight Saliency-Based Pruning

Weight saliency-based pruning [16] takes a more sophisticated approach, aiming to remove weights based on their overall importance to the network, rather than just their magnitude. The saliency of a weight refers to the change in the network's loss function if that weight were removed or perturbed. Weights that have little impact on the loss function are considered less important and are pruned. This approach attempts to quantify the contribution of each weight to the model's performance, providing a more refined criterion for pruning.

Saliency can be determined in several ways, including methods which approximate the effect of weight perturbations through a *Taylor expansion*. Specifically, the change in the loss function ΔL resulting from a small perturbation of the weight w_i can be approximated as follows:

$$\Delta L(w_i) \approx \frac{\partial L}{\partial w_i} \Delta w_i + \frac{1}{2} \frac{\partial^2 L}{\partial w_i^2} (\Delta w_i)^2$$

where $\frac{\partial L}{\partial w_i}$ represents the first-order derivative (gradient) of the loss function with respect to the weight w_i , and $\frac{\partial^2 L}{\partial w_i^2}$ represents the second-order derivative (Hessian), capturing the curvature of the loss function around w_i . This quadratic approximation evaluates the importance of each weight by considering both the gradient and the curvature of the loss landscape.

Compared to magnitude-based pruning, weight saliency-based pruning is more computationally intensive, as it requires evaluating the importance of each weight based on its influence on the network's output.

The primary advantage of weight saliency-based pruning is that it generally leads to less accuracy degradation than magnitude-based methods, as it targets weights that have minimal impact on performance. However, due to its complexity, it is often used in scenarios where accuracy preservation is critical, and the additional computational cost is justified.

7.2.2.2.2 Structured Pruning (Coarse-Grained)

Structured pruning, also known as coarse-grained pruning, focuses on larger structures such as entire neurons, channels, filters, or layers. This type of pruning maintains regular data structures, which are easier to handle on standard hardware, leading to more predictable memory access and better computational efficiency. While structured pruning is less granular, it often provides a better trade-off between compression and practical implementation on common hardware, making it more feasible for real-world deployment [38].

7.2.2.2.2.1 Block-Based Pruning

Block-based pruning [38] is a structured pruning technique where weights are grouped together based on the underlying hardware parallelism. Instead of pruning individual weights, entire blocks of weights are pruned together, often using an aggregated metric like the root mean square (RMS) value of the group to determine its importance. This approach allows for better hardware utilization, especially on SIMD-based processors, where multiple weights can be loaded and processed together, reducing latency.

For instance, in a $K \times K$ convolution, weights can be divided into blocks of size $b \times b$, and pruning decisions are made for each block based on the importance of the group as a whole. This results in a more regular access pattern and allows for optimizations such as loading weight blocks with a single SIMD instruction and reducing the memory footprint by using formats like Compressed Sparse Row (CSR).

The main benefits of block-based pruning include:

- Reduced memory footprint due to smaller sparse representations.
- Improved hardware utilization as weight groups can be processed in parallel.
- Faster inference times on SIMD-based architectures.

This method is particularly effective for processors where cache misses are not the primary source of inefficiency but maximizing parallelism is key.

7.2.2.2.2.2 Node-Based Pruning

Node-based pruning [38], also known as channel-based or neuron pruning, targets entire nodes (neurons in fully connected layers or feature maps in convolutional layers) rather than individual weights. By eliminating entire nodes, this approach effectively reduces the

size of the network while maintaining its dense structure. The final model is equivalent to a neural network with fewer neurons or channels, leading to reduced memory usage and computational complexity.

In convolutional layers, node-based pruning eliminates entire feature maps, which can be thought of as removing rows from the weight matrix. This shrinks the overall network, reducing the number of Multiply-Accumulate (MAC) operations and the memory footprint. In fully connected layers, the process eliminates entire neurons, simplifying the layer without introducing sparsity.

The key advantages of node-based pruning include:

- A smaller weight matrix and feature matrix, reducing memory usage.
- Fewer MAC operations, leading to faster inference times.
- A dense model structure that can still benefit from hardware optimizations.

An advanced form of node-based pruning can be achieved through the use of *mask layers*, which are auxiliary layers that apply a multiplicative mask to each neuron or feature map. The idea is to force certain nodes to become inactive during training by gradually driving their weights toward zero. This can be achieved using a regularization-based approach, such as *L1 regularization* (also known as Lasso).

Given a neural network with weight matrix W for a fully connected layer or a convolutional kernel, a mask layer M can be applied element-wise to the weight matrix:

$$W' = W \odot M$$

where W' is the pruned weight matrix and $\odot$ represents element-wise multiplication. Each entry in the mask matrix M corresponds to a neuron or feature map, and its value is either

0 (indicating that the neuron is pruned) or 1 (indicating that the neuron remains active)²:

$$M = (\alpha_{i,j}) \quad \text{with} \quad \alpha_{i,j} \in \{0, 1\}, \forall i, j \quad (7.2)$$

The mask values are learned during training by adding an L1 regularization term to the loss function, which encourages the mask values to move towards zero for unimportant nodes. The objective function is modified as follows:

$$L_{\text{total}} = L_{\text{original}} + \lambda \sum_i \|M_i\|_1$$

where L_{original} is the original loss function (e.g., cross-entropy), λ is a hyperparameter that controls the strength of the regularization, and $\sum_i \|M_i\|_1$ is the L1 norm of the mask values, penalizing non-zero entries in M .

During training, the regularization term pushes the values in the mask matrix M toward zero for neurons that contribute little to the final performance. Once training is complete, nodes corresponding to mask values close to zero are pruned entirely, and the mask layer can be removed. This results in a smaller network without introducing sparsity:

$$W' = W \odot M, \quad \text{where } M_i = 0 \text{ for pruned neurons.}$$

7.2.2.2.3 Balancing Granularity

The decision between structured and unstructured pruning depends largely on the target application and hardware constraints. Unstructured pruning maximizes compression and is suitable for memory-constrained environments, while structured pruning tends to strike a balance between compression and hardware efficiency, making it ideal for deployment on general-purpose hardware.

²It is not easy to train a binary variable α using Gradient Descent.

Since it can assume two values, small values updates have no effects (usually, learning rate lr is very small if compared to α).

A possible solution consists in defining a real value $\beta_{i,j}$, such that: $\alpha_{i,j} = H(\beta_{i,j})$, where H is a binarization function (e.g.: Heaviside, hysteresis, trainable threshold). Thus, (7.2) becomes:

$$M = (\beta_{i,j}) \quad \text{with} \quad \alpha_{i,j} \in \{0, 1\}, \forall i, j \quad \text{and} \quad \alpha_{i,j} = H(\beta_{i,j}) \quad (7.1)$$

$\beta_{i,j}$ is updated each iteration and $\alpha_{i,j}$ is updated on the fly.

An issue is related to the fact that the gradient of H is 0 almost everywhere, which prevents it from propagating from $\alpha_{i,j}$ to $\beta_{i,j}$.

By using the **Straight-Through Estimator (STE)** technique, $y(x) = H(x)$ is approximated with $y(x) = x$ when gradients are back-propagated (i.e., gradient of $\alpha_{i,j}$ is copied to $\beta_{i,j}$).

Layer Criticality: Not all layers are equally important

In the process of pruning, it is essential to recognize that not all layers in a neural network contribute equally to the model's overall performance and efficiency. The criticality of a layer can depend on several factors, such as its position within the network, the number of parameters it holds, and its role in the feature extraction or decision-making process.

Fully Connected vs. Convolutional Layers Fully connected layers, often located near the end of the network, tend to be memory-bound and contribute heavily to the overall model size. Pruning these layers can significantly reduce memory usage without drastically affecting accuracy. On the other hand, convolutional layers, especially those in the early stages, are critical for feature extraction and tend to be more compute-bound. These layers are less tolerant to aggressive pruning, as they directly impact the quality of the extracted features.

Energy and Latency Considerations In hardware-aware pruning, the criticality of a layer can also depend on factors like energy consumption and memory access patterns. Layers with high memory access requirements or those that are computationally intensive contribute more to energy consumption and latency during inference. By prioritizing the pruning of less critical layers [36], such as later fully connected layers or intermediate feature maps, it is possible to achieve a balance between computational efficiency and accuracy.

Prioritizing Pruning The goal of efficient pruning is to reduce the computational load while preserving accuracy. This often involves assigning different pruning thresholds to different layers based on their criticality. For instance, larger layers that dominate memory and computational requirements can be pruned more aggressively, while critical layers responsible for essential features may be pruned with more caution to avoid accuracy loss.

7.2.2.3. Quantization

Quantization is an effective technique for reducing the computational and memory requirements of deep learning models by lowering the precision of the weights and activations [7, 11, 17]. Standard deep learning models typically use 32-bit floating-point (float32) precision for both training and inference. However, both tasks, especially inference, can be performed with reduced precision, such as 16-bit or 8-bit integers, with minimal accuracy

loss. This is possible due to the inherent error tolerance of deep learning models.

Quantization can be applied to weights only or to both weights and activations. Several quantization formats are used, including reduced precision floats (e.g., 16-bit mini-floats) and low-precision integers (e.g., 8-bit integers). In some cases, custom data formats can also be utilized, depending on the target hardware.

The main benefits of quantization are twofold [11, 17]:

- **Memory Efficiency:** Quantization significantly reduces the memory required for storing models. For example, converting from 32-bit floats to 8-bit integers reduces the memory footprint by a factor of four. This reduction applies both to the weights stored in the model and the data transfers between memory and processors.
- **Computational Efficiency:** On hardware platforms that support low-precision arithmetic, quantization can improve the efficiency of computations. Arithmetic operations using low-precision integers are faster and more energy-efficient than operations using full-precision floating-point numbers.

Data Precision and Format Choices

When quantizing a neural network, one of the most important decisions is the choice of data precision and format. The precision of the weights and activations directly affects both the memory footprint of the model and its computational efficiency. Lower precision formats reduce memory usage and speed up computations but may introduce a small loss in accuracy. Conversely, higher precision formats retain more information but come with increased memory and computational costs.

The various quantization formats can be categorized into two main types [7]:

- **Smaller Float Formats**
 - **16-bits minifloat**
 - * Computation efficiency improved (some hardware can natively compute on float16).
 - * High accuracy maintained.
- **Low Precision Integer**
 - **8-bits integer**
 - * Memory reduced by 75%.

- * Data transfer energy reduced by 75%.
- * Minimal accuracy drop.
- * Computational efficiency improved.
- **16-bits integer**
 - * May be used for activations only (weight size is significantly larger than activation size).
- **Less than 8-bits (sub-byte)**
 - * **4-bits integer**
 - Requires complex packing/unpacking of data or custom hardware support
 - * **1-bit integer (BINARY NNs)**
 - Memory reduced by 96.875%.
 - MAC operations can be replaced by much simpler bit manipulations³. Significant speed-up and energy efficiency improvement, also on general purpose HW: all general purpose CPUs have instructions for bit-wise operations on words, many have popcount operations too.
 - Possible accuracy drop.

Integer Quantization: Fixed Point, Dynamic Fixed Point Two common forms of integer quantization are *Fixed Point* and *Dynamic Fixed Point*.

1. In **fixed point quantization**, real numbers are mapped to integer representations using a fixed scaling factor across the entire tensor. The scaling factor, denoted as Δ , is chosen to balance the dynamic range of the data and minimize precision loss. This method is computationally efficient and is widely supported by hardware accelerators designed for integer arithmetic.

Given a floating-point value x , the corresponding fixed-point integer value x_{int} is computed as:

$$x_{\text{int}} = \text{round} \left(\frac{x}{\Delta} \right)$$

³In particular:

- Multiplication of binary weights represented in this format becomes a binary XNOR.
- Accumulation reduces to counting the number of 1s (bitcount or popcount).

where Δ is the fixed scaling factor. After performing operations in integer space, the values can be de-quantized back to floating-point for further processing if necessary:

$$x_{\text{float}} = x_{\text{int}} \times \Delta$$

Fixed-point quantization [10] is simple and effective, particularly in applications where the distribution of weights and activations does not vary significantly across different layers.

2. **Dynamic fixed point quantization** extends the fixed-point approach by allowing the scaling factor Δ to vary across different layers or even different channels within a layer. This technique [7] adapts the precision to the varying dynamic ranges of different parts of the network, which can significantly reduce precision loss in layers with very small or very large weight values.

In dynamic fixed point quantization, the scaling factor is adjusted based on the range of the data in each layer or channel. This allows the model to represent the data more accurately compared to a global fixed scaling factor. For each layer, the scaling factor Δ_{layer} is computed dynamically to optimize the representation of the data:

$$x_{\text{int, layer}} = \text{round} \left(\frac{x_{\text{layer}}}{\Delta_{\text{layer}}} \right)$$

This approach is particularly useful for models with layers that have highly varying weight distributions, such as convolutional neural networks (CNNs), where earlier layers might have weights of different magnitudes compared to deeper layers.

Dynamic fixed point quantization provides greater flexibility and precision, making it an effective method for quantizing deep neural networks without significant loss of accuracy.

- (a) **Non-uniform quantization** provides an alternative approach by applying different quantization scales to different ranges of data. Instead of using a single uniform scale for the entire dataset, non-uniform quantization allows certain regions (e.g., smaller or larger values) to have finer precision. This is particularly useful in networks where certain activations or weights vary significantly in magnitude (**distributions of weights and activations are almost never linear**). Non-uniform quantization can be considered an extension of the dynamic fixed point approach, where the scaling factor Δ changes within layers or data ranges, providing a more accurate representation of the underlying data.

- i. **Log-domain quantization** is a specific type of non-uniform quantization where the data is quantized using a logarithmic scale instead of a linear one [19]. It maps more quantization levels close to 0 (**where most weights falls**). Logarithmic quantization ensures that the differences between small and large values are better captured, reducing the loss of precision in cases where data spans multiple orders of magnitude.

In log-domain quantization, the floating-point value x is transformed using a logarithmic function:

$$x_{\log} = \log(x)$$

The resulting value is then quantized similarly to fixed-point quantization. After the operations are performed, the values are transformed back to the original domain using the exponential function:

$$x_{\text{float}} = \exp(x_{\log})$$

- **Base-2 log quantization** is a hardware-friendly approach that leverages the properties of logarithms to simplify computations. Instead of using standard floating-point or integer operations, log-domain quantization transforms values using the base-2 logarithm, which allows multiplications to be replaced by shifts and sums—operations that are much more efficient in hardware.

Given a value x , base-2 log quantization works by first computing the logarithm in base 2:

$$x_q = \text{round}(\log_2(x))$$

This transformation turns the input values into exponents of base 2. The advantage of this approach is that multiplications can be converted into additions of the logarithmic values, and similarly, divisions become subtractions. The result can be shifted back to its original domain through a bitwise shift operation:

$$x_{\text{dequant}} = 2^{x_q}$$

In hardware implementations, the key benefit of log-domain quanti-

zation is the replacement of multiplications with bitwise shifts. Since each multiplication in the log domain can be reduced to a series of additions, bit shifts, and sums, it dramatically reduces the computational cost. For instance, multiplying two numbers X and W in the standard domain would require floating-point multiplication, but in the log domain, it simplifies to:

$$Z' = 1 \ll (X_q + W_q)$$

where $\ll$ denotes a bitwise shift operation.

This makes base-2 log quantization particularly suitable for hardware-constrained environments, as the operations involved—bitwise shifts and additions—are much less expensive in terms of power consumption and computational complexity compared to traditional multiplication and division.

While log-domain quantization is highly efficient, it does require careful rounding, especially when transforming values back into the original domain. Returning from the logarithmic space involves identifying the position of the most significant bit (the leftmost 1) and rounding the result to avoid large errors in precision.

For example, a multiplication in standard arithmetic:

$$X \times W = Z$$

in the log domain, becomes:

$$\log_2(X) + \log_2(W) = \log_2(Z)$$

which can be efficiently computed using bitwise shifts:

$$Z_q = \text{Bitshift}(X_q + W_q)$$

This operation is much faster and consumes less power on hardware that supports bitwise shifts and additions.

- **Affine Quantization** extends dynamic fixed point quantization by introducing a zero-point z , in addition to the scaling factor Δ , to handle asymmetries in the data distributions, especially in activations. This approach is partic-

ularly useful when working with activation functions such as ReLU, which output only positive values, thus requiring an asymmetric quantization.

In affine quantization, the transformation from real values to integer representations is given by:

$$a_{\text{quant}} = \text{round} \left(\frac{a_{\text{real}}}{\Delta} + z \right)$$

Here, z is the zero-point and Δ is the scaling factor. The zero-point allows shifting the range of the quantization to better capture the characteristics of the data.

For weights, a symmetric quantization is typically used with $z = 0$, as weights tend to have a symmetric distribution around zero. For activations, however, the distribution is often skewed towards positive values (e.g., after ReLU), making an asymmetric quantization with $z \neq 0$ more effective.

The basic method for determining Δ and z ensures that the entire range of the tensor a (from $a_{\min}$ to $a_{\max}$) is correctly mapped. For example, for an 8-bit quantization (e.g., $[-127, 127]$), the scaling factor Δ and zero-point z are computed as follows:

For activations:

$$\Delta = \frac{a_{\max} - a_{\min}}{2^n - 1} \quad \text{and} \quad z = \frac{a_{\max} + a_{\min}}{2}$$

For weights:

$$\Delta = \frac{2 \times \max(|a_{\min}|, |a_{\max}|)}{2^n - 1} \quad \text{and} \quad z = 0$$

In some cases, certain tools opt for a smaller Δ , which might result in poorer approximation for outlier weights, but offers higher precision around z . This trade-off can sometimes lead to higher overall accuracy, even if the extreme values are less precisely quantized. In this case, the quantization process consists of two steps:

- (a) First, calculate the intermediate quantized value:

$$\text{tmp} = \text{round} \left(\frac{a_q}{\Delta} + z \right)$$

- (b) Then, clamp the value to ensure it fits within the desired range (e.g.,

$[-127, 127]$ for 8-bit):

$$a_{\text{quant}} = \min(\max(-127, \text{tmp}), 127)$$

Binary Quantization: Weights and Activations Representation Two common forms of binary representations for weights and activations are the following [27].

1. $w, x \in \{-1, 1\}$.

In practice, -1 is represented by logic-0 for HW simplicity.

2. $w, x \in \{-\alpha, \alpha\}$.

α is set per-tensor.

The conversion is performed using the *sign()* function.

Fixed vs. Mixed Precision The bit-width N can be the same for each layer (channel), or may be different.

1. In **fixed precision**, N is the same for the entire network.
2. **Mixed precision** uses a different N layer-wise or channel-wise. This approach [22, 24] may lead to potentially higher compression for the same level of accuracy.

Various search algorithms (greedy heuristics, simulated annealing, genetic, etc.) can be used to determine the optimal bit-width N per each layer.

N_{weights} can also be different from $N_{\text{activations}}$, since in general they can have very different distributions.

Quantization Strategies

Figure 7.1 illustrates the different quantization approaches, starting from a float model [7]. The diagram highlights two main approaches: Post-Training Quantization and Quantization-Aware Training. Both methods can be applied either to the model's weights alone, or to both weights and activations, as depicted in the lower sections of the diagram.

Weights only vs. Weights + Activation Quantization

1. **Weight only quantization**

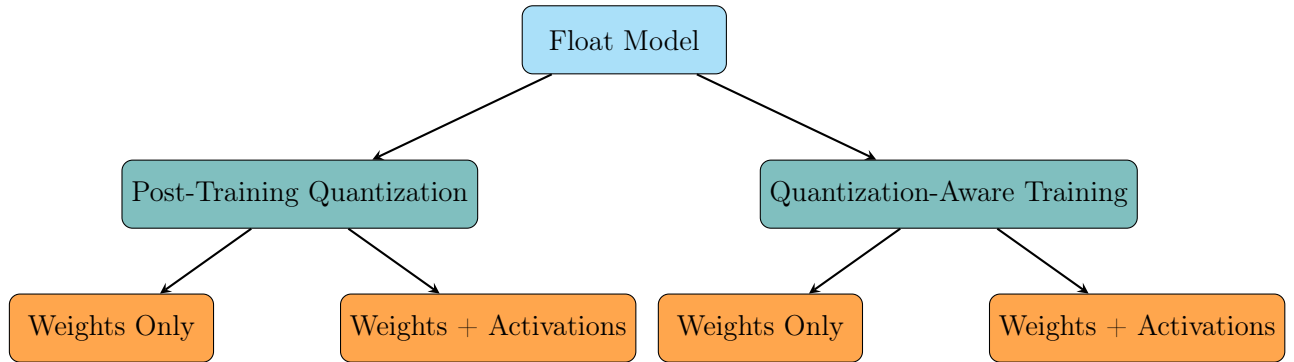


Figure 7.1: Diagram illustrating the two main quantization approaches.

- (a) **Basic version:** during inference, weights are initially stored as `int8` and are then unpacked to `float32` when the actual computations take place.

Once the weights are loaded into memory, they are immediately unpacked into 32-bit format, and the converted versions are cached to reduce latency during subsequent inferences.

- Benefits:
 - model size reduction on disk
- Drawbacks:
 - no working memory reduction
 - no latency reduction (additionally: 1st inference surely slower, due to conversion)

- (b) **Advanced version** (Dynamic range quantization): weights kept in quantized format, activations are quantized on the fly.

Operations are performed with integer HW.

- Benefits:
 - model size reduction on disk
 - working memory reduction
- Drawbacks:
 - slower/faster depending on HW

2. Weight + Activation quantization

Quantized version used for both storage and computation.

Ideally⁴, full-integer layer.

- Benefits:
 - model size reduction on disk
 - working memory reduction
 - latency reduction (ops executed in `int`)
- Drawbacks:
 - activation tensors cannot be quantized on the fly: the scaling factor Δ and the zero-point z for the activations must be determined ahead of time.

While the minimum and maximum values ($w_{\min}$ and $w_{\max}$) for each weight tensor can be directly extracted from the float model after training, the same is not true for activations. Activation tensors depend on the inputs, meaning that their minimum and maximum values ($x_{\min}$ and $x_{\max}$) vary depending on the data being processed.

For post-training quantization, a solution involves providing the converter with a set of representative inputs. These inputs are used to compute $x_{\min}$ and $x_{\max}$, which in turn allow the determination of Δ and z for both input/output and activation tensors. Typically, a small set of inputs, ranging from 100 to 1000 examples, is sufficient to set these parameters accurately. To handle potential outliers, values outside the computed range can be clamped to ensure they stay within acceptable limits.

Post-Training Quantization vs. Quantization-Aware Training

1. Post-training quantization

Conversion performed after training.

- Benefits:
 - much simpler
 - can be applied to pre-trained models
 - with weights only quantization, can be applied even with no input data available

⁴There may be unsupported ops, which are executed in `float`

- Drawbacks:
 - potentially significant accuracy drop in complex tasks

2. Quantization-aware training

Quantization simulated during training.

The optimizer can take into account the limited set of values the weights/activations can assume.

- Benefits:
 - avoid accuracy drop
- Drawbacks:
 - re-training may not be possible
 - the use of low-precision integers during back-propagation makes impossible small values updates.

A possible solution consists in the so-called *Fake Quantization*: data are kept in `float`, but the limited set of values that a value a can assume is simulated using the *round* function:

$$\begin{aligned} \textbf{Real Quant} \quad a_q &= \text{round} \left(\frac{a}{\Delta} + z \right) \\ \textbf{Fake Quant} \quad a_{fq} &= \Delta \cdot \text{round} \left(\frac{a}{\Delta} + z \right) - z \end{aligned}$$

The gradient of a_{fq} is back-propagated to a .

An issue arises from the fact that the gradient of *round* is 0 in all differentiable points. A possible solution involves the use of the **Straight-Through Estimator (STE)** technique: quantization is used only for forward pass, being replaced with a differentiable function (e.g., *identity* function) in the backward pass.

7.2.2.3.1 Weights clustering

Weights clustering [6, 11, 31] is a form of quantization where, instead of reducing the precision of each weight or activation, the number of distinct values allowed is reduced. The core idea is to replace each `float` weight with its closest allowed value, known as the cluster centroid, and store only the centroid values along with an index for each weight.

These centroids are stored in a separate structure (codebook, look-up table), and the model stores only the indexes of the centroids rather than the actual weight values.

Benefit: storage size reduction The storage size reduction achieved through weights clustering can be significant, especially when the number of centroids is much smaller than the number of original weights in the matrix.

The size of the original weight matrix can be calculated as:

$$\text{Size}_{\text{original}} = \text{Number of elements} \times \text{Bit width per element}$$

For a float32 model with a matrix of size $m \times n$, the storage size is:

$$\text{Size}_{\text{original}} = m \times n \times 32 \text{ bits}$$

When applying weights clustering, the original `float32` values are replaced with indexes that point to centroids stored in a codebook. The total storage size for a clustered matrix is the sum of two components:

1. **Codebook storage:** The size required to store the centroids.
2. **Index storage:** The size required to store the indexes pointing to the centroids.

The size of the codebook is:

$$\text{Size}_{\text{codebook}} = k \times 32 \text{ bits}$$

where k is the number of centroids. The size for storing the indexes is:

$$\text{Size}_{\text{indexes}} = m \times n \times \log_2(k) \text{ bits}$$

The $\log_2(k)$ term represents the number of bits required to encode the index of each weight's corresponding centroid. In a binary system, $\log_2(k)$ provides the minimal number of bits needed to distinguish between k unique values. For example, with 4 centroids ($k = 4$), $\log_2(4) = 2$ bits are required to encode each index.

The storage reduction ratio can be calculated as:

$$\text{Reduction Ratio} = \frac{\text{Size}_{\text{original}}}{\text{Size}_{\text{clustered}}}$$

Drawback: possible slower inference Since each weight is replaced by an index pointing to a centroid, the model needs to perform an additional lookup operation during inference. Specifically, the system first retrieves the index stored in the weight matrix and then uses it to access the corresponding centroid value from the codebook. This two-step process can introduce additional latency, particularly when the model size is large, or when the hardware lacks optimized support for these types of operations.

To reduce this impact, it is possible to avoid storing the indexes and instead directly replace each weight with its closest centroid. The model size is still reduced, as a compression algorithm (e.g., zip) can take advantage of the repeated values.

Determining Centroids During Training The process of determining centroids during training typically follows a structured approach, with the most widely used algorithm working as follows.

1. First, the centroids are initialized using a pre-trained float model. There are several methods available for initializing the centroids, with one of the most common being the *Linear* method; in this approach, the centroids are uniformly spaced between the minimum and maximum values of each weight tensor, providing a good starting point for the clustering process.
2. Once the centroids are initialized, a *k-means clustering* algorithm is applied to refine the centroids and obtain their initial values.
3. After this step, the model undergoes a fine-tuning phase.
 - (a) During the forward pass, each weight is replaced with its closest centroid.
 - (b) In the backward pass, the gradients of all weights assigned to the same cluster are summed together and used to update the centroid for the next iteration.

This process ensures that the centroids are continuously adjusted based on the training data, leading to a more accurate clustering of the weights and, ultimately, a more efficient model.

7.3. Evaluation of Optimization Techniques in Real-World Hardware Scenarios for Performance Optimization

In this section, the effectiveness of the optimization techniques outlined in Section 7.2 when applied to real-world hardware environments has been evaluated. The goal is to assess how these techniques, implemented in the context of machine learning models, influence key performance metrics such as inference time, memory usage, and energy consumption, as discussed in Section 7.1.

To perform this evaluation, a series of experiments were conducted using the hardware available at the company⁵.

These techniques are applied with reference to Model 10 (see footnote⁶), ensuring that the optimization strategies are tailored to the specific architecture and requirements of the model used in the PCB testing process.

30 different optimized versions have been evaluated.

- Reference Model: Model 10
- Optimized version 1:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Width Multiplier ($\alpha = 0.8$), Constant Pruning (20%), Magnitude-Based Pruning.
- Optimized version 2:
Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Depthwise Separable Convolutions, Resolution Multiplier ($\rho = 0.75$), Constant Pruning (40%), Block-Based Pruning (4x4).
- Optimized version 3:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Grouped Convolutions ($G = 32$), Gradually Increasing Pruning (10%-60%), Weight Saliency-Based Pruning.
- Optimized version 4:

⁵i.e., the hardware which manages the PCB testing machines (specifically the Flying Probe system)

⁶When a technique is applied to the model, it results in a (potentially) optimized version of the original model. These will be referred to as *Optimized versions* to indicate the various versions of the model obtained through the application of different optimization techniques.

Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Constant Pruning (80%), Node-Based Pruning with L1 regularization ($\lambda = 0.001$).

- Optimized version 5:

Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Depth-wise Separable Convolutions, Width Multiplier ($\alpha = 0.8$), Weights clustering (clusters = 16).

- Optimized version 6:

Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$), Weights clustering (clusters = 32).

- Optimized version 7:

Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Depth-wise Separable Convolutions, Resolution Multiplier ($\rho = 0.75$), Quantization: 8-bit integer, Base-2 log quantization, Post-Training Quantization, Weights only (advanced version).

- Optimized version 8:

Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped Convolutions ($G = 32$), Width Multiplier ($\alpha = 0.8$), Quantization: 16-bit minifloat, Quantization-Aware Training, Weights + Activation Quantization.

- Optimized version 9:

Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Quantization: 4-bit integer, Base-2 log quantization, Post-Training Quantization, Weights only (advanced version).

- Optimized version 10:

Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Depthwise Separable Convolutions, Weights clustering (clusters = 32).

- Optimized version 11:

Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$), Constant Pruning (40%), Magnitude-Based Pruning.

- Optimized version 12:

Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Spatially

separable convolutions (only applicable to some convolutional layers), Width Multiplier ($\alpha = 0.8$), Quantization: 8-bit integer, Dynamic fixed point quantization, Quantization-Aware Training, Weights + Activation Quantization.

- Optimized version 13:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Depthwise Separable Convolutions, Resolution Multiplier ($\rho = 0.75$), Constant Pruning (80%), Node-Based Pruning with L1 regularization ($\lambda = 0.001$).
- Optimized version 14:
Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped Convolutions ($G = 32$), Weights clustering (clusters = 16).
- Optimized version 15:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Quantization: Binary NNs ($w, x \in \{-\alpha, \alpha\}$, α computed according to average of tensors in Model10), Quantization-Aware Training, Weights + Activation Quantization.
- Optimized version 16:
Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Depthwise Separable Convolutions, Width Multiplier ($\alpha = 0.8$), Gradually Increasing Pruning (10%-60%), Block-Based Pruning (4x4).
- Optimized version 17:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$), Weights clustering (clusters = 16).
- Optimized version 18:
Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Depthwise Separable Convolutions, Quantization: 16-bit integer, Base-2 log quantization, Post-Training Quantization, Weights only (advanced version).
- Optimized version 19:
Batch size: 200, Convolutional layers, data reuse patterns: Weights Reuse, Grouped Convolutions ($G = 32$), Width Multiplier ($\alpha = 0.8$), Constant Pruning (60%), Node-Based Pruning with L1 regularization ($\lambda = 0.001$).
- Optimized version 20:
Batch size: 300, Convolutional layers, data reuse patterns: IFmap Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Width Multi-

plier ($\alpha = 0.8$), Quantization: 16-bit integer, Base-2 log quantization, Quantization-Aware Training, Weights + Activation Quantization.

- Optimized version 21:

Batch size: 300, Convolutional layers, data reuse patterns: Weights Reuse, Grouped Convolutions ($G = 64$), Resolution Multiplier ($\rho = 0.5$), Constant Pruning (20%), Weight Saliency-Based Pruning.

- Optimized version 22:

Batch size: 200, Convolutional layers, data reuse patterns: IFmap Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Width Multiplier ($\alpha = 0.95$), Quantization: 16-bit minifloat, Quantization-Aware Training, Weights only (advanced version).

- Optimized version 23:

Batch size: 300, Convolutional layers, data reuse patterns: Weights Reuse, Depth-wise Separable Convolutions, Gradually Increasing Pruning (10%-70%), Block-Based Pruning (4x4).

- Optimized version 24:

Batch size: 200, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped Convolutions ($G = 32$), Weights clustering (clusters = 64).

- Optimized version 25:

Batch size: 300, Convolutional layers, data reuse patterns: Weights Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Quantization: 8-bit integer, Base-2 log quantization, Post-Training Quantization, Weights only (advanced version).

- Optimized version 26:

Batch size: 200, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped Convolutions ($G = 64$), Width Multiplier ($\alpha = 0.7$), Constant Pruning (40%), Magnitude-Based Pruning.

- Optimized version 27:

Batch size: 300, Convolutional layers, data reuse patterns: Weights Reuse, Depth-wise Separable Convolutions, Quantization: Binary NNs ($w, x \in \{-\alpha, \alpha\}$, α computed according to average of tensors in Model10), Quantization-Aware Training, Weights + Activation Quantization.

- Optimized version 28:

Batch size: 200, Convolutional layers, data reuse patterns: IFmap Reuse, Grouped

Convolutions ($G = 64$), Weights clustering (clusters = 32).

- Optimized version 29:

Batch size: 300, Convolutional layers, data reuse patterns: Weights Reuse, Spatially separable convolutions (only applicable to some convolutional layers), Gradually Increasing Pruning (10%-80%), Node-Based Pruning with L1 regularization ($\lambda = 0.005$).

- Optimized version 30:

Batch size: 200, Convolutional layers, data reuse patterns: IFmap Reuse, Depthwise Separable Convolutions, Quantization: 4-bit integer, Dynamic fixed point quantization, Post-Training Quantization, Weights + Activation Quantization.

These models were evaluated by performing 1,000 inferences, each on a different image, simulating the number of potential windows in an image representing an entire PCB (object detection task).⁷

Table 7.1 provides an overview of the performance metrics for the Reference Model and the 30 optimized versions. The metrics include accuracy, latency, energy consumption, and memory size occupation⁸.

The variations in metrics reflect the theoretical considerations presented in Section 7.2 and show how the trade-off between accuracy and other metrics can be addressed. For instance, Binary Neural Networks (BNNs) show the best values in terms of latency, energy consumption, and memory usage, but with a noticeable drop in performance accuracy (more than 4% accuracy drop). On the other hand, models that exhibits the least reduction in accuracy (i.e., model based on 16-bit minifloat quantization, employing Quantization-Aware Training, Weights + Activation Quantization or based on the use of Grouped Convolutions, Resolution Multiplier, Constant Weight Saliency-Based Pruning), also demonstrate less significant improvements in terms of latency, energy consumption, and memory usage.

This highlights the balance required when choosing optimization strategies.

⁷The availability of 1,000 images allows for the use of batch sizes of 200 or 300, which is not always feasible in practice. In some cases, images may not be readily available in such quantities or they must be processed in real-time, making it impossible to wait for all the images required to form a complete batch.

⁸Model Size is computed on the compressed version of the trained model. In particular, the Deflate compression algorithm, which is a combination of LZ77 (a variant of the Ziv-Lempel algorithm) and Huffman coding, has been used.

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Table 7.1: Performance metrics for the Reference Model and the 30 Optimized Versions

8 | Conclusions and future developments

In this thesis, the potential of artificial intelligence, specifically machine learning (ML) and deep learning (DL), in automating the recognition of electronic components on printed circuit boards (PCBs) has been demonstrated. Through the development of a robust image recognition pipeline and the application of techniques such as domain randomization and optimization algorithms, models were created that not only achieved high accuracy in component detection but also met the requirements of a corporate production environment in terms of latency, memory usage, and energy efficiency.

The experimental results showed that deep learning architectures, particularly Convolutional Neural Networks (CNNs) and Residual Networks (ResNets), outperformed traditional methods like Support Vector Machines (SVMs) in handling the variability and complexity of PCB environments. Additionally, the use of adaptive domain randomization significantly enhanced the robustness of the models, reducing the need for frequent retraining under changing environmental conditions.

The integration of the developed models into the company's production pipeline demonstrated notable improvements in both efficiency and scalability, validating the feasibility of AI-driven approaches in industrial settings. These improvements are expected to reduce manual labor, decrease inspection times, and enhance product quality in large-scale electronic testing.

8.1. Future Developments

While this work addressed several challenges in PCB component recognition, there remain opportunities for further research and development. Future work could focus on the following areas.

Exploring Semantic Segmentation Techniques In addition to object detection and classification tasks, the integration of semantic segmentation techniques could significantly enhance system performance. Semantic segmentation not only identifies the location and class of objects but also provides pixel-level classification. This level of detail is particularly useful in electronic component recognition, where multiple components may be densely packed or overlap. By incorporating semantic segmentation, it would be possible to achieve finer-grained recognition of component boundaries and improve accuracy in detecting small or complex objects on PCBs.

Further Model Training with New PCB Data Another potential direction for improvement involves extending the model training with new datasets from diverse PCB designs and manufacturing environments. Continuously updating the dataset with fresh examples will enable the model to generalize better across different configurations and conditions. Additionally, the inclusion of more challenging cases, such as components under varying lighting conditions or partially obscured, will further improve the model's robustness and adaptability.

Analysis of Class Confusion Patterns Investigating class confusion patterns can provide valuable insights for refining the model. By identifying which true classes are most often confused with others, it is possible to pinpoint specific areas where the model struggles. Understanding confusion patterns could lead to targeted improvements, such as refining feature extraction methods or augmenting the dataset with more representative samples of similar classes. (See Section 6.2 for details.)

Bibliography

- [1] E. H. Adelson, C. H. Anderson, J. R. Bergen, P. J. Burt, and J. M. Ogden. Pyramid methods in image processing. *RCA engineer*, 29(6):33–41, 1984.
- [2] A. Auger and N. Hansen. Tutorial cma-es: evolution strategies and covariance matrix adaptation. In *Proceedings of the 14th annual conference companion on Genetic and evolutionary computation*, pages 827–848, 2012.
- [3] H. Bleeker, P. van Den Eijnden, and F. De Jong. *Boundary-scan test: a practical approach*, volume 1149. Springer Science & Business Media, 1992.
- [4] O. Chapelle, P. Haffner, and V. N. Vapnik. Support vector machines for histogram-based image classification. *IEEE transactions on Neural Networks*, 10(5):1055–1064, 1999.
- [5] I.-C. Chen, R.-C. Hwang, and H.-C. Huang. Pcb defect detection based on deep learning algorithm. *Processes*, 11(3):775, 2023. doi: 10.3390/pr11030775.
- [6] Y. Choi, M. El-Khamy, and J. Lee. Towards the limit of network quantization. *arXiv preprint arXiv:1612.01543*, 2016.
- [7] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer. A survey of quantization methods for efficient neural network inference. In *Low-Power Computer Vision*, pages 291–326. Chapman and Hall/CRC, 2022.
- [8] I. Goodfellow. Deep learning, 2016.
- [9] J. Guo, Y. Li, W. Lin, Y. Chen, and J. Li. Network decoupling: From regular to depthwise separable convolutions. *arXiv preprint arXiv:1808.05517*, 2018.
- [10] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan. Deep learning with limited numerical precision. In *International conference on machine learning*, pages 1737–1746. PMLR, 2015.
- [11] S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural

- networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*, 2015.
- [12] S. Han, J. Pool, J. Tran, and W. Dally. Learning both weights and connections for efficient neural network. *Advances in neural information processing systems*, 28, 2015.
- [13] A. G. Howard. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- [14] M. Huh, P. Agrawal, and A. A. Efros. What makes imagenet good for transfer learning? *arXiv preprint arXiv:1608.08614*, 2016.
- [15] C. D. Johnson. *Process Control Instrumentation Technology*. Prentice Hall, 8th edition edition, 2003.
- [16] U. Kallakuri, E. Humes, and T. Mohsenin. Resource-aware saliency-guided differentiable pruning for deep neural networks. In *Proceedings of the Great Lakes Symposium on VLSI 2024*, pages 694–699, 2024.
- [17] R. Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018.
- [18] S. Kumar, S. Datta, V. Singh, S. K. Singh, and R. Sharma. Opportunities and challenges in data-centric ai. *IEEE Access*, 12:33173–33189, 2024.
- [19] E. H. Lee, D. Miyashita, E. Chai, B. Murmann, and S. S. Wong. Lognet: Energy-efficient neural networks using logarithmic computation. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5900–5904. IEEE, 2017.
- [20] J. Liang. Image classification based on resnet. In *Journal of Physics: Conference Series*, volume 1634, page 012110. IOP Publishing, 2020.
- [21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft coco: Common objects in context. In D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, editors, *Computer Vision – ECCV 2014*, pages 740–755, Cham, 2014. Springer International Publishing.
- [22] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, et al. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

- [23] T. M. Mitchell and T. M. Mitchell. *Machine learning*, volume 1. McGraw-hill New York, 1997.
- [24] B. Moons, B. De Brabandere, L. Van Gool, and M. Verhelst. Energy-efficient convnets through approximate comp. In *Citation Bert Moons, Bert De Brabandere, Luc Van Gool, Marian Verhelst,(2016) Energy-Efficient ConvNets Trough Approximate Computing IEEE Winter Conference on Applications of Computer Vision, Lake Placid, NY, USA*, pages 7–9, 2016.
- [25] J.-H. Park, Y.-S. Kim, and H. Seo. Analysis of training deep learning models for pcb defect detection. *Sensors*, 23(5):2766, 2023. doi: 10.3390/s23052766.
- [26] F. Perdigones and J. M. Quero. Printed circuit boards: The layers’ functions for electronic and biomedical engineering. *Micromachines*, 13(3), 2022. URL <https://www.mdpi.com/2072-666X/13/3/460>.
- [27] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European conference on computer vision*, pages 525–542. Springer, 2016.
- [28] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017.
- [29] E. M. Taha, E. Emary, and K. Moustafa. Automatic optical inspection for pcb manufacturing: a survey. *International Journal of Scientific and Engineering Research*, 5(7):1095–1102, 2014.
- [30] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Boochoon, and S. Birchfield. Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 969–977, 2018.
- [31] F. Tung and G. Mori. Clip-q: Deep network compression learning by in-parallel pruning-quantization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7873–7882, 2018.
- [32] A. Tzotsos and D. Argialas. Support vector machine classification for object-based image analysis. *Object-based image analysis: Spatial concepts for knowledge-driven remote sensing applications*, pages 663–677, 2008.
- [33] D. Wang, D. Tan, and L. Liu. Particle swarm optimization algorithm: an overview. *Soft computing*, 22(2):387–408, 2018.

- [34] S. Williams, D. Patterson, L. Oliker, J. Shalf, and K. Yelick. The roofline model: A pedagogical tool for program analysis and optimization. In *2008 IEEE hot chips 20 symposium (HCS)*, pages 1–71. IEEE, 2008.
- [35] J. Wu. Introduction to convolutional neural networks. *National Key Lab for Novel Software Technology. Nanjing University. China*, 5(23):495, 2017.
- [36] T.-J. Yang, Y.-H. Chen, and V. Sze. Designing energy-efficient convolutional neural networks using energy-aware pruning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5687–5695, 2017.
- [37] E. Yazan and M. F. Talu. Comparison of the stochastic gradient descent based optimization techniques. In *2017 International Artificial Intelligence and Data Processing Symposium (IDAP)*, pages 1–5. IEEE, 2017.
- [38] J. Yu, A. Lukefahr, D. Palframan, G. Dasika, R. Das, and S. Mahlke. Scalpel: Customizing dnn pruning to the underlying hardware parallelism. *ACM SIGARCH Computer Architecture News*, 45(2):548–560, 2017.
- [39] Z. B. Zabinsky et al. Random search algorithms. *Department of Industrial and Systems Engineering, University of Washington, USA*, 2009.
- [40] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, and X. Hu. Data-centric artificial intelligence: A survey, 2023. URL <https://arxiv.org/abs/2303.10158>.
- [41] L. Zhang and J. Wu. End-to-end deep learning framework for printed circuit board manufacturing defect classification. *Scientific Reports*, 12(12559), 2022. doi: 10.1038/s41598-022-16302-3.
- [42] X. Zhang, X. Zhou, M. Lin, and J. Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6848–6856, 2018.

A | Parameter Distributions for Domain Randomization

With respect to the distributions shown in Subsection 4.1.2.1, the following distributions are used.

A.1. Univariate Uniform distributions

$$\mathcal{U}(-0.2, 0.2); \quad \mathcal{U}(0, 2.5); \quad \mathcal{U}(0, 1); \quad \mathcal{U}(0, 1); \quad \mathcal{U}(-0.2, 0.2); \quad \mathcal{U}(20, 100); \quad \mathcal{U}(0, 2)$$

A.2. Univariate Triangular distribution

$$\begin{aligned} &\mathcal{T}(-0.2, 0, 0.2); \quad \mathcal{T}(0, 1.25, 2.5); \quad \mathcal{T}(0, 0.5, 1); \\ &\mathcal{T}(0, 0.5, 1); \quad \mathcal{T}(-0.2, 0, 0.2); \quad \mathcal{T}(20, 60, 100); \quad \mathcal{T}(0, 1, 2) \end{aligned}$$

A.3. Univariate Normal distribution

$$\begin{aligned} &\mathcal{N}(0, 0.15); \quad \mathcal{N}(1.25, 1); \quad \mathcal{N}(0.5, 0.1); \\ &\mathcal{N}(0.5, 0.1); \quad \mathcal{N}(0, 0.15); \quad \mathcal{N}(60, 25); \quad \mathcal{N}(1.25, 1.125) \end{aligned}$$

A.4. Multivariate Normal distribution

$$\mathcal{N}_7 \left(\begin{bmatrix} 0 \\ 1.25 \\ 0.5 \\ 0.5 \\ 0 \\ 60 \\ 1.25 \end{bmatrix}, \begin{bmatrix} 0.15 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.15 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 25 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1.125 \end{bmatrix} \right)$$

B | Probability vector $\mathbf{p}$ for (Adaptive) Domain Randomization

$\mathbf{p}_i$: probability that the i -th parameter is randomized (See Algorithm 4.1)

$$p = \begin{bmatrix} 0.9 \\ 0.9 \\ 0.9 \\ 0.9 \\ 0.9 \\ 0.9 \\ 0.9 \end{bmatrix}$$

C | Parameter Distributions for Adaptive Domain Randomization

With respect to the distributions shown in Subsection 4.1.2.1, the following ranges and initial values are used to define the search space of the coefficient distributions to be optimized.

C.1. Univariate Uniform distributions

C.1.1. $\mathcal{U}(a, b)$: Lower Ranges, Upper Ranges, and Initial Values

Lower Ranges:

- $\mathcal{U}(a, b)$, lower range for $a = -1$, lower range for $b = -1$;
- $\mathcal{U}(a, b)$, lower range for $a = -\infty$, lower range for $b = -\infty$;
- $\mathcal{U}(a, b)$, lower range for $a = -\infty$, lower range for $b = -\infty$;
- $\mathcal{U}(a, b)$, lower range for $a = -\infty$, lower range for $b = -\infty$;
- $\mathcal{U}(a, b)$, lower range for $a = -1$, lower range for $b = -1$;
- $\mathcal{U}(a, b)$, lower range for $a = 0$, lower range for $b = 0$;
- $\mathcal{U}(a, b)$, lower range for $a = -0$, lower range for $b = 0$.

Upper Ranges:

- $\mathcal{U}(a, b)$, upper range for $a = 1$, upper range for $b = 1$;
- $\mathcal{U}(a, b)$, upper range for $a = \infty$, upper range for $b = \infty$;
- $\mathcal{U}(a, b)$, upper range for $a = \infty$, upper range for $b = \infty$;
- $\mathcal{U}(a, b)$, upper range for $a = \infty$, upper range for $b = \infty$;
- $\mathcal{U}(a, b)$, upper range for $a = 1$, upper range for $b = 1$;
- $\mathcal{U}(a, b)$, upper range for $a = 100$, upper range for $b = 100$;
- $\mathcal{U}(a, b)$, upper range for $a = \infty$, upper range for $b = \infty$.

Initial Values:

- $\mathcal{U}(a, b)$, initial value for $a = 0$, initial value for $b = 0$;
- $\mathcal{U}(a, b)$, initial value for $a = 0$, initial value for $b = 0$;
- $\mathcal{U}(a, b)$, initial value for $a = 0$, initial value for $b = 0$;
- $\mathcal{U}(a, b)$, initial value for $a = 0$, initial value for $b = 0$;
- $\mathcal{U}(a, b)$, initial value for $a = 0$, initial value for $b = 0$;
- $\mathcal{U}(a, b)$, initial value for $a = 50$, initial value for $b = 50$;
- $\mathcal{U}(a, b)$, initial value for $a = 1.25$, initial value for $b = 1.25$.

C.2. Univariate Triangular distributions

C.2.1. $\mathcal{T}(a, m, b)$: Lower Ranges, Upper Ranges, and Initial Values

Lower Ranges:

- $\mathcal{T}(a, m, b)$, lower range for $a = -1$, lower range for $m = -1$, lower range for $b = -1$;
- $\mathcal{T}(a, m, b)$, lower range for $a = -\infty$, lower range for $m = -\infty$, lower range for $b = -\infty$;
- $\mathcal{T}(a, m, b)$, lower range for $a = -\infty$, lower range for $m = -\infty$, lower range for $b = -\infty$;
- $\mathcal{T}(a, m, b)$, lower range for $a = -\infty$, lower range for $m = -\infty$, lower range for $b = -\infty$;
- $\mathcal{T}(a, m, b)$, lower range for $a = -1$, lower range for $m = -1$, lower range for $b = -1$;
- $\mathcal{T}(a, m, b)$, lower range for $a = 0$, lower range for $m = 0$, lower range for $b = 0$;
- $\mathcal{T}(a, m, b)$, lower range for $a = -0$, lower range for $m = -0$, lower range for $b = 0$.

Upper Ranges:

- $\mathcal{T}(a, m, b)$, upper range for $a = 1$, upper range for $m = 1$, upper range for $b = 1$;
- $\mathcal{T}(a, m, b)$, upper range for $a = \infty$, upper range for $m = \infty$, upper range for $b = \infty$;
- $\mathcal{T}(a, m, b)$, upper range for $a = \infty$, upper range for $m = \infty$, upper range for $b = \infty$;
- $\mathcal{T}(a, m, b)$, upper range for $a = \infty$, upper range for $m = \infty$, upper range for $b = \infty$;
- $\mathcal{T}(a, m, b)$, upper range for $a = 1$, upper range for $m = 1$, upper range for $b = 1$;
- $\mathcal{T}(a, m, b)$, upper range for $a = 100$, upper range for $m = 100$, upper range for $b = 100$;
- $\mathcal{T}(a, m, b)$, upper range for $a = \infty$, upper range for $m = \infty$, upper range for $b = \infty$.

Initial Values:

- $\mathcal{T}(a, m, b)$, initial value for $a = 0$, initial value for $m = 0$, initial value for $b = 0$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 0$, initial value for $m = 0$, initial value for $b = 0$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 0$, initial value for $m = 0$, initial value for $b = 0$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 0$, initial value for $m = 0$, initial value for $b = 0$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 0$, initial value for $m = 0$, initial value for $b = 0$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 50$, initial value for $m = 50$, initial value for $b = 50$;
- $\mathcal{T}(a, m, b)$, initial value for $a = 1.25$, initial value for $m = 1.25$, initial value for $b = 1.25$.

C.3. Univariate Normal Distribution

C.3.1. $\mathcal{N}(\mu, \sigma^2)$: Lower Ranges, Upper Ranges, and Initial Values

Lower Ranges:

- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = -1$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = -\infty$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = -\infty$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = -\infty$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = -1$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = 0$, lower range for $\sigma^2 = 0$;
- $\mathcal{N}(\mu, \sigma^2)$, lower range for $\mu = 0$, lower range for $\sigma^2 = 0$.

Upper Ranges:

- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = 1$, upper range for $\sigma^2 = 0.4$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = \infty$, upper range for $\sigma^2 = 10$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = \infty$, upper range for $\sigma^2 = 10$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = \infty$, upper range for $\sigma^2 = 10$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = 1$, upper range for $\sigma^2 = 0.4$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = 100$, upper range for $\sigma^2 = 25$;
- $\mathcal{N}(\mu, \sigma^2)$, upper range for $\mu = \infty$, upper range for $\sigma^2 = 10$.

Initial Values:

- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 0$, initial value for $\sigma^2 = 0.2$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 0$, initial value for $\sigma^2 = 5$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 0$, initial value for $\sigma^2 = 5$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 0$, initial value for $\sigma^2 = 5$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 0$, initial value for $\sigma^2 = 0.2$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 50$, initial value for $\sigma^2 = 12.5$;
- $\mathcal{N}(\mu, \sigma^2)$, initial value for $\mu = 1.25$, initial value for $\sigma^2 = 5$.

C.4. Multivariate Normal Distribution

C.4.1. $\mathcal{N}_k(\mu, \Sigma)$: Lower Ranges, Upper Ranges, and Initial Values

$$\mathcal{N}_7 \left(\begin{bmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \\ \mu_4 \\ \mu_5 \\ \mu_6 \\ \mu_7 \end{bmatrix}, \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} & \sigma_{14} & \sigma_{15} & \sigma_{16} & \sigma_{17} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} & \sigma_{24} & \sigma_{25} & \sigma_{26} & \sigma_{27} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 & \sigma_{34} & \sigma_{35} & \sigma_{36} & \sigma_{37} \\ \sigma_{41} & \sigma_{42} & \sigma_{43} & \sigma_4^2 & \sigma_{45} & \sigma_{46} & \sigma_{47} \\ \sigma_{51} & \sigma_{52} & \sigma_{53} & \sigma_{54} & \sigma_5^2 & \sigma_{56} & \sigma_{57} \\ \sigma_{61} & \sigma_{62} & \sigma_{63} & \sigma_{64} & \sigma_{65} & \sigma_6^2 & \sigma_{67} \\ \sigma_{71} & \sigma_{72} & \sigma_{73} & \sigma_{74} & \sigma_{75} & \sigma_{76} & \sigma_7^2 \end{bmatrix} \right),$$

Lower Ranges:

lower range for $\mu_1 = -1, \quad \mu_2 = -\infty, \quad \mu_3 = -\infty, \quad \mu_4 = -\infty, \quad \mu_5 = -1, \quad \mu_6 = 0, \quad \mu_7 = 0;$

lower range for $\sigma_i^2 = 0, \quad i \in \{1, 2, 3, 4, 5, 6, 7\};$

lower range for $\sigma_{ij} = -100, \quad i, j \in \{1, 2, 3, 4, 5, 6, 7\}, \quad i \neq j.$

Upper Ranges:

upper range for $\mu_1 = 1, \quad \mu_2 = \infty, \quad \mu_3 = \infty, \quad \mu_4 = \infty, \quad \mu_5 = 1, \quad \mu_6 = 100, \quad \mu_7 = \infty;$

upper range for $\sigma_i^2 = 0.4, \quad i \in \{1, 5\};$

upper range for $\sigma_i^2 = 10, \quad i \in \{2, 3, 4, 7\};$

upper range for $\sigma_i^2 = 25, \quad i = 6;$

upper range for $\sigma_{ij} = 100, \quad i, j \in \{1, 2, 3, 4, 5, 6, 7\}, \quad i \neq j.$

Initial Values:

initial value for $\mu_1 = 0, \quad \mu_2 = 0, \quad \mu_3 = 0, \quad \mu_4 = 0, \quad \mu_5 = 0, \quad \mu_6 = 1.25, \quad \mu_7 = 50;$

initial value for $\sigma_i^2 = 0.2, \quad i \in \{1, 5\};$

initial value for $\sigma_i^2 = 5, \quad i \in \{2, 3, 4, 7\};$

initial value for $\sigma_i^2 = 12.5, \quad i = 6;$

initial value for $\sigma_{ij} = 0$, $i, j \in \{1, 2, 3, 4, 5, 6, 7\}$, $i \neq j$.

List of Figures

- 1.1 High resolution photo of a PCB used in Electronic Testing. 11
- 1.2 Component recognition on a PCB. 17
- 1.3 IC recognition on a PCB. 18
- 1.4 Tantalum capacitor recognition on a PCB. 19
- 1.5 Ceramic capacitor recognition on a PCB. 20

- 3.1 Pipeline of the Machine Learning Workflow 40
- 3.2 Pipeline of Data Preprocessing Workflow 42

- 5.1 Pipeline from Trained Model to Inference for Image Classification and Ob-
ject Detection 59

- 6.1 Comparison of model accuracy on the training and test sets. 70
- 6.2 Log-scaled confusion matrix for Model 10. The values have been log-
transformed using the natural logarithm of $1 + x$, where x is the original
value in the matrix (1 has been added to avoid computing $\log 0$, which is
not defined) to better highlight differences between predictions, especially
when the accuracy, precision, and recall are very high. 72
- 6.3 Normal distributions corresponding to the second component of the multi-
variate normal distribution (*Contrast* parameter) during the optimization
process, highlighting how its coefficients μ and Σ vary over epochs. 75

- 7.1 Diagram illustrating the two main quantization approaches. 103

List of Tables

6.1 Confusion Matrix for Multiclass Classification 69

6.2 Performance metrics (accuracy, precision, recall, and F1-score) expressed
as percentages for the training and test sets across different models. 73

6.3 Accuracy for models with different distributions used for parameter ran-
domization. 74

7.1 Performance metrics for the Reference Model and the 30 Optimized Versions113

Acknowledgements

I am sincerely grateful to my supervisor, Prof. Daniele Apiletti, for his constant availability, guidance, and support throughout the course of this thesis, as well as during the courses he taught, both during my bachelor's and master's studies.

I would also like to express my gratitude to Ing. Anna Grassino, R&D Director at Seica S.p.A., for the incredible opportunity she offered me, as well as to all my colleagues in the Software team at the company. In particular, I would like to thank Paolo Chiartano, Ing. Francesco Grassino, Luca Covolo and Marco Marigo, who have closely followed and supported me throughout the course of my project.

A special thanks goes to my family, particularly my parents and my sister Silvia, who have always supported me throughout my studies, offering encouragement at every step of the way.

